# Gene Regulatory Network-Guided Causal Generative Modeling of Single-Cell Transcriptomics

A thesis submitted to McGill University in partial fulfillment of the requirements for the degree of Master of Science.

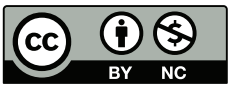
*Yazdan Zinati*

January 2026

Faculty of Engineering
Department of Electrical & Computer Engineering
McGill University, Montréal

Supervisor: Amin Emad

© 2026 Yazdan Zinati



Gene Regulatory Network-Guided Causal Generative Modeling of Single-Cell Transcriptomics
© 2026 by Yazdan Zinati is licensed under CC BY-NC 4.0. To view a copy of this license, visit
https://creativecommons.org/licenses/by-nc/4.0/

No part of this thesis was created using generative artificial intelligence.

# Abstract

# Résumé

"You don't see yet, Genry, why we perfected and practice Foretelling?"

"No—"

"To exhibit the perfect uselessness of knowing the answer to the wrong question."

— Ursula K. Le Guin, *The Left Hand of Darkness* (1969)

To my parents, this thesis is dedicated to you for being my support through it all.

# Acknowledgments

My sincere thanks go to Prof. Amin Emad for welcoming me into computational biology and guiding me through every step of this research journey.

Endless conversations, thoughtful feedback, and well-timed jokes from the members of the COMBINE lab kept this thesis on course and my sanity intact; their presence is felt in each chapter. I have been fortunate to benefit from Abdulrahman Takiddeen's mentorship and Safyan Aman Memon's company during late lab sessions.

The work described in this thesis was made possible thanks to the support by the Natural Sciences and Engineering Research Council of Canada (NSERC), the Government of Canada's New Frontiers in Research Fund (NFRF), the Canada Foundation for Innovation (CFI), the McGill Initiative in Computational Medicine, Fonds de recherche du Québec, the Digital Research Alliance of Canada, and Calcul Québec.

# Contents

**Abstract** ii

**Résumé** iii

**Acknowledgments** v

**1 Introduction** 5

Motivation . . . . . 5

Thesis Organization & Contributions . . . . . 6

Publications & Author Contributions . . . . . 8

**2 Background Material and Literature Review** 9

The Biological Basis of Protein-Protein Interactions . . . . . 9

Biological Processes, Pathways, and Networks . . . . . 9

Physiochemical Fundamentals of Protein Interactions . . . . . 11

Protein Interactions and Pathologies . . . . . 12

Experimental Approaches to Discovering Protein-Protein Interactions . . . . . 13

Yeast Two-Hybrid and Related Methods . . . . . 14

Affinity-based Methods . . . . . 14

Energy Transfer Methods . . . . . 15

Other methods . . . . . 15

Computational & Statistical Models of Protein-Protein Interactions . . . . . 16
Non-Machine Learning Methods for PPI Inference . . . . . 16
Machine Learning Methods for PPI Inference . . . . . 19
Deep Learning Methods for PPI Inference . . . . . 22
The Issue of Generalization in PPI Inference Models . . . . . 28

**3 GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks 32**

Abstract . . . . . 32
Introduction . . . . . 32
Results . . . . . 34
GRouNdGAN generates scRNA-seq data using causal generative adversarial networks . . . . . 34
GRouNdGAN generates realistic scRNA-seq data . . . . . 35
GRouNdGAN imposes a causal GRN in the simulated data . . . . . 39
The imposed GRN can be reconstructed from the simulated data . . . . . 41
GRouNdGAN-simulated data preserves trajectories and can be used for pseudo-time inference . . . . . 43
GRouNdGAN achieves high performance on other datasets . . . . . 46
Benchmarking GRN inference methods using GRouNdGAN confirms prior insights from curated and experimental datasets . . . . . 47
In silico perturbation experiments using GRouNdGAN . . . . . 49
Discussion . . . . . 50
Methods . . . . . 54
Datasets and preprocessing . . . . . 54
GRouNdGAN's model architecture . . . . . 55
Training procedure and hyperparameter tuning . . . . . 58

Causal GRN preparation . . . 59
Evaluation of the resemblance of real and simulated cells . . . 60
Baseline simulator models . . . 61
In silico perturbation experiments using GRoundGAN . . . 62
Trajectory inference and pseudo-time analysis . . . 62
GRN inference methods . . . 63
Statistics and reproducibility . . . 64
Reporting summary . . . 64
Data availability . . . 64
Code availability . . . 64
References . . . 65
Acknowledgements . . . 69
Author contributions . . . 69
Competing interests . . . 70
Additional information . . . 70

**4 Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit 73**

Abstract . . . 73
Introduction and Background . . . 74
Development of the protocol . . . 75
Applications of the method . . . 76
Comparison with other methods . . . 77
Experimental design . . . 80
Expertise needed to implement the protocol . . . 84
Discussion and Limitations . . . 84

Materials and Methods . . . 86
Equipment . . . 86
Equipment setup . . . 88
Procedure . . . 92
Step 1. Preprocessing . . . 94
Step 2. GRN Creation . . . 95
Step 3. Model Training . . . 98
Step 4. Simulation . . . 101
Step 5. Assessing simulated data quality (Optional) . . . 103
Step 6. GRN inference benchmarking (Optional) . . . 105
Step 7. Perturbation prediction (Optional) . . . 107
Timing . . . 109
General notes and troubleshooting . . . 109
Expected results . . . 110
Supplementary Information . . . 112
Authors' contributions . . . 112
Funding . . . 112
Data Availability . . . 112
Code Availability . . . 113
Competing interests . . . 113
References . . . 113

**5 Discussion** **114**

Appropriate regularization methods . . . 114
Regularization in RAPPPID . . . 114
Regularization in INTREPPPID . . . 115

Questions regarding sequence identity and its role in data leakage . . . . . 116
Distal impacts of efficient architectures . . . . . 120
As a means to improve generalization . . . . . 120
Lowering barriers to PPI inference research . . . . . 121
Environmental impacts . . . . . 122

**6 Conclusion** **125**

**A Supplementary Material for "GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks"** **127**

**B Supplementary Material for "Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit"** **133**

# List of Figures

**Background Material and Literature Review** **9**

2.1 A high-level schematic of the human pathways in the Reactome knowledgebase 10

2.2 A historical timeline of PPI inference methodologies. . . . . 17

2.3 Number of parameters in language models over time. . . . . 27

**GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks** **32**

1 Architecture and training procedure of GRouNdGAN. . . . . 36

2 Performance of GRouNdGAN in generating realistic scRNA-seq data. . . . 38

3 Performance of GRNBoost2 in recovering the imposed edges versus unimposed edges (PBMC-CTL). . . . . 42

4 PAGA graphs and cell type marker gene activation patterns in experimental and simulated BoneMarrow dataset. . . . . 44

5 Consistency in trajectories and pseudo-temporal orderings between data generated by GRouNdGAN and the experimental BoneMarrow hematopoietic dataset. . . . . 45

6 TF knockout experiments and their effect on cell type generation. . . . . 48

**Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit** **73**

1 Schematic overview of the GRouNdGAN pipeline. . . . . 78

2 Cell type-specific simulation of cells using GRouNdGAN. . . . . 82

3 Robustness analysis of simulated data. . . . . 85

4 TensorBoard visualizations during GRouNdGAN training. . . . . 102

5 Example of expected plot outputs saved by the evaluation and benchmarking commands. . . . . 108

6 Example of expected plot outputs saved by the perturbation command. . . . 110

**Supplementary Material for "GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks" 127**

S1 Relative Degree of Proteins in the *H. sapiens* STRING C3 dataset. . . . . 128

S2 Accuracy of positive edges across edge confidence stratified by STRING channels. 129

S3 Accuracy of positive edges across edge confidence stratified by STRING channels. 130

S4 Accuracy of positive edges as a function of similarity between testing and training proteins in C3. . . . . 131

S5 Distribution of Interaction Probabilities of Trastuzumab and Pertuzumab with all Proteins of the Human Proteome as Predicted by RAPPPID. . . . . 132

**Supplementary Material for "Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit" 133**

S1 Precision-Recall curves of various PPI inference methods . . . . . 145

S2 Receiver-Operator curves of various PPI inference methods . . . . . 146

S3 The effect of orthologous locality task. . . . . 147

# List of Tables

**GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks** **32**

1 Performance of different simulators in generating realistic scRNA-seq data using the PBMC-CTL dataset . . . . . 37

2 Performance of different simulators in generating realistic scRNA-seq data using the PBMC-All dataset . . . . . 40

3 Performance of GRouNdGAN in generating realistic scRNA-seq data using the Dahlin, Tumor-All, and Tumor-malignant datasets . . . . . 46

**Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit** **73**

1 Performance of GRouNdGAN in generating realistic scRNA-seq data using cell-type specific GRNs using the PBMC dataset. . . . . 83

2 Arguments relevant to the preprocessing command in the configuration file. . 94

3 Arguments relevant to the GRN creation command in the configuration file. 96

4 Arguments relevant to the training command in the configuration file. . . . . 99

5 Arguments relevant to the generation command in the configuration file. . . 103

6 Arguments relevant to the evaluation command in the configuration file. . . . 104

7 Arguments relevant to the benchmarking command in the configuration file. 106

8 Arguments relevant to the perturbation command in the configuration file. . 107

9 Troubleshooting table. . . . . 111

**Supplementary Material for "GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks"** **127**

S1 Ranges of hyperparameters used for training of RAPPPID. . . . . 127

S2 The final values of hyperparameters selected. . . . . 127

S3 Statistics for the STRING C1, C2, and C3 datasets. . . . . 128

S4 Repeatability of results across randomly generated C3 datasets. . . . . 128

**Supplementary Material for "Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit"** **133**

S2 Hyperparameters used to train INTREPPPID in all experiments. . . . . 137

S4 Performance of INTREPPPID tested on increasing negative-to-positive ratios. . 139

S6 Performance of INTREPPPID with scrambled and not scrambled orthologue information. . . . . 141

S7 Change in INTREPPPID performance when omitting orthology data. . . . . 142

S9 Analysis of online availability of PPI-related tools. . . . . 144

# List of Abbreviations

| | |
|---|---|
| **AE** | Autoencoder |
| **AEP** | Affinity evaluation and prediction |
| **AI** | Artificial intelligence |
| **ANN** | Artificial neural network |
| **AP** | Average precision |
| **API** | Application programming interface |
| **AP-MS** | Affinity purification, mass spectroscopy |
| **ASA** | Accessible surface area |
| **ASGD** | Averaged stochastic gradient descent |
| **ASL** | Asymmetric loss |
| **ATP** | Adenosine triphosphate |
| **AUPR** | Area under the precision-recall curve |
| **AWD-LSTM** | ASGD Weight-Dropped LSTM |
| **AUROC** | Area under the receiver operating characteristic |
| **BERT** | Bidirectional encoder representations from transformers |
| **BFDR** | Bayesian false-discover rate |
| **BiFC** | Bimolecular fluorescence complementation |
| **BioGRID** | Biological general repository for interaction datasets |
| **BioID-MS** | Proximity-dependent biotin identification, mass spectrometry |
| **BLAST** | Basic local alignment search tool |
| **BLOSUM** | Blocks substitution matrix |

| | |
|---|---|
| **BRET** | Bioluminescence resonance energy transfer |
| **CD-HIT** | Cluster database at high identity with tolerance |
| **CLI** | Command-line interface |
| **CNN** | Convolutional neural networks |
| **coFrac-MS, CF-MS** | Co-fractionation, mass spectrometry |
| **CO-IP** | Co-immunopercipitation |
| **CPU** | Central processing unit |
| **DIP** | Database of interacting proteins |
| **DNA** | Deoxyribonucleic acid |
| **DNN** | Deep neural network |
| **D-SCRIPT** | Deep sequence contact residue interaction prediction transfer |
| **ESM** | Evolutionary scale model |
| **FLOP** | Floating point operation |
| **FRET** | Förster resonance energy transfer, or fluorescence resonance energy transfer |
| **GELU** | Gaussian error linear unit |
| **GO** | Gene ontology |
| **GPT** | Generative pre-trained transformer |
| **GPU** | Graphics processing unit |
| **GRU** | Gated recurrent unit |
| **HER2** | Human epidermal growth factor receptor 2 |
| **HIPPIE** | Human integrated protein–protein interaction reference |
| **HSSP** | Homology-derived secondary structure of proteins |
| **HuRI** | Human reference interactome |
| **INTREPPPID** | Incorporating triplet error for predicting PPIs using deep learning |
| **IMEX** | International molecular exchange consortium |
| **JSD** | Just sustainability design |
| **KEGG** | Kyoto encyclopedia of genes and genomes |
| **KISS** | Kinase substrate sensor |

| | |
|---|---|
| *k*NN | *k*-nearest neighbours |
| LLM | Large language model |
| LSTM | Long short-term memory |
| MaMTH | Mammalian-membrane two-hybrid |
| MAPPIT | Mammalian protein-protein interaction trap |
| MCC | Matthew's correlation coefficient |
| MIST | Molecular interaction search tool |
| MLM | Masked langauge model |
| MMseqs | Many-against-Many sequence searching |
| MoE | Mixture of experts |
| MP-BERT | MindSpore ProteinBERT |
| MSRP | Manufacturer's suggested retail price |
| MYTH | Membrane yeast two-hybrid |
| NCBI | National center for biotechnology information (USA) |
| NDCG | Normalized discounted cumulative gain |
| NHK | Nippon Hōsō Kyōkai |
| NLP | Natural language processing |
| NT-ASGD | Non-monotonically triggered averaged stochastic gradient descent |
| OMA | Orthologous matrix |
| OOD | Out of distribution |
| PAM | Point accepted mutation |
| PCA | Principle component analysis |
| PDB | Protein data bank |
| PEPPI | Pipeline for the extraction of predicted protein-protein interactions |
| PFC | Position-wise fully-connected |
| PIPE | Protein-protein interaction prediction engine |
| PIPR | Protein–Protein Interaction Prediction Based on Siamese Residual RCNN |
| pLLM, pLM | Protein (large) language model |
| PPI | Protein-protien interaction |

| | |
|---|---|
| **Prose** | Protein sequence embeddings |
| **psi-blast** | Position-specific iterated blast |
| **rapppid** | Regularized automatic prediction of protein–protein interactions |
| **rcnn** | Recurrent convolutional neural network |
| **rna** | Ribonucleic acid |
| **rnn** | Recurrent neural network |
| **sae** | Stacked autoencoder |
| **saint** | Significance analysis of interactome |
| **sd** | Standard deviation |
| **sds-page** | Sodium dodecyl sulfate–polyacrylamide gel electrophoresis |
| **sgd** | Stochastic gradient descent |
| **SimCLR** | Simple framework for contrastive learning of visual representations |
| **skempi** | Structural database of kinetics and energetics of mutant protein interactions |
| **snc** | Skill-normalized concordance |
| **sota** | Simple framework for contrastive learning of visual representations |
| **sprint** | Scoring protein interactions |
| **ssl** | Self-supervised learning |
| **string** | Search tool for recurring instances of neighbouring genes |
| **svm** | Support vector machine |
| **swa** | Stochastic weight averaging |
| **tdp** | Thermal design power |
| **tpca** | Thermal proximity co-aggregation |
| **tuna** | Transformer-based uncertainty aware model for ppi prediction |
| **umap** | Uniform manifold approximation and projection |
| **usd** | United States dollar |
| **vram** | Video random-access memory |
| **y2h** | Yeast two-hybrid |
| **xl-ms** | Cross-linking, mass spectrometry |

# Chapter 1

# Introduction

## Motivation

Of all the macromolecules one might find in the cells of living organisms, proteins are the class which are overwhelmingly responsible for the execution of the cell's functions, operating within and beyond its borders. Proteins are at the heart of vital cell functions like metabolism, DNA repair, cell polarity, and scaffolding.

To execute these myriad functions, proteins often participate in sometimes complex chains of chemical and physicochemical reactions with other proteins more broadly referred to as "protein-protein interactions" or PPIs. Understanding which proteins interact with other proteins is vital to our knowledge of *how* cells carry-out their many functions as well as the ways in which they fail in the case of disease.

In fact, cancers and other diseases which are caused by protein mutations often result in a change in protein binding affinity, altering how those proteins interact with others [1]. Infection and diseases caused by viruses like HIV and SARS-CoV-2 are also carried out by interactions between host and viral proteins [2, 3]. What's more, treatments for these diseases sometimes come in the form of so-called "biologic" therapies like small peptides, antibiotics, or mRNA vaccines, which in turn carry out their functions through the same fundamental processes as PPIs do.

While the study of PPIs has immeasurably shaped our understanding of biology, the methods by which we've traditionally come to characterize them require immense investments of time, labour, and capital. These methods, which include yeast two-hybrid (Y2H), affinity chromatography, and mass spectroscopy, are primarily conducted at the bench with the aid of specialized material, equipment, and expertise [4, 5, 6].

Consuming a fraction of the labour and capital resources, computational methods for identi-

fying putative PPIs are a compelling methodology for complimenting the challenges posed by traditional experimental methods. A single Y2H experiment, for instance, is conducted over multiple days [7, 8]. Contrastingly, the interactions between a given protein and the entire human proteome can be conducted in approximately three minutes using the INTREPPPID method through the online PPI.bio portal [9]. This is to make no mention of the discrepancies in capital investment between these two examples, with Y2H requiring expensive specialist equipment and materials, while the RAPPPID & INTREPPPID models runs on commodity computer hardware [10, 9].

Building upon decades of painstaking research, sizable databases of PPIs have been compiled across a growing number of organisms [11, 12]. These databases, in turn, have greatly enabled methods for identifying putative PPIs using data analysis and mining. Combined with the explosion of advances in the field of deep learning in recent years, it is increasingly viable for designers of PPI inference methods to leverage neural networks to increase benchmark accuracies.

Despite best efforts, accurate PPI inference that generalizes to all use-cases remains regrettably out of reach. Surprisingly prevalent and persistent challenges in the nature of training datasets and model architecture frustrate efforts to train PPI inference models which perform well on out-of-distribution examples [13, 14, 15]. While these hurdles have long been known to researchers, the challenges remain relevant and affect recent methods [15].

This thesis contains a collection of three studies aimed at improving PPI inference performance on out-of-distribution data, as well as characterizing new challenges on the horizon that threaten the generalization of tomorrow's PPI inference models when trained on datasets with maximum sequence identity thresholds ranging from 50% to 90%.

## Thesis Organization & Contributions

This thesis is organized into four additional chapters whose main technical contributions are:

- **Chapter 2** – This thesis begins with a comprehensive review of the biological and computational literature which inform this work.
- **Chapter 3** – This chapter introduces RAPPPID, a simple model for PPI inference which relies on judicious applications of regularization to achieve state-of-the-art performance at the time of publication. This study also highlights that contemporaneous models fail to generalize to proteins outside the training set, and introduces tokenization typically applied to natural language to amino acids.

- **Chapter 4** – Expanding on the work done on RAPPPID, Chapter 4 reports on an investigation into leveraging orthology to further generalize PPI inference to unseen proteins belonging to evolutionarily-distant organisms. The resulting method, INTREPPPID, outperforms RAPPPID and other contemporaneous models when trained on human PPIs, and subsequently evaluated on other organisms. To do so, a novel "orthologous locality" loss term is defined to reshape the protein embedding latent space, "pushing" orthologous proteins close to one another and "pulling" non-orthologous proteins apart.

- **Chapter 5** – In which we reveal a before-uncharacterized source of data leakage in PPI inference models which incorporate pre-trained protein large language models (PLLMs). Furthermore, we demonstrate how careful attention to the creation of training datasets can result in PLLMs which are both performant and do not leak data. Finally, failures to generalize across PLLM-based and non-PLLM-based are described.

## Publications & Author Contributions

- **Chapter 3**

  Szymborski, Joseph, and Amin Emad. 2022. "RAPPPID: Towards Generalizable Protein Interaction Prediction with AWD-LSTM Twin Networks." *Bioinformatics* 38 (16): 3958–67.
  DOI: 10.1093/bioinformatics/btac429.

  A.E. and J.S. conceived the study, designed the project and the algorithm, and wrote the article. J.S. implemented the pipeline and performed the statistical analyses of the results. All authors read and approved the final article.

- **Chapter 4**

  Szymborski, Joseph, and Amin Emad. 2024. "INTREPPPID—an Orthologue-Informed Quintuplet Network for Cross-Species Prediction of Protein–Protein Interaction." *Briefings in Bioinformatics* 25 (5): bbae405. DOI: 10.1093/bib/bbae405.

  A.E. and J.S. contributed to project conceptualization, methodology as well as writing, editing, and reviewing. A.E. contributed to supervision and funding acquisition. J.S. contributed to software, data curation, investigation and visualization. All authors read and approved the final article.

- **Chapter 5**

  The manuscript in Chapter 5 is under-review and is currently available as a pre-print on the bioRxiv online pre-print repository.

  Szymborski, Joseph, and Amin Emad. 2025. "A Flaw in Using Pre-Trained pLLMs in Protein-Protein Interaction Inference Models." *bioRxiv*. DOI: 10.1101/2025.04.21.649858.

  A.E. and J.S. contributed to project conceptualization, methodology as well as writing, editing, and reviewing. A.E. contributed to supervision and funding acquisition. J.S. contributed to software, data curation, investigation and visualization. All authors read and approved the final article.

# Chapter 2

# Background Material and Literature Review

The studies of which this thesis is composed are multi-disciplinary in scope and nature. They encompass a wide range of fields as varied as molecular biology, biocuration, machine learning, and deep neural networks. This chapter serves as a gentle but comprehensive introduction to the relevant aspects of these fields to help guide the reader in their understanding of the chapters that follow it.

## The Biological Basis of Protein-Protein Interactions

Proteins are the subject, verb, and object of a dizzying number of molecular functions that make-up life. This chapter section discusses those functions, the scientific phenomena that causes them to occur, and the sometimes grim outcomes of their dysregulation.

### Biological Processes, Pathways, and Networks

The number of interactions between two proteins ("binary interactions") in humans has been estimated at around 600 000, while the literature is estimated to contain upward of 110 000 (or $\approx$ 18%) of those interactions [16, 17]. As there are upwards of 20 000 human proteins with over 400 million possible combinations, the proportions of proteins that interact with one another (*i.e.*, members of the "interactome") are relatively few. This makes for a rather sparse network of interactions. Despite this, the proportion of non-interacting proteins which appear in the literature is far smaller than that of those which do interact. A mere 2 171 non-interacting pairs found in literature are present in the Negatome database, an effort by researchers at the

Technical University of Munich to collect evidence of non-interacting protein pairs [18]. If the aforementioned estimates are to be believed, approximately one thousandth of one percent of non-interacting protein pairs have been verified experimentally. An additional layer of complexity atop this network of interactions is that the sequence, speed, and environment in which protein interactions occur greatly impact their downstream effects. The large and complex network formed by these interactions has led to comparisons to a hairball [19, 20].

The structural organization, metabolism, proliferation, DNA repair, and death of a cell are just a sample of the very many functions for which PPIs are primarily responsible for. The functions are carried out by an often-diverging chain of interactions that are typically referred to as "pathways". The latest version of Reactome, a knowledgebase which catalogues peer-reviewed reactions and pathways, identifies 2 799 pathways in *Homo sapiens* broadly sorted into 29 categories reflecting their function and role within the cell, ranging from signal transduction to extracellular matrix organization (Figure 2.1) [21].

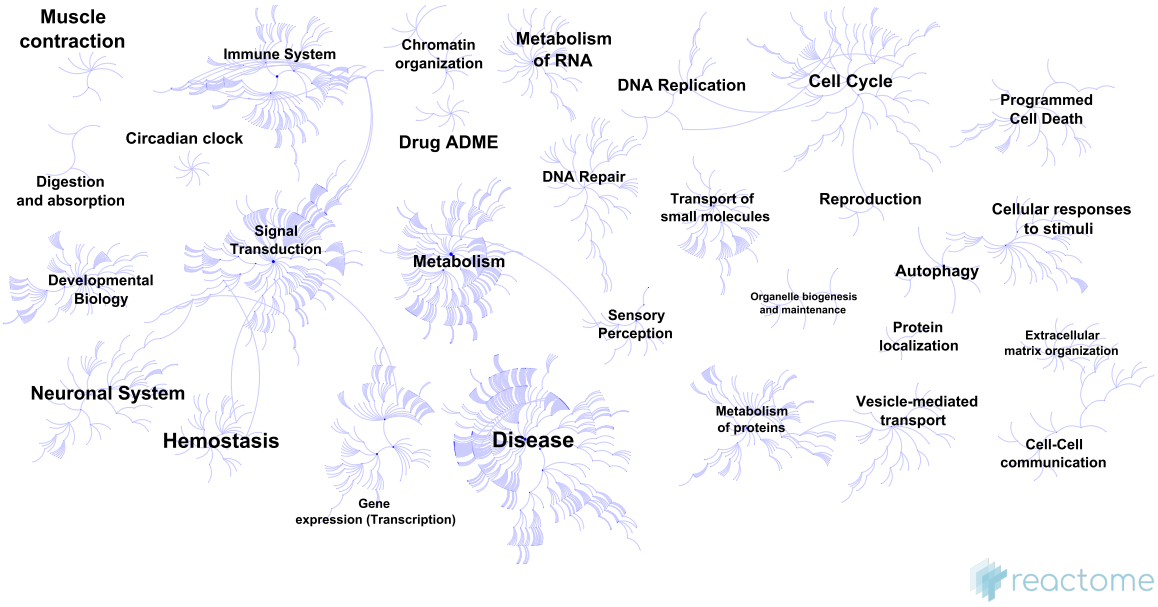



**Figure 2.1: A high-level schematic of the human pathways in the Reactome knowledgebase.** Each blue curved line segment represent a protein interaction within a broader pathway. Pathways are sorted in 29 broad categories that reflect their function and role. © 2025 Reactome / CC-BY 4.0.

These pathways are composed of interactions which can involve pairs of discrete proteins or many proteins acting in concert as a complex [22]. While these complexes may be composed of proteins which are different to one another in both sequence and structure ("heter-oligomeric" complexes), those comprise entirely of the same protein are also common ("homo-oligomeric"). In addition to other proteins, proteins interact with other macromolecules like ribonucleic acids (RNA), deoxyribonucleic acids (DNA), and even glycans to perform these various functions. These interactions, however, are less understood and, to our current knowledge, fewer in number [23, 24]. Proteins also regularly interact with small molecules like adenosine triphosphate (ATP)

and ions like those of the metals zinc and calcium [25, 26, 27]. For the purposes of this thesis, we will restrict our scope to binary interactions between proteins.

## Physiochemical Fundamentals of Protein Interactions

Protein interactions, at their most fundamental level, are chemical reactions. These reactions can alter the structural or chemical properties of the interacting proteins or other macromolecules. Additionally, enzymes are a class of protein which intercede to catalyze other biochemical reactions.

Proteins within a cell travel through its different liquid phases via a random process known as Brownian diffusion [28, 29]. This undirected movement results in random collisions between proteins which then have an opportunity to physically and/or chemically interact with each other.

The strength with which two or more proteins interact is referred to as their "binding affinity" with respect to another protein. One way of quantifying such an interaction is its dissociation constant ($K_D$), which measures the concentration of ligand at which half of the protein is bound[1] [30]. For an equilibrium between a protein P and a ligand L

$$P + L \underset{k_{off}}{\overset{k_{on}}{\rightleftharpoons}} PL$$

where $k_{on}$ and $k_{off}$ reflects the forward and reverse rate of reaction, the value of $K_D$ is defined as

$$K_D = \frac{k_{off}}{k_{on}} = \frac{[P][L]}{[PL]}$$

[31].

Protein interactions vary greatly with regard to their reaction kinetics, with some protein interactions being transient and short-lived (*i.e.*, high $K_D$ value), while others are long-lived stable interactions (*i.e.*, small $K_D$ value).

Some kinase reactions are short-lived. One such example is given by the Mitogen-activated protein kinase (*MAPK*) cascade, whose involvement in cell proliferation, differentiation, stress, and apoptosis set the stage for its role in, among other things, metabolic disease and cancers [32, 33, 34]. Specifically, the isoforms of the mitogen-activated protein kinase kinase (*MEK1/2*) and the isoforms of the extracellular signal-regulated kinase (*ERK1/2*) bind very quickly ($k_{on}$ =

[1] This definition only holds for binary interactions involving a single binding site, an assumption held throughout this thesis.

$3.23 \times 10^8$ $M^{-1}s^{-1}$) and dissociate relatively quickly ($k_{off}$ = 18.7 $s^{-1}$) [35]. In comparison, Biotin and Avidin form very long lasting reactions, with a comparable association rate to *MEK–ERK* ($k_{on}$ = $7 \times 10^7$ $M^{-1}s^{-1}$) and a much slower dissociation rate ($k_{off} \approx 10^{-7}s^{-1}$) [36]. This is reflected in the difference of their dissociation constants. The *MEK–ERK* reaction has a dissociation constant of $5 \times 10^{-8}$ M while the Biotin–Avidin constant is much smaller at $7 \times 10^{-15}$ M.

Proteins most commonly selectively bind their ligands to some degree, however some "promiscuous" proteins are capable of almost indiscriminately binding to other proteins [37]. This promiscuity, however, is often evolutionarily balanced by an increase in protein instability [37].

The binding affinity and specificity of a protein to another protein is largely determined by their three-dimensional structure and chemistry — their "molecular complementarity" [38]. Our understanding of how a protein's structure factors into its function was put into focus when researchers solved the first protein structures, that of myoglobin in 1958 and hæmoglobin in 1960 [39, 40, 41]. In the early 1960s, shortly after the first protein structures were resolved, the role of amino acid sequences in the structure of a protein was largely crystallized through work done by Christian B. Anfinsen and his colleagues. Anfinsen for his part would go on to win the 1972 Nobel Prize in Chemistry, having established "...the connection between the amino acid sequence and the biologically active conformation" [42, 43]. Anfinsen advanced a "thermodynamic hypothesis" which states "...that the native conformation is determined by the totality of inter-atomic interactions and hence by the amino acid sequence, in a given environment" [43]. This would later be commonly referred to by many as "Anfinsen's dogma", by way of comparison to Crick's "central dogma of molecular biology" [44, 45, 46].

Together, these works uncovered a triune of protein biochemistry, whereby a protein's sequence determines its structure and its structure in turn determines its function. As is the case with most statements of fact with wide-sweeping implications, this is a gross oversimplification and hides within it many pernicious exceptions and conditions [47, 48, 44, 45]. Nonetheless, it shall serve as a useful guide for our discussion of the basis of protein interactions.

## Protein Interactions and Pathologies

Having established that proteins are essential for many essential functions of the cell, it requires little imagination to arrive at the conclusion that their dysfunction can have grave implications for organismal health. Our understanding of how protein interaction networks and how diseases alter their fabric has lead to many therapies and treatments. For instance, network analysis on an organism's interactome can be useful in identifying which proteins are "indispensable" to normal cell function and therefore whose dysfunction is more likely to cause disease and illness [49, 50]. This can be done by analyzing the effects of removing entire proteins from the protein

interaction graph [49], or by observing the result of a protein which no longer interacts with one of its usual partners due to *e.g.*, mutations which alter one of multiple binding sites [51]. The latter are known as "edgetic" perturbations, and give us particular insight into genetic diseases which cause small changes in proteins rather than the drastic sort which eliminates all function or expression of the protein [51].

Analyses of the "edgetic" changes are particularly relevant in the context of cancers which are largely driven by germline and somatic mutations which alter protein function [52]. Network analysis of the binding partners of mutant drivers of breast cancer, such as *TP53* and *PTEN*, helped identify new proteins previously unknown to be implicated in the onset of cancer [53]. Similarly, network analysis has helped us understand some of the mechanisms by which hereditary diseases like Huntingtin's disease progress, implicating novel interactors of the mutant *HTT* protein [54]. The new discoveries can then help identify new targets for the development of small-molecule therapies to restore wild-type function [55, 56].

The clinical utility of protein interaction networks aren't limited to cancers and hereditary diseases. Early networks of between human proteins and the SARS-CoV-2 virus, the cause of Coronavirus disease 2019 (COVID-19), helped quickly identify druggable protein targets months after a worldwide pandemic was announced [3].

The study of protein-protein interaction networks, therefore, is one of the rare fields of study in the biological sciences whose pursuit readily advances both basic and translational research.

## Experimental Approaches to Discovering Protein-Protein Interactions

A huge number of PPI interactions have been characterized to date, with over 20 billion interactions across more than a thousand organisms being catalogued in version 12 of the STRING database [11]. The scale of the studies whose findings populate databases like STRING are increasing as well, with as many as 53 000 PPIs being identified in a single study.

Protein-protein interaction have been discovered using a wide variety of methods. Many methods involve identifying the unknown partners ("prey" proteins) of a known target protein ("bait" protein). Each method presents its own unique advantages and challenges. These limitations, including high costs, biases and lack of sensitivity, present as obstacles to uncovering the complete network of proteins which interact [5, 6].

This section serves as an overview of common methods for discovering and verifying PPIs.

## Yeast Two-Hybrid and Related Methods

The Y2H system is one of the oldest and best studied methods for experimentally discovering and validating PPIs, originating from work by Fields and Song 36 years ago [57, 4]. The Y2H method involves fusing the binding domain of a transcription factor to the bait protein and that same transcription factors activation domain to the prey proteins [57]. When the two proteins interact, they effectively reconstitute the function of the transcription factor, which results in the expression of some desired reporter gene. In their original work, Fields and Song used the *Gal4-UAS* system, which allowed for screening by the ability of cells to grow on galactose or for the blue colour they produce when grown on indicator plates [57].

(MYTH) is a variation of the Y2H system which adapts the method to the study of membrane proteins [58].

Many related methods for studying and observing PPIs involve fusing domains to subjects of study such that bait-prey interactions reconstitute functional systems whose activity is directly or indirectly observable [5]. Examples include MaMTH and BiFC, whose interactions cause the reconstitution of ubiquitin and a fluorescent complex, respectively [59, 60].

The *JAK/STAT3* signal transduction pathway has also proved useful in identifying PPIs experimentally. Both the MAPPPIT and KISS methods involve fusing kinase and/or receptor domains which, upon prey-bait interactions, causes activation of the *JAK/STAT3* signalling pathway and the downstream expression of a reporter gene [61, 62].

## Affinity-based Methods

Affinity methods involve the physical capture, through the use of antibodies, of target proteins which have bound proteins.

Co-immunoprecipitation (CO-IP) is a useful technique for validating hypothesized PPIs. Primary antibodies which bind the target protein are first incubated with cell lysate, often from cells which over-express the protein of interest [63]. Secondary antibodies, conjugated to agarose or magnetic beads, are used to isolate the target protein [63]. Following the separation of the lysate's constituents on an SDS-PAGE gel, the presence of the proteins hypothesized to interact is confirmed by Western blot [63].

Whereas CO-IP is suitable for validating suspected interaction partners, affinity purification mass-spectroscopy (AP-MS) is a method which can be used for the discovery of new interaction partners. Baits are fused with short peptide sequences known as affinity tags (*e.g.*, FLAG or HA) [64]. Purification is then conducted much in the same way as it is for CO-IP, but primary

antibodies here target the affinity tag [64]. The most significant difference is that, rather than identifying interacting partners using a Western blot, AP-MS relies on mass spectrometry. Mass spectrometry allows for the identification of partners without making assumptions as to the proteins which have interacted with the bait, as is the case with CO-IP [64].

Both AP-MS and CO-IP require that protein interactions survive cell lysis which, depending on the strength of the interaction, may not be the case. BIOID-MS is a method whereby a promiscous biotin ligase (*e.g.*, BirA) is fused to a bait protein, resulting in the biotinylation of interactors [65]. The biotinylated proteins can then be isolated using avidin/streptavidin affinity columns and the partners can be identified by mass spectrometry [65].

### Energy Transfer Methods

The methods for PPI discovery and validation here thus far have not been suitable for real-time observation of protein interaction dynamics. Förster resonance energy transfer, or sometimes fluorescence resonance energy transfer, (FRET) is capable of spatial and temporal resolution of PPIs in living cells [66, 5].

FRET shares some similarity with BIFC, but rather than fusing fragments of one fluorescent protein, fluorophores are fused to both proteins. One of the fluorophores (the "donor") emits at a wavelength which excites the other (the "acceptor") [66, 5]. The acceptor only emits when the donor is near enough to excite it, unlike with BIFC where the reconstitution of the fluorescent complex is permanent [5].

Bioluminescence resonance energy transfer (BRET) is a similar method which helps reduce background noise and photobleaching which are obstacles caused by FRET's reliance on external illumination [66]. Rather than fusing two fluorophores, BRET experiments replace the "donor" fluorophore with luciferase, eliminating the need for external illumination to excite the "donor" [66].

### Other methods

Cross-linking mass spectrometry (XL-MS) is a method involves forming covalent bonds between nearby protein residues using a chemical cross-linker. The cross-linked proteins are then identified using mass spectrometry. XL-MS is both high-throughput and allows us to identify protein complexes. Cross-linking, however, can often have low-efficiency and are slow, limiting the interactions that can be detected using XL-MS [6].

Many of the methods here involve high-impact manipulations involving greatly altering the

natural environments of cells or proteins, which can distort or mask certain reactions. coFrac-MS is a method for identifying possible protein complexes while avoiding any such harsh conditions at the cost of measuring functional association via co-expression/co-localization rather than direct PPIs [6]. This is achieved by fractionating cell lysates under native conditions and identifying the protein constituents of fractions by mass spectrometry [6].

Thermal proximity co-aggregation (TPCA) is a method that exploits the finding that proteins which interact co-aggregate when thermally denatured [67]. By comparing the melting curves of proteins, interaction partners are inferred [67]. This method can successfully identify protein complexes, is sensitive to weakly bound proteins and can be performed *in situ*.

## Computational & Statistical Models of Protein-Protein Interactions

Methods for computationally inferring PPIs span a wide range, with the field largely moving from statistical methods to machine learning methods and finally to deep learning models (Figure 2.2). I have found that, much as musical artists resist being assigned genre labels, so too do models of PPI inference. Nevertheless, an effort was made to thematically group models into broad categories on the basis of the methodologies they employ. Viewing these groups over the passage of time gives us useful insight into the trends and discoveries that have shaped the field of PPI inference.

The list of studies reviewed here is not exhaustive, but rather a representative cross-section of the methodologies that have been used for PPI inference. They are presented here with the aim that they might help situate the novel approaches presented later in this thesis within the broader field.

### Non-Machine Learning Methods for PPI Inference

#### Interaction Site Prediction

The earliest methods for PPI inference largely centred identifying on identifying similarities across proteins and networks belonging to other species, and interpolating interactions between them on that basis [68, 69, 70]. The earliest study of this nature of which I am aware also is arguably the simplest; Kini and Evans described a method for predicting interaction sites in 1996 by noting the prevalence of proline residues which flank such regions [68]. This method was used to identify the existence of a polymerization site in the epitope of fibrinogen which

they found to be bound by a specific monocolonal antibody [68]. A further generalization of the identification of interaction sites on the basis of the composition of amino acid residues and their three-dimensional structure closely followed in the work of Jones and Thornton the following year [70]. Jones and Thornton devised a scoring system for surface patches of the

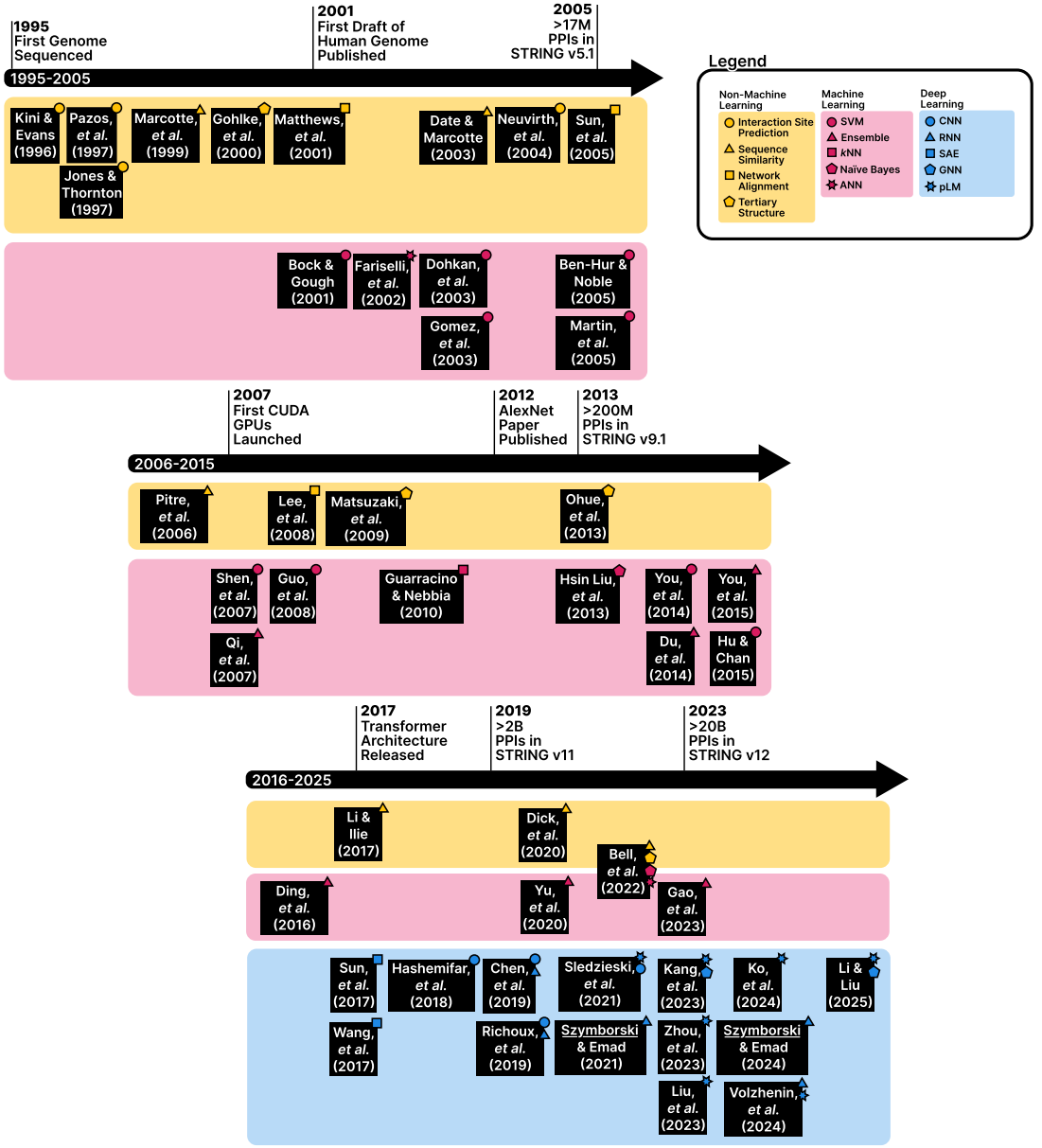



**Figure 2.2: A historical timeline of PPI inference methodologies.** A timeline of papers representative of the last 30 years of advances in PPI inference research, with particular focus to the methodologies employed. Each black box represents a study, and is placed along a timeline across three decades beginning from 1995. Coloured rounded boxes group the timeline according to broad methodological themes: non-machine learning (yellow), machine learning (red), deep learning (blue). Emblems at the right of each black box indicate the methodology studied in each publication. Emphasis has been placed on publications of which I am an author.

resolved three-dimensional structure of proteins which, among other factors, accounted for its solvation potential, hydrophobicity, accessible surface area (ASA), and planarity [70]. Seven years later, a study would revisit the analysis of surface properties of proteins which might help identify interactions sites by characterizing differences between the interfaces of bound and unbound protein complexes [71]. Another early method by Pazos *et al.* published in 1997 uses the correlation of mutations between proteins to identify putative interaction sites [69].

#### Sequence Similarity

The similarity between protein sequences has been demonstrated to have relationship to their relative structure and function for over 30 years [72]. This relationship has been leveraged to infer interactions between proteins on the basis of their similarity to each other, or to other homologous proteins which interact. One of the earliest such methods was devised in 1999 by Marcotte *et al.* who describe a "Rosetta Stone" method, which uncovers interacting proteins which appear as fused singular proteins in other species [73]. This finding serves as the seed for later work expanding and generalizing these ideas by Date and Marcotte in 2004 [74]. Specifically, Date and Marcotte constructed what they termed "phylogenic profile" vectors for each protein for which PPIs were to be constructed. Each element of these 57-dimension $\mathbf{p}$ vectors corresponds to a reference organism $j$. The value of each element $p_{i,j} = \max\left[-\log\left(E_{i,j}\right)^{-1}, 1\right]$, where $E_{i,j}$ is the lowest expectation value ($E$-value) found between the protein in question and the the proteome of organism $j$, as measured by the BLAST software [75, 74]. The mutual information between the $\mathbf{p}$ vectors of pairs of proteins correlate with their functional similarity [74]. Functional similarity was determined using the KEGG knowledgebase [76]. Notably, Date and Marcotte's method infers functional associations between proteins, sometime termed "functional interactions", which are not to be confused with physical interactions.

Sequence similarity methods remain the most common variety of non-machine learning models that are developed in recent years. These methods infer interactions on the basis of having previously observed other pairs of proteins with similar amino acid sequences which themselves interact with one another. A pioneering example of these methods is Pitre, *et al.*'s PIPE method [77].

PIPE begins by sliding overlapping windows over two proteins (protein $A$ and $B$) whose interaction status is being evaluated. A search for proteins whose sequence contains similar regions to the overlapping windows over protein $A$ is conducted. Subsequently, proteins which interact with the proteins found in the search are catalogued in a list $R$. Finally, a search is conducted for similarities between the windows from the sequence of protein $B$ and those in $R$ to construct a final score matrix. Each element $s_{i,j}$ of the score matrix corresponds to instances where the $j^{\text{th}}$ window from protein $B$ matched a member of $R$ who was added to the list due to its interaction

with a protein who shared sequence similarity to the $i^{\text{th}}$ window from protein $A$. An *ad hoc* visual analysis of the scoring matrix is then performed, whereby strong peaks in regions of the scoring matrix are indications of possible interactions.

The original PIPE algorithm was very computationally demanding, with its authors citing an approximate 1 000 hours of computational time required to conduct the evaluations that appear in the original publication [77]. The computational performance and sensitivity of the PIPE algorithm has since been iterated upon and improved over the course of 14 years [78, 79, 80, 81]. Li and Ilie developed SPRINT, a method reminiscent of PIPE in its theory, and boldly claimed in their 2017 publication that that theirs was "...the only sequence-based program that can effectively predict the entire human interactome: it requires between 15 and 100 [minutes], depending on the dataset" [82].

Bell *et al.* would later describe PEPPI, a pipeline which integrates multiple signals to infer PPIs, including protein sequence and stucture similarity.

#### Tertiary Structure Methods

The non-machine learning methods discussed thus far have largely operated on the basis of the amino acid sequences of proteins. Other methods rely on the tertiary structure of proteins to screen for pairs which are likely to interact.

Tsukamoto *et al.* introduced their Affinity Evaluation and Prediction (AEP) aglorithm in 2009, whereby the three-dimensional structure of proteins are first voxelized [83]. The complementarity of these structures are then computed and used to infer interaction. Matsuzaki *et al.* outlined another such method in the same year, but this time using a protein docking software (ZDOCK) to assess complementarity [84, 85]. Both these methods require exhaustive searches and are very computationally costly.

MEGADOCK, a method describe in a 2013 publication by Ohue *et al.*, was conceived to address these computational performance concerns [86]. Ohue *et al.* claimed a 7.5-fold improvement in docking calculation speed all while maintaining "an acceptable level of accuracy".

### Machine Learning Methods for PPI Inference

From the years 1995 to 2000, PPI inference methods were primarily predicated on expert analyses and heuristics. These, often partially manual, methods are time consuming and often have limited accuracy. This is perhaps partly responsible for the limited scope of the applied examples of these publications, likely further exacerbated by the accessibility and scope of the contemporaneous

databases available. Work by Bock and Gough is an early, if not the first, example of machine learning deployed in the then-nascent field of PPI inference [87]. Bock and Gough use the Support Vector Machine (SVM) algorithm, which would become one of the most common machine learning approaches for inferring PPIs before deep learning would become prevalent [87].

#### Support Vector Machines

The SVM is a very well-studied machine learning algorithm whose underlying concepts arose in the 1960s and which would eventually enjoy a great deal of popularity in the early-to-mid 2000s across a number of fields, not least of which being computational biology [88, 89, 90]. SVMs operate by maximizing the distance between a hyperplane and the nearest data point on either of its sides, resulting in what's termed as the "maximum-margin hyperplane" [90]. While SVMs were originally characterized a linear classifier, they can be easily adapted to non-linearly separable classification tasks by applying a kernel which projects the data into a higher dimension where such linear maximum-margin hyperplanes are more likely to exist [91, 90].

Many of the studies which sought to apply the SVM algorithm to the PPI inference task concern themselves primarily with feature engineering and devising appropriate kernels. In the same year, Gomez *et al.* and Dohkan *et al.* used SVMs by representing proteins as a "bag of domains" as well as incorporating amino acid composition features [92, 93]. Representing variable-length amino acid sequences as fixed-length vectors is a subject which has received much focus, as they are required by the SVM algorithm as well as other machine learning approaches [94, 95, 96]. Martin *et al.* used features inspired by chemical signatures to represent variable-length amino acid sequences into fixed vector representations [96].

All applications of SVMs to the binary PPI inference task must conceive of kernels which allow for the pairwise comparison of proteins. That is to say that methods must devise of a kernel $\tilde{k}\left[(A,B),(X,Y)\right] \rightarrow \mathbb{R}$ which calculates the similarity of the pair of proteins $A$ and $B$ with the pair $X$ and $Y$. Ben-Hur and Noble term this variety of kernel a "pairwise kernel" [97]. A pairwise kernel often makes use of a second "sequence" kernel which computes the similarity between two protein sequences [97]. Martin *et al.*, for instance, use the sequence kernel

$$k(A,B) = s(A) \cdot s(B)$$

where $k$ is the sequence kernel and $s$ is the aforementioned signature function which projects protein sequences into fixed vectors. Martin *et al.* go on to define the pairwise kernel as

$$\tilde{k}\left[(A,B),(X,Y)\right] = k(A,B)k(X,Y)$$

where $\tilde{k}$ is the pairwise kernel [96]. Ben Hur and Noble concurrently and independently described the same kernels and termed the sequence kernel a "spectrum kernel" [97].

Shen *et al.* use a so-called "conjoint triad" of residues as features which take into consideration the neighbourhood of each residue and their physiochemical properties. A sequence kernel which computes the distance between these residue feature vectors was used and can be expressed, for two proteins $A$ and $B$, as

$$k(A,B) = ||A - B||^2.$$

This sequence kernel is then incorporated into a symmetric pairwise kernel of the form

$$\tilde{k}\left[(A,B),(X,Y)\right] = \min\left[k(A,X) + k(B,Y),\ k(A,Y) + k(B,X)\right].$$

Supervised machine learning methods like SVM require large representative training sets to perform optimally, and as a result it is desirable to scale these methods to handle such datasets. You *et al.* explore a method to improve scalability by parallelizing a SVM model for PPI inference using the MapReduce algorithm [98, 99].

#### Ensemble Methods

Ensemble methods combine multiple weaker predictors to form a single, more performant one. An early example of such a method is provided by Qi *et al.* who describe a Mixture-of-Experts (MOE) method composed of four experts [100]. The first expert ($P$) was trained on high-throughput PPI experiments, the second expert ($E$) trained on gene expression data, the third expert ($S$) was trained on sequence-based features like homology, and the fourth and final expert ($F$) were trained on functional annotations. Bell *et al.*'s PEPPI method can be viewed as an ensemble method, as it classifies the interaction status of pairs of proteins by computing the consensus of four independent models which (1) perform look-ups in existing PPI databases, searches for (2) sequence and (3) structure interologues, and finally (4) a shallow neural network which operates on amino acid sequence triads [101].

The Random Forest algorithm is very well represented among studies which use ensemble methods [102, 103, 104, 105, 106, 107]. The Random Forest models consist of multiple shallow decision trees whose output are averaged to assign a final inferred label [102]. In contrast to Qi

*et al.*'s MOE model, the features upon which sub-models are trained are chosen at random in Random Forest models rather than being selected manually [102]. Additionally, it is typical for Random Forest models to have many more sub-models than the four found in Qi *et al.*'s MOE model and Bell *et al.*'s PEPPI.

### Other Methods

While the most common traditional machine learning algorithms used for inferring PPI models are SVMs and Random Forest, other algorithms are also sometimes used. Guarracino and Nebbia created a PPI inference model using the $k$-nearest neighbours ($k$NN) algorithm [108, 109]. For this method, Guarracino and Nebbia computed amino acid composition feature vectors for each protein in a interacting or non-interacting pair, concatenated those vectors. To infer the interaction status of a new pair of proteins $x$, the $L^1$ (Manhattan) distance between the concatenated amino acid feature vectors of $x$ and those in the training set is measured to find its $k$ nearest neighbours [110]. If the majority of these neighbours are features of interacting pairs, the members of $x$ are deemed to interact, and *vice versa*.

Another algorithm sometimes used for inferring PPI interactions is the naïve Bayes classifier [111, 112]. Hsin Liu *et al.* use the naïve Bayes classifier to combine 91 likelihood ratios which reflect the divergence in co-evolution between proteins [112]. Bell *et al.* similarly rely on a naïve Bayes classifier to ascertain a consensus between the five independent models they train.

In 2002, Fariselli *et al.* published a work which made the rare use of an artificial neural network (ANN) for PPI inference. It would be another 15 years before neural networks would become common place for such a task [113].

## Deep Learning Methods for PPI Inference

While implementations of ANNs have been around since the late 1950s in some form or another, it would not be until approximately five decades later in the 2010s when networks with very many layers would see widespread use [114]. Much of the groundwork for these "deep" neural networks (DNNs) was laid between the early 1980s and the mid 1990s, with the development and implementation of concepts such as Hopfields networks, backpropagation, convolutional neural networks (CNNs), and recurrent neural networks (RNNs) [115, 116, 117, 118, 119, 120].

The term "deep learning" as applied to neural networks would be coined at the turn of the millennium but widespread adoption of deep neural networks came on the back of multiple simultaneous developments — including general-purpose computing on graphics processing units (GPUs), improvements to neural network implementations, and the advancement of big

data algorithms [121, 120]. A watershed moment for deep learning occurred in 2012 when Krizhevsky, *et al.* published AlexNet, a network for image classification that marked the first time a neural network won the ImageNet competition [122]. Computer vision and would be the field that arguably benefitted the most from the flurry of DNN research that followed AlexNet until self-attention and the transformer architecture would revolutionize the field of natural language processing (NLP) [123].

The adoption of DNN architectures to the task of PPI inference largely follow the trends of the broader field, with readily available examples of such models based on stacked autoencoders, RNNs, CNNs, protein language models and graph neural networks.

#### Stacked Autoencoders

In 2017, Sun *et al.* claimed in an article published in BMC Bioinformatics:

> To our knowledge, this research is the first to apply a deep-learning algorithm to sequence-based PPI prediction, and the results demonstrate its potential in this field. [124]

Sun *et al.*'s first-of-kind deep PPI inference method would adopt the stacked autoencoder (SAE) architecture [124, 125]. A similar SAE method for PPI inference by Wang *et al.* would be submitted for publication in Molecular BioSystems 23 days later [126]. These methods would foreshadow a burgeoning field of DNN-based PPI inference methods in the years to come.

SAEs are a neural network architecture that involve chaining the outputs and inputs of a series of autoencoders [125]. Autoencoders (AEs) are in turn neural networks which are composed of two sub-networks: an encoder and a decoder. The encoder is a network which projects an input into some smaller dimension, and a decoder approximates the encoder's inverse function. That is to say,

$$g : \mathbb{R}^n \rightarrow \mathbb{R}^m \ , \ g' : \mathbb{R}^m \rightarrow \mathbb{R}^n$$

where $g$ is an encoder, $g'$ is a decoder, and $n >> m$ [125]. Given some input $x \in \mathbb{R}^n$, the encoder computes a latent vector $z \in \mathbb{R}^m$. The decoder, being optimized to invert the encoding given the latent vector, means that

$$g'(z) = \hat{x} \approx x$$

where $\hat{x}$ is the reconstructed input encoded by latent vector $z$ [125].

While important to the initial development of DNN-based PPI inference, the SAE architecture

remained uncommon outside these two studies, being primarily superseded in the next few years by RNNs and CNNs.

### Convolutional Neural Networks

Retellings of the invention of the CNN often begin with Kunihiko Fukushima who published the idea while working at the NHK Broadcasting Science Research Laboratories in 1980 [118, 120]. Using the name "neocognitron" for the concept — an iteration of his earlier "cognitron" — Fukushima created a feature detector network which took inspiration from research by David Hunter Hubel and Torsten Nils Wiesel who conducted research on the cells of the visual neural cortex of mammals and their relation to pattern recognition [127, 128, 129, 120]. The two layer types used in the neocognitron, "S-layers" and "C-layers", resemble convolutional and pooling layers, respectively — both commonly used neural network layers used for computer vision tasks [118].

Nearly 20 years later, LeCun *et al.* would describe how fully-connected layers could be constrained to be convolutional, thereby describing the CNNs we know today [130, 131]. Briefly, CNNs apply convolutions to input tensors using a learnable kernel. CNN's are accordingly well suited to computer vision tasks. When convolutions are only applied in one-dimension, however, they lend themselves to sequences of features [132].

An early example of CNN's being applied to the PPI inference task is Hashemifar *et al.*'s DPPI method [133]. CNN's are, in the context of PPI inference methods however, are commonly used in conjunction with RNNs.

### Recurrent Neural Networks

In 1990, Jeffrey L. Elman of the University of California, San Diego published a study entitled "Finding Structure in Time" in the journal Cognitive Science. This seminal article provides one of the earliest descriptions of an RNN architecture, later to be known eponymously as an Elman network [119]. RNNs were conceived with the goal of better modelling time-series data using neural networks. RNNs add a time-based inductive bias to the architecture of neural networks, where the locality of elements in the time series is made relevant. Incremental improvements on RNN architecture would ultimately cumulate into the Long Short-Term Memory (LSTM) network seven years later [134]. Devised by Sepp Hochreiter and Jörgen Schmidhuber, the LSTM gated architecture addressed many of the problems plaguing older RNN constructions by improving gradient flow and training stability as well as performance on tasks that require long-range dependencies [134]. The following year Hochreiter would publish a simplified construction similar to the LSTM architecture which he named the Gated Recurrent Unit (GRU).

RNNs are useful for any number of domains with a sequence of elements, be that a string of letter or words, as is the case for the field of NLP, a series of closing stock prices over the last quarter, as might be the case for quantitative finance, or a sequence of amino acid residues found in a protein — a scenario which is most relevant to this thesis.

Compared to Elman or Jordan networks, LSTMs and GRUs greatly reduce the risk of vanishing gradients due to long time-series. Despite this, LSTMs and GRUs still often suffer from training instability when trained on protein sequences due to their long length.

My methods, RAPPPID and INTREPPPID, both rely on LSTM encoders to learn latent vector representations of amino acid sequences. In both cases, to mitigate the training stability issue, amino acid sequences are first tokenized using the Unigram algorithm resulting in common groupings of common amino acids to be considered as one time-step when encoding the RNN [135, 136].

Richoux *et al.*'s work and Chen *et al.*'s PIPR both use a combination of RNN and CNN layers [137, 138]. The CNN layers, followed by pooling layers, serve to greatly reduce the number of time-steps, thereby increasing training stability of the downstream LSTM (in the case of Richoux *et al.*) and GRU (in the case of PIPR) layers.

#### Protein Language Models

Some two decades after Claude E. Shannon described a method for estimating the information entropy of the words that appear in printed English in 1950, Thomas M. Cover and Roger C. King would elaborate the idea by describing a "gambling" method for inferring the next letter of an English phrase [139, 140]. This is achieved by placing "sequential bets on the next symbol of text" using information about the underlying information entropy distribution [140]. Cover and King's method would, however, rely on human subject to "...work interactively at a computer terminal for a period of approximately five hours..." to generate predictions of a text sample approximately 250 words long, for an average of 50 words per hour [140]. It would be another quarter century before Bengio *et al.* would propose to model language neural networks using a similar method [141]. Bengio *et al.* began by modelling language as the "...conditional probability of the next word given all the previous ones...", given by

$$\hat{P}(W) = \prod_{t=1}^{T} \hat{P}\left(w_t | w_1, \ldots, w_{t-1}\right),$$

where $w_t$ is the $t^{\text{th}}$ word in the sequence $W = w_1, \ldots, w_T$ [141]. Concisely, Bengio *et al.* summarize their proposed method for modelling language using the following numbered list of steps:

1. associate with each word in the vocabulary a distributed *word feature vector* (a real-valued vector in $\mathbb{R}^m$),
2. express the joint *probability function* of word sequences in terms of the feature vectors of these words in the sequence, and
3. learn simultaneously the *word feature vectors* and the parameters of that *probability function*.[141]

While research on neural language models, particularly using RNNs, continued in the years following Bengio *et al.*'s introduction of the concept, it would be the introduction of self-attention and the transformer architecture that would result in huge gains in predictive performance and reignite a field [142, 123]. While originally trained on a neural machine translation task, transformers would later be employed to model language in a 2018 white paper by OpenAI, inc. introducing a method named GPT. GPT models language in a similar manner to that proposed by Bengio *et al.*, conditioning the probability of the next member of the sequence given all previous ones. The following year, researchers at Google's AI language group would release BERT, a language model that diverges from Bengio *et al.*'s "causal" approach [143]. Instead, BERT infers masked regions of sequences given *all* other tokens in the sequence. This variety of language model is referred to as a "masked language model" (MLM).

Rather than sequences of natural language, protein language models (pLLMs) apply the idea of neural language models to sequences of amino acids of which proteins are made-up. Early seminal work in modelling protein sequences by next-residue prediction include the UniRep and Prose method [144, 145, 146]. Both methods are based on LSTMs, as was common for contemporaneous language models [142]. The stated utility of these methods is primarily the generation of useful vector representation of proteins. As discussed at length in section 2, machine learning methods thus far have relied on various methods to generate fixed-length features for protein sequence with mixed-results. Prior to pLLMs, these methods often relied on heuristics that effectively produced a hash or signature of proteins, or learned embeddings which were often limited due to compromises intrinsic to the RNN or CNN architectures.

In the years that would follow the release of GPT and BERT, attention-based language models — of natural language and protein sequences alike — would balloon in parameter size (Figure 2.3). This would lead to the *terme de l'art* "large language model" or LLM in reference to the scale of the number of parameters. Much like the *deep* in "deep learning" or *big* of "big data", *large* is a relative qualifier whose definition in absolute terms is murky at best and rapidly changing with developments in the field.

A variety of attention-based pLLM models would be released by a number of groups in rapid succession between 2022 and today [147, 148, 149, 150, 151, 152, 153, 154]. When used as part of PPI inference models, these vector representations of proteins generated by these pLLMs

have resulted in great improvements in reported predictive performance. One of the earliest methods to use pLLM embeddings for PPI inference is the D-SCRIPT method, which made use of the ProSE language [155, 146]. D-SCRIPT computes a "broadcast matrix" from the protein embeddings of the proteins whose interaction status is being inferred. Using convolutional and pooling layers, a contact map between the two proteins is inferred as well as the probability of interaction. Kang *et al.*'s AFTGAN method uses embeddings from the ESM-1B pLLM to generate features for proteins in a graph of PPI interactions [156]. A graphical attention network is then used to learn PPI-specific features, before a fully-connected layer is used to infer interaction status [156]. Geometric learning is once more paired with pLLM's in work conducted by Li and Liu and their MuToN method. Unlike AFTGAN, MuToN uses attention-based message passing to learn structural features from the three-dimensional structure of the protein in question. MuToN uses a pre-trained pLLM to obtain embeddings of each residue in the protein as part of the features used to train the message passing network.

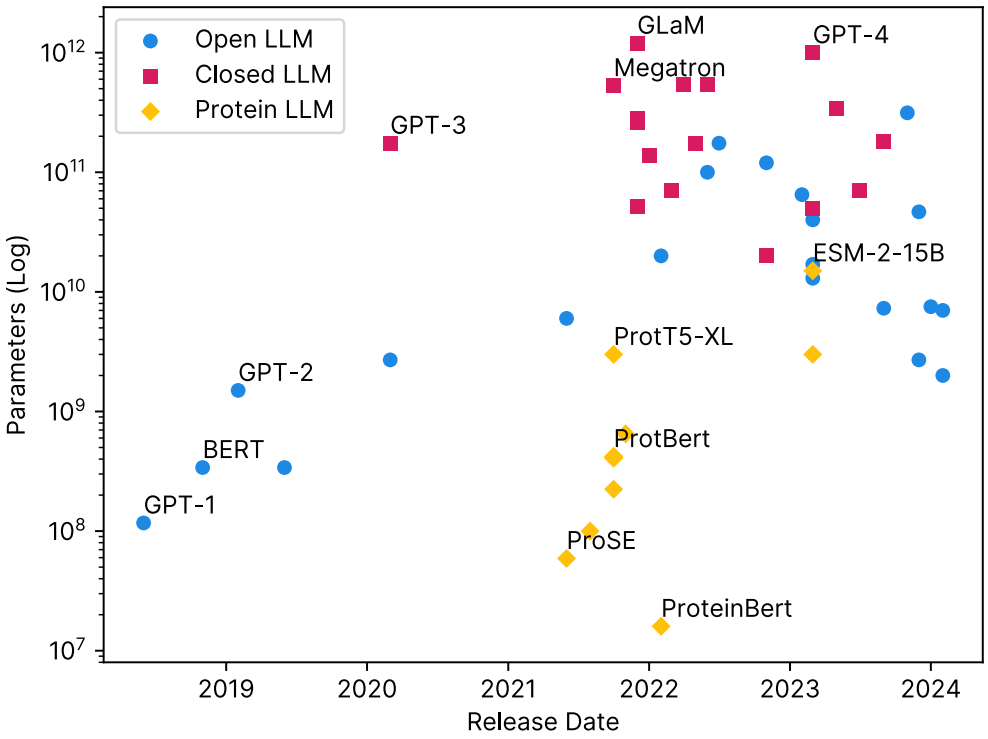



**Figure 2.3: Number of parameters in language models over time.** The number of model parameters (y-axis, log scale) found in LLMs as a function of their release date (x-axis). Round, blue markers indicate natural language models with open-source architectures. Red, square markers indicate natural language models whose architecture is closed-source. Parameter counts in these cases are difficult to verify and are either reported by the model's authors or otherwise estimated. Yellow, diamond markers report parameter counts of protein language models. The names of some notable models are included above their markers. Models included in this figure were released between 2018 and 2024.

Other pLLM-based methods use more conventional approaches to PPI inference. For instance, Zhou *et al.* simply compute the Hadamard product of protein vectors generated by ESM-1B and input the result to a four-layer fully-connected neural network which infers the interaction status [157]. Liu *et al.* input both protein sequences directly into a language model, delimited by a special "separation" token [151]. Unlike the other pLLM-based PPI inference models here, however, Liu *et al.* train their own pLLM (MP-BERT) from scratch. Ko *et al.* developed TUNA, which uses features from ESM2-150M as input to a twin transformer-based neural network [158]. A final neural Gaussian process layer is used to predict uncertainty and provide interval estimates [158, 159].

## The Issue of Generalization in PPI Inference Models

In a 2012 letter to *Nature Methods*, Park and Marcotte sounded the alarm about a pervasive source of data leakage among methods which purport to make accurate inferences on paired biological data such protein-protein or protein-drug interactions [13]. Data leakage occurs when data or information from the testing set appears in the training set [160]. The implication being that the cross-validation being performed is invalidated by the lack of independence between the training and testing datasets [161, 162].

This oversight in the preparation of evaluation datasets had caused methods to appear as they had made accurate predictions of the interaction status of protein pairs, but instead had gravely overfitted to the training set and were incapable of generalizing to real-world use-cases. So severe was this masking that models which were evaluated under the flawed scheme which appeared to achieve AUROC scores as high as 0.82±0.01 were in reality scoring 0.58±0.03 when evaluated on a corrected dataset [13]. The collapse of metrics to near-random levels was consistent across seven models tested, all representative of methodologies common to the time.

As described in Section 2, significant changes in the adopted methodologies for PPI inference between when Park and Marcotte published their warning in 2012 and today. The methods tested had largely been superseded by new ones based on deep learning (Section 2), each purporting to having achieved ever more accurate testing scores. A study in 2024 found, however, that leading methods failed to correct for data leakage in precisely the same way Park and Marcotte first described [15]. When re-evaluated on a corrected dataset, the familiar collapse of testing metrics to near-random levels is replicated once more.

Notable exceptions among methods of this era are RAPPPID and INTREPPPID, which were developed with a focus on generalization and minimizing data leakage during evaluation [10, 9]. Both methods far out-performed their contemporaries on datasets which control for data leakage as result.

Around the same time as pLLM adoption for PPI inference grew, so did awareness of the challenges that Park and Marcotte first described. This resulted in these methods increasingly adopting stricter evaluation datasets which control for data leakage [158, 157]. The adoption of pre-trained pLLMs, however, presented a new potential source of data leakage. pLLMs commonly used for PPI inference are typically trained on datasets that are exhaustive and are representative of all known protein sequences. The extent and nature of the data leakage caused by the incorporation of pre-trained pLLMs in PPI inference models is explored in length by a study of which I am an author and whose text is reproduced in Chapter **??** [163]. In that same study, I demonstrate that current PPI inference models, including those which do not incorporate pLLMs, fail to generalize in multiple practical real-world application. When evaluated on a dataset of protein interactions between proteins which originate from the SARS-CoV-2 virus and human proteins, all PPI inference models tested failed to make accurate predictions, obtaining scores which are consistent with random classifiers [163]. Likewise, when faced with pairs comprised of sequences with small mutations, the inferred probability of interaction computed by models had no correlation with their true changes in binding affinity [163]. Together, these results indicate that many challenges remain for PPI inference, particularly with respect to their ability to generalize.

# Background and Context for "*GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks*"

In Chapter 2, I explored both the landscape of PPI inference methodology (Section 2), as well as their persistent and pervasive problem of these methods failing to generalize to out-of-distribution testing samples (Section 2). It is in this context that, in 2021, my supervisor and I submitted for publication RAPPPID. Prior to RAPPPID few PPI inference models, and no deep learning techniques to my knowledge, were evaluated on datasets which appropriately controlled for data-leakage. Indeed, when we tested existing PPI inference models at the beginning of RAPPPID's development, it became clear that models (particularly those using deep-learning methods) were still failing to generalize as Park and Marcotte observed in 2012 and Bernett *et al.* would go on to observe in 2024 [13, 15].

RAPPPID is a PPI inference model which was created to answer the question: could conventional deep PPI inference models be modified to overcome their failure to generalize if a focus on regularization was applied?

Inspired by then-recent studies, we began developing a RNN-based PPI inference model which incorporated LSTM-specific regularization techniques outlined by Merity *et al.* in 2017 [164]. Merity *et al.*'s AWD-LSTM is a carefully tuned approach to regularizing and optimizing RNNs which take into consideration how traditional approaches sometimes are inappropriate for RNNs.

As introduced in Subsection 2, a common problem encountered when applying RNN architectures is that of vanishing gradients. RNN variants like LSTM and GRU, which help to lessen the negative effects of large sequences on training stability, succumb under a number of time-steps which is even still below the average protein sequence length. While other RNN-based PPI inference methods address this limitation through the use of CNNs to greatly reduce the length of the sequence, we identified a few problems with this approach.

Firstly, by initially filtering sequences through the use of CNNs and pooling layers, the resolution of the time-steps is greatly reduced. By reducing the resolution, the LSTM's ability to model long-term dependencies is rendered useless, thereby squandering an opportunity to model the non-linear nature of PPI contact surfaces in protein sequences. Secondly, the addition of convolutional layers added to a very constrained parameter budget. A very small number of PPI training pairs and a dataset which controls for data leakage makes overfitting both likely and easy to detect. For this reason, the number of parameters that make up the RAPPPID was carefully selected.

Ultimately, use of the Unigram tokenization algorithm via the Sentencepiece library allowed us to increase training stability while never bringing the resolution lower than three amino acids, grouped according to their common co-occurence [136]. Additionally, rather than contributing to overfitting, this tokenization approach had a regularization effect due to a process called "sub-word regularization".

The final result is a model which, achieved then-state-of-the-art accuracy on out-of-distribution evaluation datasets [10]. The regularization, optimization, and training modifications as applied to a fairly conventional deep PPI inference architecture are responsible for RAPPPID's performance, as ablation experiments show [10].

RAPPPID is proof positive that, despite a literature replete with PPI models that fail to generalize, it is possible to improve performance on out-of-distribution evaluation sets of simple architecture through thoughtful application of regularization and modification to the training process.

The contents of Chapter 3 were first published on the 30th of June, 2022 as an article of the same title in Volume 38, Issue 16 of Oxford University Press' *Bioinformatics* journal and is made available under the Creative Commons Attribution License, © Joseph Szymborski and Amin Emad, 2022. DOI: 10.1093/bioinformatics/btac429

# Chapter 3

# GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks

## Abstract

We introduce GRouNdGAN, a gene regulatory network (GRN)-guided reference-based causal implicit generative model for simulating single-cell RNA-seq data, in silico perturbation experiments, and benchmarking GRN inference methods. Through the imposition of a user-defined GRN in its architecture, GRouNdGAN simulates steady-state and transient-state single-cell datasets where genes are causally expressed under the control of their regulating transcription factors (TFs). Training on six experimental reference datasets, we show that our model captures non-linear TF-gene dependencies and preserves gene identities, cell trajectories, pseudo-time ordering, and technical and biological noise, with no user manipulation and only implicit parameterization. GRouNdGAN can synthesize cells under new conditions to perform in silico TF knockout experiments. Benchmarking various GRN inference algorithms reveals that GRouNdGAN effectively bridges the existing gap between simulated and biological data benchmarks of GRN inference algorithms, providing gold standard ground truth GRNs and realistic cells corresponding to the biological system of interest.

## Introduction

Unraveling gene regulatory interactions, often represented as a gene regulatory network (GRN), plays a crucial role in studying biological processes under different conditions[1,2,3], simulating knockdown and knockout experiments[4,5], and identifying therapeutic drug targets[6,7]. Many al-

gorithms have been proposed for GRN reconstruction using bulk or single-cell RNA sequencing data (scRNA-seq), alone[8,9,10,11,12,13,14] or with other modalities[15,16,17,18]. While these advances have provided great biological insights, evaluating the performance of GRN Inference algorithms remains challenging[19,20] due to the lack of reliable ground truth for the biological processes under study. Existing evaluation approaches often resort to curated databases[21,22,23]. However, the regulatory interactions in these databases are aggregated from a wide range of datasets and are not specific to a biological system, making them not ideal benchmarks due to context specificity of gene regulation. Another strategy is to verify regulatory interactions by conducting perturbation experiments on the system under study[20]. However, this approach is tedious, lengthy, and expensive. Another approach is to employ scRNA-seq simulators. Despite great progress in this domain, most simulators lack the essential properties for this task, such as the preservation of gene identities and simulation based on a user-provided ground truth GRN. For example, scGAN[24], cscGAN[24], scDESIGN2[25], and SPARSIM[26] capture inter-gene correlations in their datasets. However, since they do not explicitly impose a known GRN (that can act as the ground truth), they are not suitable for benchmarking GRN inference methods.

A small subset of simulators (e.g., BoolODE[19], SERGIO[4], and GeneNetWeaver (GNW)[27]) explicitly incorporate GRNs capturing transcription factor (TF)-gene dynamics. GNW was used to generate benchmarks for Dialogue for Reverse Engineering Assessment and Methods (DREAM) challenges[27,28,29]. However, being a bulk RNA-seq tool, its simulated datasets do not replicate the distribution of experimental scRNA-seq datasets nor exhibit their statistical properties, despite attempts to adapt it for this purpose by externally inducing dropout events[10,30]. SERGIO and BoolODE were designed to simulate scRNA-seq data using stochastic differential equations (SDEs)[4,19] and have been used to benchmark a variety of GRN inference methods. However, often a mismatch between the benchmarking results based on experimental and their simulated data have been reported, which may be attributed to differences between the simulated and experimental datasets. For example, in the BEELINE study (using BoolODE), some of the top ranking methods on simulated benchmarks reported near random performance on curated and experimental benchmarks[19]. Both simulators enable the user to simulate more realistic datasets by carefully selecting the values of the SDE parameters. Moreover, using a reference dataset, SERGIO allows fine tuning the added technical noise in an iterative procedure until the generated and reference datasets are matched[4]. While these steps can help improve the resemblance to real datasets, they are often suboptimal and pose an undue burden on the user: for example, in SERGIO, the user must evaluate the similarity based on five different statistics and change three parameters iteratively until desired resemblance is achieved (which itself can be subjective). In addition, since the GRN is imposed on the "clean" dataset, but technical noise is added afterwards, this step may change the encoded causal relationships in a non-trivial way (which may explain why the performance of GRN inference methods dropped close to random when applied to the noisy dataset[4]). Furthermore, SERGIO and BoolODE do not readily pre-

serve gene identities and make simplifying assumptions regarding cooperative regulation (see Discussion). These shortcomings demonstrate the need for simulators capable of generating realistic scRNA-seq data that retain the regulatory dynamics specified by a user-defined GRN. Importantly, with the growing interest in causal inference, simulators that impose causal GRNs are of great need.

GRouNdGAN (GRN-guided in silico simulation of single-cell RNA-seq data using Causal generative adversarial networks) is a causal implicit generative model for reference-based GRN-guided simulation of scRNA-seq data inspired by CausalGAN[31]. Given an input GRN and a reference scRNA-seq dataset, it can be trained to generate simulated data that is both indistinguishable from the reference data and faithful to the causal regulatory interactions of the input GRN. Unlike model-based simulators that rely on simplifying assumptions regarding co-regulatory patterns, in GRouNdGAN these patterns are *learned* through complex functions. This allows it not to compromise on the underlying complexity of the system and to model elaborate regulatory dynamics. GRouNdGAN provides state-of-the-art performance in realistic scRNA-seq data generation, while preserving gene identities, causal gene regulatory interactions, and cellular dynamics (e.g., lineage trajectory and pseudo-time ordering). This is achieved through implicit parameterization without the need for manual fine-tuning. Using GRouNdGAN, we benchmark eight GRN inference methods and find the results to be aligned with BEELINE's experimental results[19]. Furthermore, the causal structure of GRouNdGAN enables it to be used for sampling from both interventional and observational data distributions, enabling in silico knockout experiments.

## Results

### GRouNdGAN generates scRNA-seq data using causal generative adversarial networks

GRouNdGAN is a deep learning model that generates scRNA-seq data while imposing a user-defined causal GRN to describe the regulatory relationships of the genes and TFs. Its architecture builds on the causal generative adversarial network[31] and includes a causal controller, target generators, a critic, a labeler and an anti-labeler (Fig. 1). Training includes two steps. First, the causal controller that generates TF expression values is pre-trained (Fig. 1B) as the generator of a Wasserstein GAN (WGAN)[32] with gradient penalty (WGAN-GP) (Methods). In the second step (Fig. 1C), TF expression values (generated by the pre-trained causal controller) and randomly generated noise are provided as input to the target generators that produce target genes' expressions, while incorporating the TF-gene relationships of the input GRN. To achieve

this, as input, each target generator only accepts a noise value and the generated expression values of TFs that causally regulate it (Fig. 1C, Methods). The generated expression of genes and TFs are then fed to a library-size normalization (LSN) layer[24].

The two steps of training above are performed based on the same reference experimental scRNA-seq training dataset. The goal of the first step (pre-training, Fig. 1B) is to train the causal controller to learn scRNA-seq data distribution and generate realistic data, irrespective of any GRN. The GRN edges between a set of regulators (e.g., TFs) and their targets are imposed in the second step of training (Fig. 1C). To achieve this, in the second step of training, only the expression of regulators (TFs) outputted by the causal controller are used and its output of target genes' expressions are discarded and instead are re-generated using the target generator neural networks in Fig. 1C to enable imposition of causal relationships.

The critic's role (in both steps of training) is to quantify the Wasserstein distance (Supplementary Notes) between the reference and simulated data. The target generators are trained in an adversarial manner to generate realistic datapoints indistinguishable from the reference datapoints by the critic. The labeler/anti-labeler estimate the TF expression values *only* from the target genes' expressions to ensure that the generated causal TF-gene dependencies are encoded. The combination of the model architecture (in that each target generator only receives a noise value and the simulated expression of its regulating TFs in the GRN) and the labeler/anti-labeler ensure that the GRN edges are causally imposed in the simulated data. Details are provided in Methods, the architectural choices in Supplementary Tables 1 and 2, and an ablation study in Supplementary Notes and Supplementary Table 3.

## GRouNdGAN generates realistic scRNA-seq data

We first trained GRouNdGAN on three datasets (results for three additional datasets will be discussed later). The first dataset contained scRNA-seq profiles of 68,579 human peripheral blood mononuclear cells (PBMCs) from 10x Genomics[33] corresponding to eleven cell types ("PBMC-All"). We formed a dataset of the most common cell type in the PBMC-All dataset containing 20773 CD8+ Cytotoxic T-cells ("PBMC-CTL"). Additionally, we obtained the scRNA-seq (MARS-seq) profile of 2730 cells corresponding to differentiation of hematopoietic stem cells to different lineages from mouse bone marrow[34] ("BoneMarrow"). We used GRNBoost[28] to identify fifteen TFs for each gene to form the input GRN. Note that these identified interactions may contain both spurious correlations and causal relationships in the reference dataset. However, when they are used as input to GRouNdGAN, they will correspond to causal interactions in the *simulated* dataset (see Kocaoglu et al. for theoretical details of causality in this architecture[31]). For each dataset, we trained GRouNdGAN on a randomly selected training set and evaluated on a held-out test set (Methods, Supplementary Data 1 –

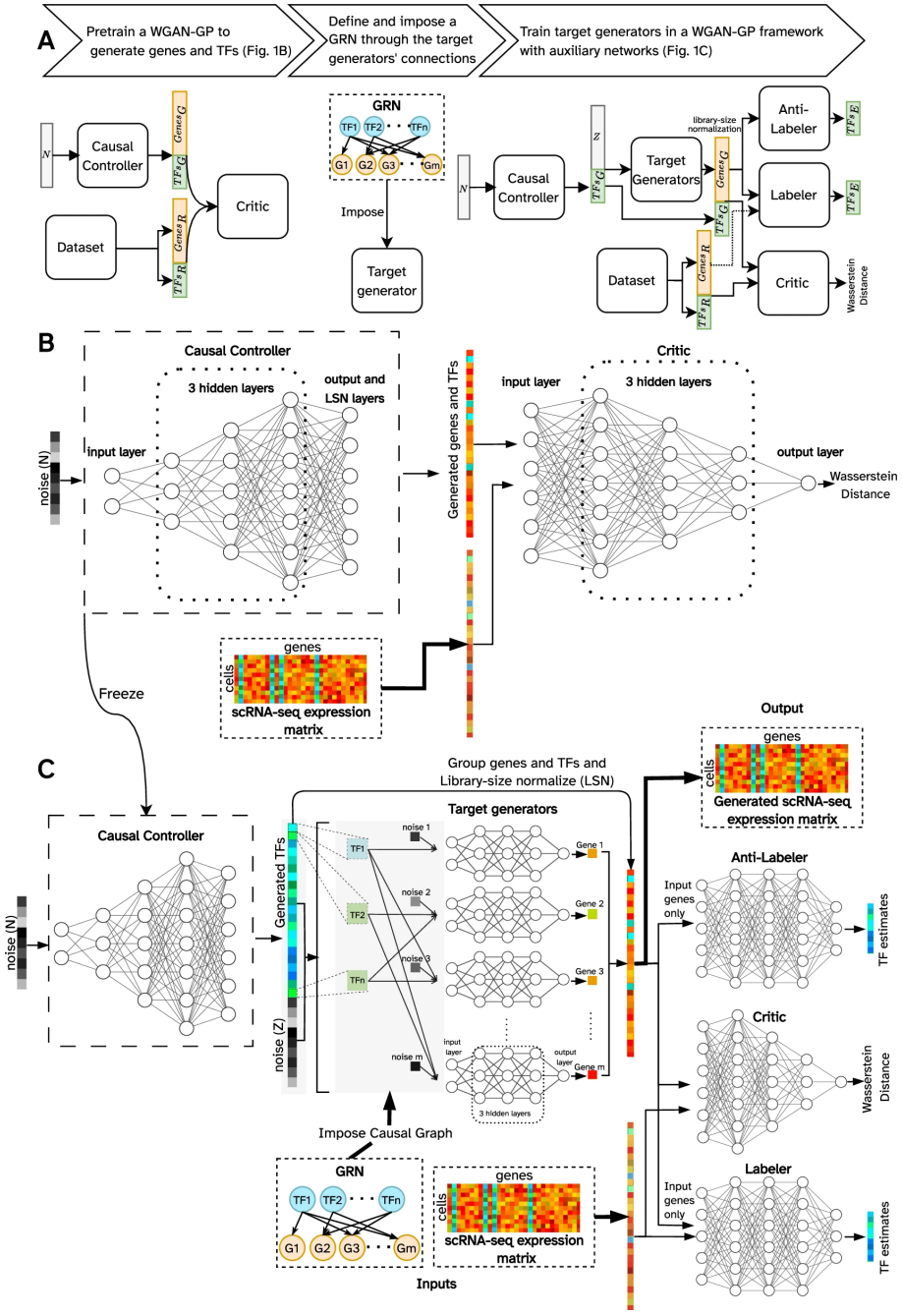



**Figure 1: Architecture and training procedure of GRouNdGAN.** **A** The flowchart representing the steps of the training procedure and the overall architecture of the model in each step. Subscripts G, R, and E represent generated, real, and estimated, respectively. **B** A WGAN-GP is pre-trained to generate realistic simulated cells using the reference (real) training set. **C** The LSN layer of the generator of the trained WGAN-GP (panel **B**) is removed, its weights are frozen, and is used as the causal controller to generate unnormalized TF expression values (expression of target genes generated by the causal controller are discarded). These TF expression values along with a noise vector are provided as input to the target generators, following the provided causal GRN. The generated gene and TF expression values are reorganized and passed through the LSN layer. The normalized simulated expression vectors and experimental reference data (the same training set as **B** are then passed to the critic to estimate Wasserstein distance between the reference and the generated data distributions. The anti-labeler estimates TF values based on generated target gene expressions. The labeler performs a similar task, but in addition to receiving generated values, it also utilizes target gene expression values from the reference data. Labeler and anti-labeler ensure that the causal GRN is incorporated by the target generators. Details of the model are provided in Methods.

**Table 1: Performance of different simulators in generating realistic scRNA-seq data using the PBMC-CTL dataset**

| Simulator | Cosine distance | Euclidean distance | MMD | RF AUROC | miLISI |
|---|---|---|---|---|---|
| GRouNdGAN | **0.00057** | **182** | **0.026** | **0.54** | **1.891** |
| scGAN[24] | 0.00095 | 222 | 0.031 | 0.59 | 1.888 |
| scDESIGN2[25] | 0.00100 | 229 | 0.065 | 0.76 | 1.736 |
| SPARsim[26] | 0.00104 | 235 | 0.309 | 0.95 | 1.625 |
| Control | 0.00019 | 99 | 0.012 | 0.50 | 1.909 |

The metrics are calculated between a simulated dataset of 1000 cells and the held-out test set of 1000 real cells (see Supplementary Data 1 – Sheet 2 for training set performance). Each gene in the imposed GRN of GRouNdGAN is regulated by 15 TFs (constructed using GRNBoost2 from the experimental training set). For the first three metrics, a value closer to zero is preferred, for RF AUROC a value closer to 0.5 is preferred, and for miLISI a value closer to 2 is preferred. For the first two metrics, the values correspond to the distance of the mean centroids of the real and simulated cells. The RF AUROC of control corresponds to perfect performance (of a random classifier). The other control metrics are calculated using the two halves of the real test dataset. Best performance values (excluding control) are in bold-face.

Sheets 2 and 3). Figure 2A, B and Supplementary Figs. 1–3 show the t-SNE plots of reference and simulated cells, qualitatively revealing their similarity.

We quantitatively assessed the resemblance using Euclidean distance, Cosine distance, maximum mean discrepancy (MMD)[35], mean integration local inverse Simpson's index (miLISI)[36], and the area under the receiver operating characteristic curve (AUROC) of a random forests (RF) classifier distinguishing simulated and experimental cells (Methods). As a "control" (and to enable calibration of these scores), we also calculated these metrics using two halves of the reference test set (corresponding to real cells). Table 1 shows the performance of GRouNdGAN, control, and three state-of-the-art simulators for the PBMC-CTL dataset (see Supplementary Data 1 – Sheet 2 for training set performance). We did not include BoolODE in these tables since it is a reference-free simulator and is not designed to match a particular reference dataset. We tried generating samples using SERGIO to match the PBMC-CTL dataset, but the simulated dataset did not show high resemblance to the PBMC-CTL dataset (see Supplementary Notes, Supplementary Tables 4 and 5 and Supplementary Fig. 4). However, we decided not to include these results in Table 1, since SERGIO relies on the user's judgement for distribution matching and the quality of the final simulated dataset is user dependent. Instead, we only included reference-based simulators that automatically generate scRNA-seq data resembling experimental data, in order to assess data quality systematically and fairly.

One of the challenges faced by GANs is the possibility of mode collapse: when applied to heterogenous datasets, they may learn to generate only a limited set of examples, instead of generating a variety of examples from the entire distribution of the training data. We trained GRouNdGAN using PBMC-All, which contains multiple cell types, to determine whether it

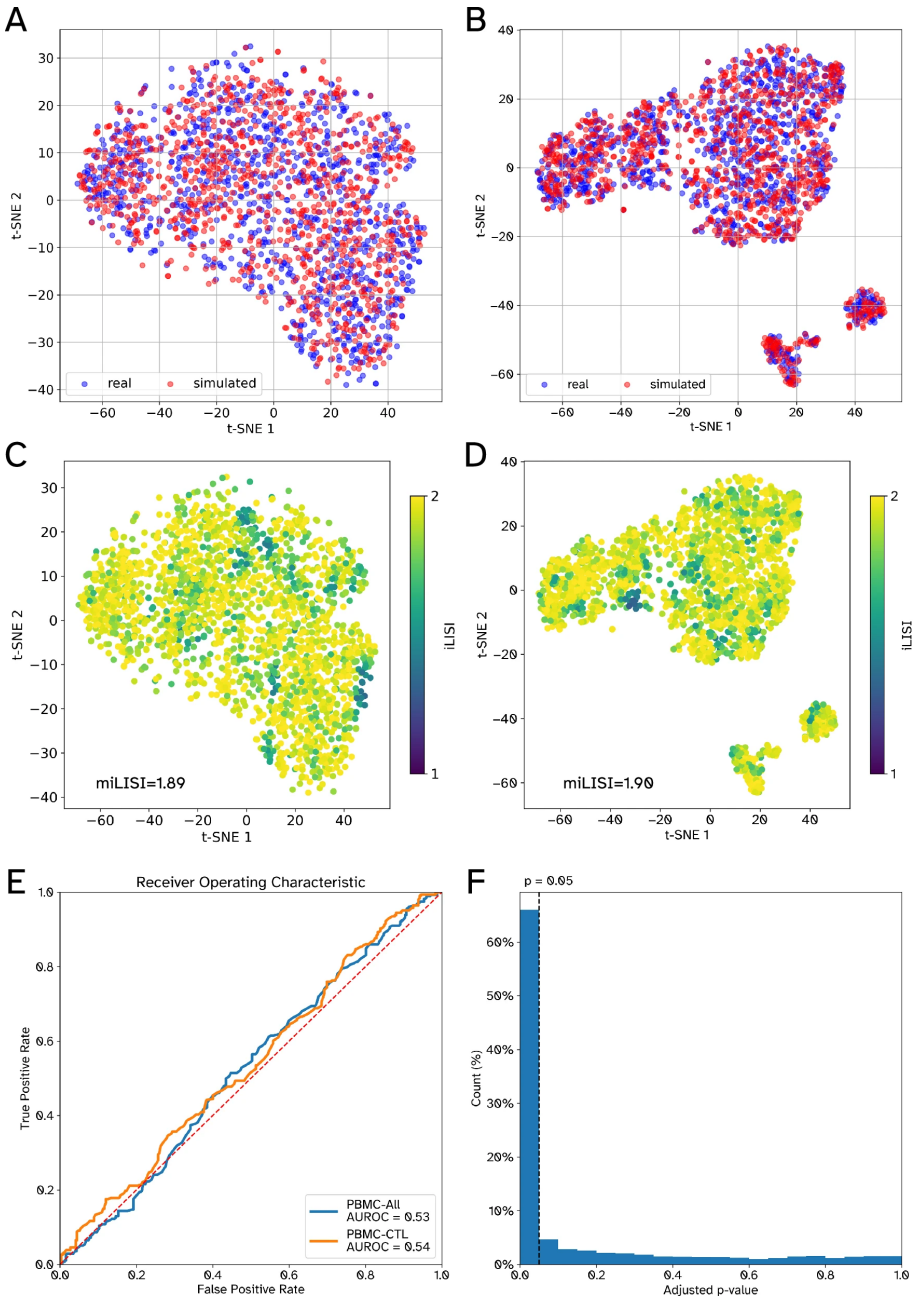



**Figure 2: Performance of GRouNdGAN in generating realistic scRNA-seq data.** All plots correspond to 1000 real (experimental) held-out test cells and 1000 simulated cells. Each gene in the imposed GRN of GRouNdGAN is regulated by 15 TFs (identified using GRNBoost2 from the experimental training set). Panels **A** and **B** depict the t-SNE plots of real (blue) and simulated (red) cells for PBMC-CTL and PBMC-All, respectively, while panels **C** and **D** also show their iLISI distributions. **E** The ROC curve of a Random Forest classifier in distinguishing between simulated and real cells. **F** The distribution of adjusted $p$-values corresponding to the TF perturbation study on the PBMC-CTL dataset. The $p$-values are obtained using two-sided Wilcoxon signed ranked tests and are adjusted for multiple hypotheses following the Benjamini–Hochberg procedure. Source data are provided as a Source Data file.

can generate samples from different cell types present in this dataset, or it will suffer from mode collapse and only generates samples corresponding to the most represented cell type. This is particularly important, since many scRNA-seq datasets are heterogenous and contain different cell types or cells in different states. If cell type information of the reference dataset is available a priori, one can easily generate realistic samples of each cell type separately (similar to the PBMC-CTL analysis) and use either cell type-specific GRNs, a shared GRN, or a combination of both. However, we were particularly interested to know whether our model could generate realistic samples *without* this knowledge, a more challenging task and useful for when such information is unavailable. Also, not requiring cell type information reduces the amount of user involvement (to annotate cell types) and improves model's usability. We repeated the analysis using the PBMC-All dataset, containing eleven cell types. Although GRouNdGAN did not receive cell type/cluster information, we did not observe mode collapse (Supplementary Fig. 2), and it was able to generate cells from distinct clusters (Fig. 2B, test set miLISI = 1.90). GRouNdGAN outperformed all simulators that did not use cell cluster information (Table 2) and compared to those that utilized this information, it was still the top performing based on MMD and RF AUROC and the second best according to the Euclidean distance (6% higher than the best) and miLISI (0.5% lower than the best) (Table 2). Moreover, we observed a high degree of concordance between the cell type marker expressions in the experimental data and GRouNdGAN-simulated data (Supplementary Notes and Supplementary Fig. 5). Finally, GRouNdGAN outperformed all other simulators based on all metrics when we repeated the analysis using the BoneMarrow dataset, containing continuous cell states and much fewer cells (Supplementary Data 1 – Sheet 3). These results show that GRouNdGAN is a stable simulator of scRNA-seq data and even without extra cell type information in heterogenous datasets, it automatically generates realistic samples reflecting different cell types (while imposing a GRN). See Supplementary Notes and Supplementary Data 2 for stability analysis and the effect of different GRN properties (including number of TFs) on the performance.

## GRouNdGAN imposes a causal GRN in the simulated data

To assess the ability of GRouNdGAN in imposing the input causal GRN, we performed in silico TF knockout experiments (one of GRouNdGAN's capabilities) on the simulated cells ($n = 1000$) from the PBMC-CTL dataset. We performed a forward pass after setting the expression of each TF to zero (one at a time) in the causal controller output, while keeping all other parameters and TF expressions unchanged, ensuring that the perturbations were performed on the same batch of cells forming matched case/control experiments. There was no change in the expression of genes that were not regulated by the knocked-out TF (as expected), while the expression of the regulated genes changed. Figure 2F shows the distribution of the adjusted $p$-values (Benjamini–Hochberg) for each TF-gene edge in the GRN, comparing the expression of a gene across all cells before

**Table 2: Performance of different simulators in generating realistic scRNA-seq data using the PBMC-All dataset**

| Simulator | Cell cluster labels/ratios provided as input | Cosine distance | Euclidean distance | MMD | RF AUROC | miLISI |
|---|---|---|---|---|---|---|
| GRouNdGAN | No | 0.00028 | **168** | **0.017** | **0.53** | 1.90 |
| cscGAN[24] | Yes | 0.00025 | 242 | 0.030 | 0.56 | **1.91** |
| cWGAN | Yes | 0.00053 | 239 | 0.027 | 0.57 | 1.89 |
| scGAN[24] | No | 0.00081 | 300 | 0.035 | 0.58 | 1.90 |
| scDESIGN2[25] | Yes | 0.00032 | 181 | 0.046 | 0.86 | 1.81 |
| scDESIGN2[25] | No | 0.00057 | 256 | 0.124 | 0.91 | 1.53 |
| SPARsim[26] | Yes | **0.00019** | **158** | 0.287 | 0.96 | 1.66 |
| SPARsim[26] | No | 0.00035 | 245 | 0.307 | 0.96 | 1.39 |
| Positive control | NA | 0.00022 | 116 | 0.012 | 0.50 | 1.91 |

Baseline models that enable utilization of cell cluster labels or ratios are run with and without this information. The metrics are calculated between a simulated dataset of 1000 cells and the held-out test set of 1000 real cells. Each gene in the imposed GRN of GRouNdGAN is regulated by 15 TFs (constructed using GRNBoost2 from the experimental training set). For the first three metrics, a value closer to zero is preferred, for RF AUROC a value closer to 0.5 is preferred, and for miLISI a value closer to 2 is preferred. For the first two metrics, the values correspond to the distance of the mean centroids of the real and simulated cells. The RF AUROC of control corresponds to perfect performance (of a random classifier). The other control metrics are calculated using the two halves of the real test dataset. Best performance values (excluding control) are in bold-face.

and after the knockout of one of its regulating TFs (two-sided Wilcoxon signed-rank tests). In the majority (66.5%) of cases, the knockout of a gene's regulating TF significantly (adjusted $p$-value $< 0.05$) altered its expression, showcasing that the GRN is indeed imposed in the expression profiles of simulated cells. Note that since each gene is regulated by multiple TFs, we cannot expect that knockout of a single TF result in a significant change in the expression of its target genes in all cases. The analysis above shows that GRouNdGAN does not simply ignore the input TF values to instead rely solely on the noise input to generate realistic cells, and the effect of TF-gene edges are indeed imposed. Moreover, since each gene is regulated by multiple (15) TFs, but the knockout experiment is one TF at a time, this further shows that individual edges are imposed by the model.

GRouNdGAN does not require the input GRN to be the true causal GRN of the training data (we will discuss this further later). To further clarify this, we replaced GRNBoost2 with PPCOR[37] to form the GRN, which is a method based on (partial) correlation for inferring GRNs (see Supplementary Data 1 – Sheet 2 for resemblance metrics). Repeating the knockout experiments above ($n = 1000$ cells) resulted in similar results showing that in the majority (75.2%) of cases, the knockout of a gene's regulating TF significantly altered its expression (Supplementary Fig. 6). Taken together, these results show that GRouNdGAN can successfully impose different GRNs in the simulated data, while generating realistic cells.

## The imposed GRN can be reconstructed from the simulated data

To test whether the imposed GRN can be reconstructed from the generated data, we first applied GRNBoost[28] to the reference PBMC-CTL dataset and recorded the top ten TFs for each gene. For each gene, we created two sets of five TFs based on their even/odd parity in the ranked list: TFs ranked 1st, 3rd, 5th, 7th, and 9th were connected to the gene and were imposed ("positive control GRN"), while the remaining five TFs were not ("negative control GRN"), resulting in two GRNs with identical densities and the same number of TFs and target genes. When considering the positive and negative control GRNs as ground truths (separately), the GRN reconstructed using GRNBoost2 from the reference (training) dataset ($n = 19{,}773$) resulted in comparable AUPRC (0.28 and 0.24, respectively, Supplementary Data 3 – Sheet 2), reflecting that they contain edges of comparable importance.

We simulated data ($n = 19{,}773$ cells) using GRouNdGAN by imposing only the positive control GRN, and used GRNBoost2, PIDC[10] GENIE3[9], and PPCOR[37] to reconstruct the underlying GRN (Fig. 3, Supplementary Fig. 7, Supplementary Data 3 – Sheet 2). All methods reconstructed the imposed edges with a much higher AUPRC compared to the reference dataset, while they did not assign high scores to the unimposed edges, resulting in close to random AUPRC. Figure 3 shows the performance of GRNBoost2 applied to the reference, GRouNdGAN-simulated, and scGAN-simulated data ($n = 19{,}773$ cells). The performance for scGAN (both GRNs) was lower than the reference dataset, showcasing that it had not preserved the TF-gene relationships of the experimental dataset well. The performance was relatively consistent between positive (Fig. 3A, C) and negative control (Fig. 3B, D) GRNs using the reference and scGAN-simulated data; however, it was much higher for the positive control GRN (compared to negative control GRN) using GRouNdGAN-simulated data. Similar patterns were observed when we repeated these analyses using the PBMC-All ($n = 67{,}579$ cells) and BoneMarrow ($n = 2230$ cells) datasets (Supplementary Data 3 – Sheets 3 and 4, Supplementary Figs. 8 and 9). These results show that the imposed edges were accentuated by GRouNdGAN, while the unimposed edges were disrupted and could not be found by GRN inference methods.

Next, we asked whether GRNBoost2 can reconstruct the imposed GRN, if GRNs of different densities are used to simulate data. Using data for which each gene was regulated by 15, 10, 5, and 3 TFs, we observed that in all cases the GRN imposed by GRouNdGAN could be inferred from the simulated data (Supplementary Data 3 – Sheets 5 and 6). Finally, we repeated the controlled analysis above, swapping the role of positive and negative control GRNs. Similar to the results of Fig. 3, the imposed edges were inferred by GRNBoost2, while the unimposed edges were not accurately discovered (Supplementary Data 3 – Sheet 7). These results show that 1) GRouNdGAN imposes the causal GRN, 2) the GRN inference methods can identify the imposed edges, and 3) the TF-gene relationships present in the reference dataset but unimposed

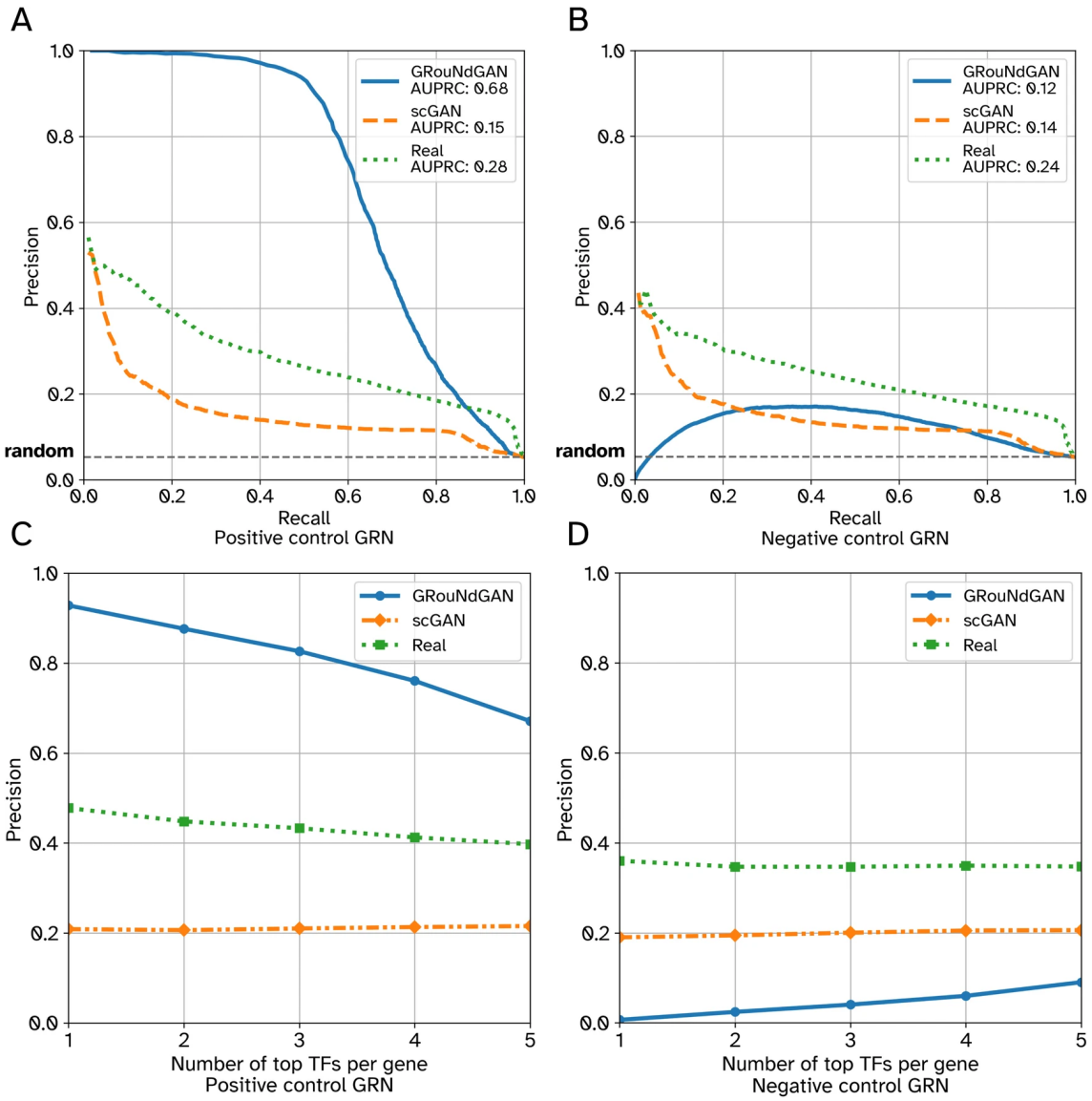



**Figure 3: Performance of GRNBoost2 in recovering the imposed edges versus unimposed edges (PBMC-CTL).** Left column shows the AUPRC (**A**) and Precision at k (per gene) (**C**) when the imposed edges (positive control GRN) were considered the ground truth. Right column shows the AUPRC (**B**) and Precision at k (per gene) (**D**) when the unimposed edges (negative control GRN) were considered the ground truth. Precision at k (per gene) refers to the precision when top k TFs for each gene is used to form the reconstructed GRN. All three datasets had the same number of cells ($n = 19{,}773$). Source data are provided as a Source Data file.

by GRouNdGAN are disrupted during simulation. The last property is particularly desirable for benchmarking GRN inference methods to ensure that the regulatory relationships present in the reference dataset (but unimposed) do not bias the simulated data, which could result in an inflated false positive rate.

Next, we repeated the analyses above on the PBMC-CTL dataset, but this time with input GRNs learned by PPCOR (a GRN inference based on partial correlation) from the training set. Similar to the results above, GRNBoost2 could reconstruct the imposed edges, and the importance

of the imposed edges were accentuated by GRouNdGAN (compared to the original training dataset) (Supplementary Data 3 – Sheet 8). We also imposed two randomly generated GRNs (with similar density to the GRNs above in which each gene was regulated by 5 TFs). Once again, the imposed edges were accentuated by GRouNdGAN, however imposing a random GRN (whose induced patterns may be inconsistent with real reference data) came at the cost of lower resemblance between simulated and reference data. (See Supplementary Notes and the Discussion for more details on why imposing a GRN that is inconsistent with the reference data (e.g., a random GRN) is a contradictory requirement to generating simulated data resembling the reference data).

## GRouNdGAN-simulated data preserves trajectories and can be used for pseudo-time inference

In addition to deciphering discrete states, scRNA-seq data is often used to determine continuous cell transitions during biological processes such as differentiation, using trajectory and pseudo-time inference[38,39,40,41,42,43,44]. We set to determine whether GRouNdGAN-simulated data conforms to the transitional states and pseudo-time of the reference data. This is particularly important for benchmarking GRN inference methods that utilize pseudo-time information[12,13,14]. For this purpose, we used the BoneMarrow dataset corresponding to differentiation of hematopoietic stem cells to different lineages and simulated $n = 2230$ cells using GRouNdGAN (same as the number of real cells in the reference training dataset, to remain consistent).

We used Partition-based graph abstraction (PAGA)[38] for trajectory inference of the reference and GRouNdGAN-simulated data. PAGA computes graph-like maps of data manifolds faithful to the topology of data, retaining its continuous and discrete structures. Supplementary Fig. 10 shows the PAGA graphs, where nodes capture discrete states and the edges signify transitions between them. To compare the identified trajectories, we used markers of the cell types present in this dataset (Supplementary Table 6)[38]. Figure 4 shows the expression patterns of two marker genes for erythroid cells (*Gata1*, *Klf1*), neutrophils (*Mpo*, *Ctsg*), and basophils (*Mcpt8*, *Prss34*) for PAGA trajectory graphs of both datasets (Supplementary Figs. 11–17 show marker genes of all cell types). The comparison showed similar activation patterns between corresponding regions of PAGA graphs of the experimental and GRouNdGAN data, revealing several trajectories (e.g., erythroid and neutrophil branches). Strikingly, even the number of nodes activated in each figure scaled similarly between the two datasets: e.g., the markers of basophils were consistently expressed in only one node in each dataset (Fig. 4E, F).

We annotated different cell types present in both datasets using these marker genes (following PAGA's official tutorial) (Fig. 5A, B). The GRouNdGAN's annotated graph showed similar

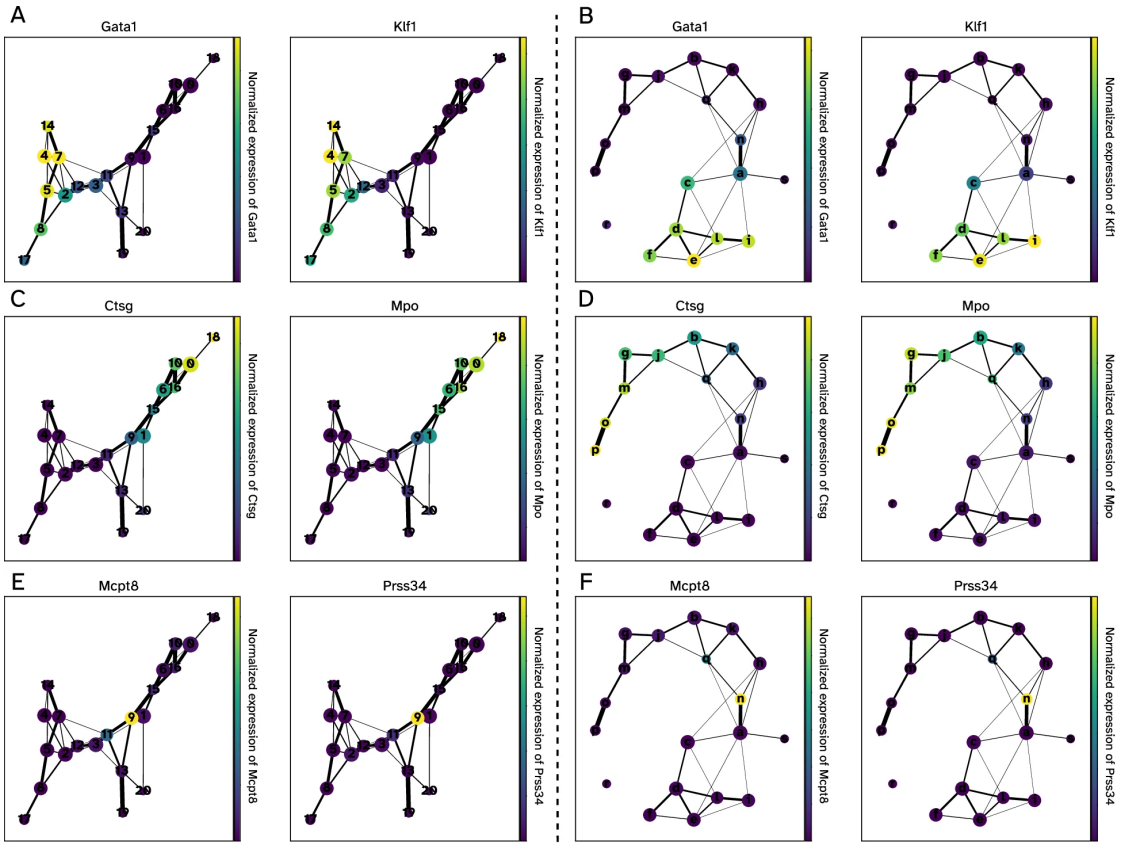



**Figure 4: PAGA graphs and cell type marker gene activation patterns in experimental and simulated BoneMarrow dataset.** In each figure, nodes represent Louvain clusters (capturing discrete states), edges represent transitions between states, and edge weights indicate confidence in the presence of connections. Low-connectivity edges below a threshold of 0.01 were discarded. Left panels (**A**, **C**, **E**) show the PAGA graph computed on GRouNdGAN-generated cells, and right panels (**B**, **D**, **F**) show the PAGA graph for the original BoneMarrow dataset. Node colors represent normalized expression of marker genes for erythroid cells (**A**, **B**), neutrophils (**C**, **D**), and basophils (**E**, **F**). Source data are provided as a Source Data file.

topological features to that of the reference data and captured known biological properties of hematopoiesis, which were also noted by Wolf et al.[38] using the reference dataset (e.g., proximity between monocytes and neutrophils or association between megakaryocyte and erythroid progenitors). These observations show that the key characteristics of the reference data are retained in GRouNdGAN-simulated data.

We used the diffusion pseudotime algorithm[39] (in scanpy[45]) to infer the progression of cells through geodesic distance (Fig. 5C, D). We noticed strong concordance when examining pseudotime values assigned to cell subgroups in the reference and GRouNdGAN-generated data. To track these patterns, we focused on marker expression changes along erythroid, neutrophil, and monocyte trajectories (Supplementary Figs. 11–13), similar to the analysis performed by Wolf et al.[38]. In both datasets, the activation of neutrophils' marker, *Elane*, and monocytes' marker, *Irf8*, were predominantly observed towards the later stages of their trajectories (Fig.

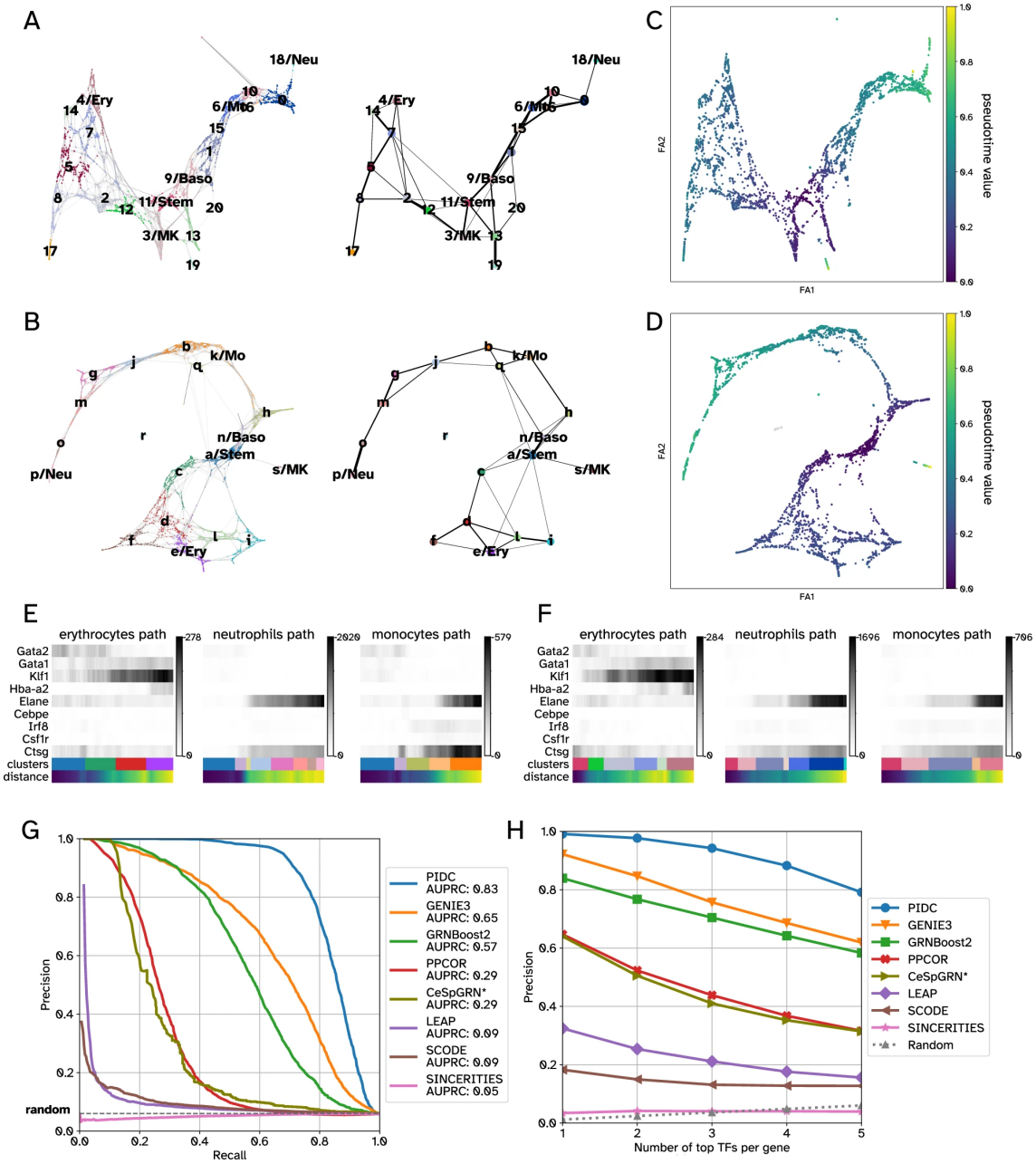



**Figure 5: Consistency in trajectories and pseudo-temporal orderings between data generated by GRouNdGAN and the experimental BoneMarrow hematopoietic dataset.** Panels **A** and **B** show the cell type annotations of simulated and experimental data, respectively. The figures on the left show the force-directed graphs (obtained using ForceAtlas2[66]) and the figures on the right show the PAGA graphs. Panels **C** and **D** show the PAGA-initialized force-directed graphs with cells colored by their inferred pseudo-time value. The following abbreviations are used: Stem for stem cells, Ery for erythroid cells, Neu for neutrophils, Mo for monocytes, MK for Megakaryocytes, and Baso for Basophils. Panels **E** and **F** display expression changes in the marker genes of erythrocyte (*Gata2*, *Gata1*, *Klf1*, and *Hba-a2*), neutrophil (*Elane* and *Cebpe*), and monocyte (*Irf8*, *Csf1r*, and *Ctsg*) branches for experimental (**E**) and simulated data (**F**). The distance refers to the geodesic distance from a root cell (here a stem cell). A darker shade of gray shows higher expression. Panels **G** and **H** show the performance of different GRN inference methods on data generated by GRouNdGAN using the BoneMarrow dataset. The benchmarking results are based on a dataset with $n = 2230$ simulated cells (except for CeSpGRN). Source data are provided as a Source Data file. * Due to the high memory requirement of CeSpGRN, we were only able to run it on a subset of data containing $n = 1000$ cells and $m = 100$ genes (the set of regulating TFs and the edges connecting them to these genes remained unchanged).

**Table 3: Performance of GRouNdGAN in generating realistic scRNA-seq data using the Dahlin, Tumor-All, and Tumor-malignant datasets**

| | Dahlin | | Tumor-All | | Tumor-malignant | |
|---|---|---|---|---|---|---|
| | **Training** | **Testing** | **Training** | **Testing** | **Training** | **Testing** |
| Cosine distance | 0.00011 | 0.00012 | 0.00141 | 0.00061 | 0.00059 | 0.00098 |
| Euclidean distance | 51 | 53 | 227 | 145 | 134 | 174 |
| MMD | 0.014 | 0.026 | 0.020 | 0.021 | 0.014 | 0.015 |
| RF AUROC | 0.52 | 0.54 | 0.53 | 0.52 | 0.53 | 0.57 |
| miLISI | 1.91 | 1.91 | 1.90 | 1.89 | 1.90 | 1.89 |

Each gene in the imposed GRN of GRouNdGAN is regulated by 15 TFs (constructed using GRNBoost2 from the experimental training set). For the first three metrics, a value closer to zero is preferred, for RF AUROC a value closer to 0.5 is preferred, and for miLISI a value closer to 2 is preferred. The metrics are calculated between a simulated dataset of 1000 cells and a set of 1000 real cells (all cells in the test set and 1000 randomly selected cells from the training set). For the first two metrics, the values correspond to the distance of the mean centroids of the real and simulated cells.

5E, F). Additionally, we observed the activation of erythroid maturity marker genes *Gata2*, *Gata1*, *Klf1*, and *Hba-a2* roughly in sequential order along the erythroid trajectory in both datasets (Fig. 5E, F), concordant with previous findings[38]. The striking similarity between the activation patterns of these markers in both datasets and the preservation of their ordering highlights GRouNdGAN's ability to capture dynamic transcriptional properties of scRNA-seq data, leading to correct trajectory inference and pseudo-time ordering.

## GRouNdGAN achieves high performance on other datasets

In addition to the datasets discussed earlier, we applied GRouNdGAN to three other datasets. One corresponded to cells undergoing hematopoiesis (similar to the BoneMarrow dataset), but with much larger number of cells ($n = 44{,}802$)[46] (which we called Dahlin dataset). The second dataset corresponded to malignant cells and cells present in the tumor microenvironment of 20 fresh biopsies from follicular lymphoma tumors ($n = 136{,}147$), which we called the Tumor-All dataset[47]. We also formed a dataset containing only the malignant cells present in this dataset ($n = 89{,}203$ cells), which we called the Tumor-malignant dataset. Table 3 and Supplementary Data 1 - Sheet 2 show the performance of GRouNdGAN in generating realistic scRNA-seq data, while Supplementary Figs. 18–20 show the tSNE plots of simulated and real cells. For all three datasets, different metrics show that the generated data has a high degree of similarity to experimental data and there is a small difference between the training and test set performances.

We next repeated the trajectory inference, pseudo-time ordering, and analysis of cell type marker expression along the pseudo-time axis on the Dahlin dataset using both GRouNdGAN-simulated data ($n = 43{,}802$ cells) and real cells ($n = 43{,}802$), as we had done using the BoneMarrow dataset

(Supplementary Fig. 21). Similar to the results from the BoneMarrow dataset, we observed a high degree of concordance between the activation patterns of cell type markers in the simulated and real datasets and the preservation of their orderings. Taken together, these results suggest that GRouNdGAN can be used to generate realistic data from different biological contexts, while imposing a causal GRN and preserving various biological properties.

## Benchmarking GRN inference methods using GRouNdGAN confirms prior insights from curated and experimental datasets

The BEELINE study[19] found conflicting results when benchmarking GRN inference methods on curated benchmarks and synthetic ones: top two performing models on synthetic data where among the bottom three on curated benchmarks. We re-investigated this using GRouNdGAN-simulated data with the BoneMarrow dataset and the positive and negative control GRNs (described earlier). We used seven GRN inference methods used by BEELINE, GRNBoost2[8], GENIE3[9], PIDC[10], PPCOR[37], LEAP[12], SCODE[13], and SINCERITIES[14], capturing a wide range of methods with a broad spread of performances reported on synthetic and curated benchmarks in the BEELINE study. We also included CeSpGRN[48], which is a cell-specific GRN inference method. We observed a striking resemblance between the performance pattern (order) of the methods on GRouNdGAN-simulated data and curated benchmarks from BEELINE (Fig. 5G, H and Supplementary Data 3 – Sheet 3 versus Fig. 4 of BEELINE[19]): the only difference was the swapping of LEAP and SCODE's order. We repeated this analysis using the PBMC-CTL dataset (Supplementary Data 3 – Sheet 2), obtaining similar patterns. We also tested these methods using the PBMC-All dataset (Supplementary Data 3 – Sheet 4). While the results were generally consistent, the order of GENIE3 and GRNBoost2 were swapped, which may be due to their ability in working with datasets containing multiple distinct cell types (also see Discussion).

Overall, PIDC[10] outperformed all methods using both datasets, followed by GENIE[39] and GRNBoost[28]. LEAP[12], SCODE[13] and SINCERITIES[14] (which use pseudo-time and could only be applied to the BoneMarrow dataset) performed worse than others, matching their behavior in the BEELINE study on curated benchmarks (but not on synthetic benchmarks). All methods performed close to random on unimposed negative control GRN edges. These results not only show that GRouNdGAN can be used for benchmarking GRN inference methods, but also shows that the insights obtained from it matches those obtained from curated and experimental benchmarks (such as those used in Figs. 4–6 of the BEELINE study), the formation of which requires extensive amount of resources and effort.

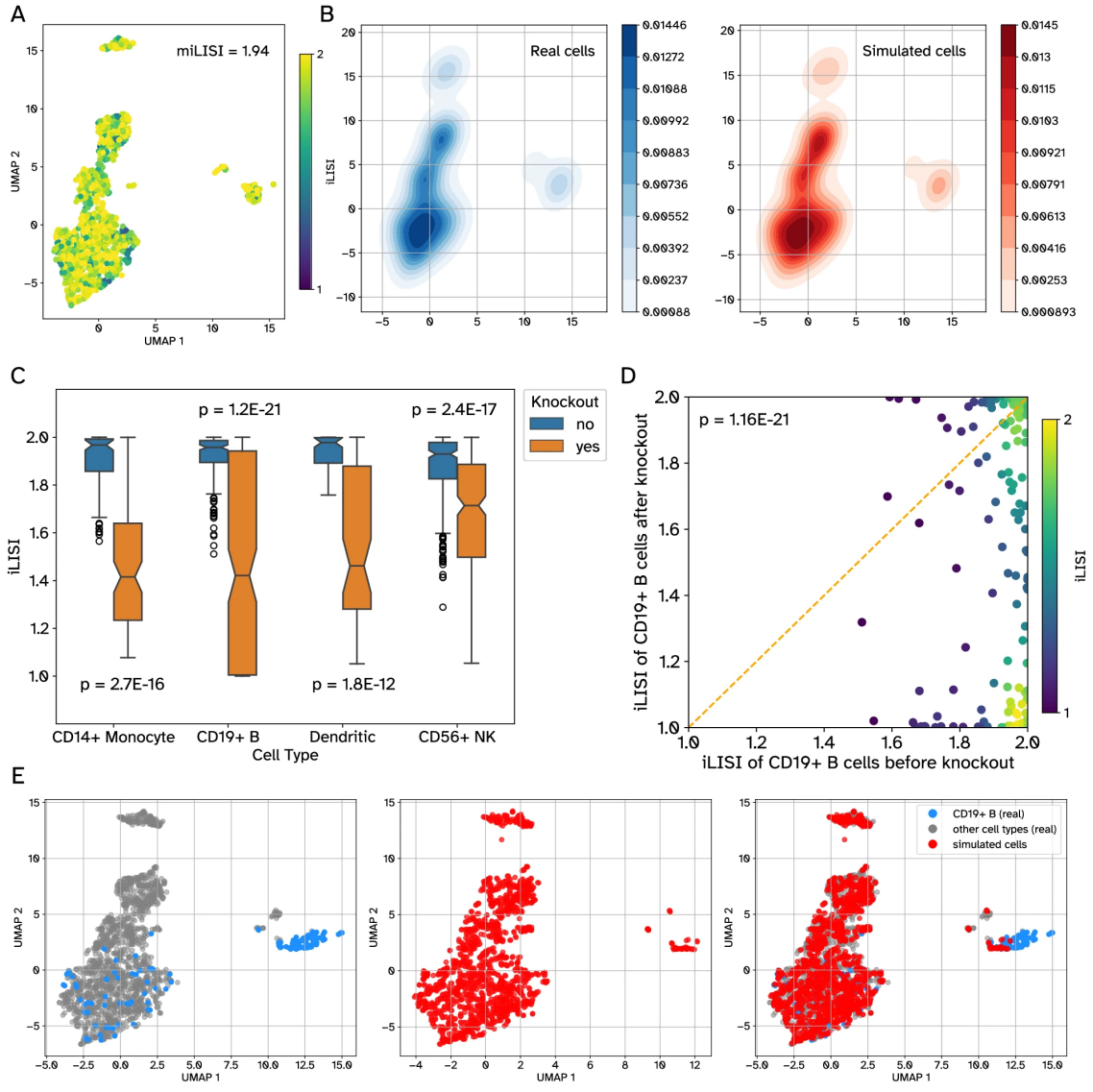



**Figure 6: TF knockout experiments and their effect on cell type generation.** **A** The UMAP shows the distribution of $n = 2000$ randomly selected cells from the experimental PBMC-All dataset. The color shows the iLISI value of each cell (miLISI = 1.94) calculated from a UMAP embedding, jointly obtained from the experimental cells and the same number of GRouNdGAN-generated cells. TFs were omitted as features when generating UMAP plots. In the simulated data, each gene was regulated by 15 TFs, identified using GRNBoost2. **B** Each plot shows the density of cells in the embedding space: the left plot (blue) corresponds to the real cells and the right plot (red) shows the simulated cells. **C** The boxplots show the distribution of iLISI values of $n = 2000$ real cells, calculated along with $n = 2000$ unperturbed simulated cells (blue) and with $n = 2000$ perturbed simulated cells (orange). For each cell type, top three most differentially expressed TFs were knocked out. The $p$-values reported in this figure were calculated using one-sided Wilcoxon signed rank tests. The center horizontal line corresponds to the median. Each box spans the interquartile range, and its limits show the upper (Q3) and lower quartiles (Q1). Whiskers extending from the boxes reach 1.5 times above and below the interquartile range. Outliers are shown individually if they fall outside the whiskers. **D** The scatter plot shows the iLISI values of CD19+B cells calculated along with unperturbed simulated cells (x-axis) and along with perturbed simulated cells (y-axis). The circles correspond to real cells and their colors reflect the density of datapoints in that region. The $p$-value reported in this figure was calculated using a one-sided Wilcoxon signed rank test. **E** The UMAP embedding of real cells (left), simulated cells after knockout of top TFs of CD19+B cells (middle), and all together (right). Source data are provided as a Source Data file.

### In silico perturbation experiments using GRouNdGAN

In many studies, one is interested to characterize the relationship between TFs' expression and phenotypic labels (e.g., cell types). Differential expression (DE) analysis is a common approach to identify TFs (or other genes) most associated with a specific cell type. However, to directly test whether perturbing the expression of such candidate TFs change the expression of genes representative of a cell type, perturbation experiments (e.g., TF knockout) are desired. As a causal implicit generative model, GRouNdGAN provides the capability to sample from interventional distributions, thereby enabling in silico perturbation experiments. This functionality is provided at inference time by enabling manipulation of TFs' expressions as the outputs of the causal controller. This change is propagated along the GRN (through the model architecture), impacting the expression of genes regulated by the perturbed TF(s). By enabling a deterministic mode of operation, GRouNdGAN allows for exact comparison of gene expressions before and after perturbation while maintaining the invariance of other parameters (Methods).

To test this functionality, we asked whether the knockout of top 3 TFs most differentially expressed (Mann Whitney U test) between each cell type and the rest (identified from the reference dataset) would result in less cells being generated in the vicinity of the cells of that type. If positive, this implies that the perturbation in the TFs' expression has modified the gene expression profile of generated cells so that they no longer resemble that particular cell type (confirming our expectation). Figure 6A shows the UMAP embedding of the reference dataset (miLISI = 1.94 when compared with simulated data) and Fig. 6B shows the density plots of the experimental and simulated data, revealing a high degree of resemblance. We focused on the changes in the iLISI values of experimentally profiled cells of a specific type (annotated in the original study), when jointly mapped to the same embedding space with simulated data, before and after knockout. For the changes in the iLISI values to be observable, it was necessary to focus on cell types that occupy relatively distinct regions of the embedding space: CD14+ monocytes, CD19+ B cells, Dendritic cells, CD56+ natural killer cells (Supplementary Fig. 22); otherwise, the presence of other cell types in that region would confound the results.

Figure 6C shows the iLISI distributions for each cell type (only for real cells), calculated along with unperturbed simulated cells (blue) and perturbed simulated cells (orange). Figure 6D, E show the iLISI values and the distribution of generated cells after the knockout experiment for CD19+ B cells as an example (see Supplementary Figs. 23–26 for other cell types). In all cases, the iLISI values significantly reduced after the knockout experiment (Fig. 6C, Supplementary Table 7), showcasing that after the knockout of top TFs associated with a cell type, GRouNdGAN generated much fewer cells resembling that cell type (even though it did not have the knowledge of the cell type annotations). Instead, the simulated cells were dispersed into other cell types, yet they retained meaningful positions within the overall dataset embedding space. Also, the

miLISI of the other cell types remained relatively unchanged (Supplementary Table 7).

Next, we repeated the analyses above using Tumor-All dataset that contained different cell types including malignant cells, but also other cell types present in the tumor microenvironment. We considered cell types that made up of at least 0.25% of the dataset (Supplementary Table 8). When cells of two or more types populated the same regions of the embedding space, we kept the cell type containing more cells. Malignant cells, T cells, B cells, and plasma cells satisfied the conditions above, which we included in our analyses. Similar to the previous analyses, we observed that the KO of top 3 TFs of each cell type results in a significant reduction in the iLISI values (Supplementary Fig. 27, Supplementary Table 8).

These analyses suggest that in silico TF perturbation experiments using GRouNdGAN produces results concordant with results directly obtained from real biological data, a direction that we will explore further in the future studies.

## Discussion

GRouNdGAN is a causal implicit generative model designed to simulate realistic scRNA-seq datasets based on a user-defined GRN and a reference dataset. By incorporating GRN connections in its architecture and including auxiliary tasks, it imposes causal TF-gene relationships in the gene expression profile of simulated cells. These causal relationships are verifiable by in silico knockout experiments and identifiable by GRN inference methods. (In addition to the experimental analyses performed in this study, an interested reader should refer to Kocaoglu et al.[31] for theoretical evidence of causality of data generated by this architecture.)

We demonstrated that GRouNdGAN achieves state-of-the-art performance on various tasks including realistic scRNA-seq data generation, GRN inference benchmarking, and in silico knockout experiments. Even when applied to heterogenous datasets, it generated realistic datasets without utilizing information regarding cell clusters or cell types, achieving best or second-best performance compared to models that used this information. We should note that it is trivially possible to use cell type/cluster information with GRouNdGAN: one can simply provide subsets of the experimental dataset corresponding to a cell type/cluster as a reference, one at a time, with a cell-type specific (or shared) GRN (similar to our analyses using PBMC-CTL and PBMC-NaiveT in Supplementary Data 2 – Sheet 2). However, the fact that this model does not require this information broadens its applicability. We showed that GRouNdGAN-generated data preserves complex cellular dynamics and patterns of the reference dataset such as lineage trajectories, pseudo-time orderings, and patterns of cell type markers' activation. One consideration in generating realistic and generalizable cells is the number of training examples. In the BoneMarrow dataset (which had only 2.7k cells), the performance of all simulators

dropped on the test set compared to the training set. When we generated cells using another similar hematopoiesis dataset (Dahlin), but with much larger number of cells ( 45k), the testing and training performance difference significantly reduced (Table 3). Similarly, the training and testing performance on all the other datasets that had more than ten thousand cells remained on par with each other, suggesting that a large number of cells improves the realism of the simulated data (which is expected). In spite of this, the cells generated by GRouNdGAN could capture trajectories and pseudo-time orderings not only in the larger Dahlin dataset, but also in the smaller BoneMarrow dataset.

GRouNdGAN imposes user-defined causal regulatory interactions, while excluding those present in the reference dataset, but not in the input GRN. This makes GRouNdGAN-simulated datasets ideal for benchmarking GRN inference algorithms with the input GRN as the ground truth. We benchmarked eight GRN inference algorithms on the simulated BoneMarrow dataset and showed that our results coincide with results reported by BEELINE[19] on curated and experimental benchmarks. This highlights GRouNdGAN's ability to bridge the gap between experimental and simulated datasets for GRN benchmarking without requiring excessive user manipulation. We assessed five of these methods (that did not require pseudo-time) on the PBMC-CTL dataset and observed that their ordering was consistent with the simulated BoneMarrow data. We then applied them to the PBMC-All dataset, containing multiple distinct cell types. While the general trend was consistent with the other benchmarks, the order of GENIE3 and GRNBoost2 switched. This could be related to the ability of these methods on handling distinct cell types. One should also note that the results of benchmarking on PBMC-All is not conclusive, and follow-up studies are required to fully characterize the effect of distinct cell types on the performance of GRN inference methods. In particular, GRNs are cell type- and context-specific, and such a study should impose different GRNs on different subpopulations of the dataset. Due to the flexibility provided by GRouNdGAN, one can evaluate a variety of GRN inference methods including supervised methods[49,50] or cell-specific GRN inference methods[48,51]. We included CeSpGRN[48] in our benchmarks, but a comprehensive study that evaluates this and other methods of this class from different perspectives is of great interest.

Since GRouNdGAN generates data based on a structural causal model, it can be used to sample synthetic cells from both observational and interventional distributions. Using this property, we conducted in silico perturbation experiments by knocking out key regulating TFs in various cell types and observed that the targeted cell-states were suppressed in the generated dataset. These results, while preliminary, show potential for developing a method based on GRouNdGAN to predict the results of TF knockout experiments. However, such method would inevitably need access to a high quality GRN underlying the real data, requiring additional data modalities such as (sc)ATAC-seq or TF ChIP-seq.

GRouNdGAN maintains key advantages over existing simulators that incorporate GRNs such

as SERGIO and BoolODE. BoolODE, unlike GRouNdGAN, is a reference-free (de novo) and rule-based simulator that relies on user-defined parameters instead of a reference dataset to artificially introduce properties of single-cell data (e.g., dropout, batch effect, existence of distinct populations, and variation among experimental samples and conditions). As a rule-based simulator, it allows for flexible selection of simulation parameters such as choice of noise strength, dropout rate, or kinetic parameters (for mRNA transcription, protein translation and mRNA and protein degradation rates). However, extensive user knowledge is required to leverage this flexibility to avoid generating unrealistic datasets and recent studies have argued that one should use discretion when interpreting results obtained from such de novo simulations[52]. This may explain why some of GRN inference methods that performed best on BoolODE's synthetic benchmarks, were among the worst on the experimental benchmarks of BEELINE. Moreover, as a reference-free simulator, BoolODE is not designed to match a specific reference dataset. As a result, it does not lend itself to benchmarking GRN inference methods in distinct contexts characterized by experimental reference datasets.

On the other hand, SERGIO is a hybrid method that contains elements of reference-free simulators, but also enables matching the simulated cells to a specific reference dataset by adding technical noise. SERGIO simulates data by first generating a "clean" dataset based on SDEs. Then, the user would iteratively fine-tune three noise parameters until the simulated and reference datasets are matched. However, this data-matching procedure puts extra burden on the user and might lead the user to overly rely on fine-tuning technical noise parameters, instead of SDE parameters. Our experiments with SERGIO (described in Supplementary Notes) showed that in spite of our best efforts for distribution matching, there were clear differences between the SERGIO-simulated data and the reference dataset. Another consideration is that over relying on added technical noise to achieve data-matching can potentially corrupt the data, rendering the underlying regulatory patterns unobservable in the final simulated dataset. This might explain the near-random GRN inference performance of various methods reported by SERGIO on their noisy datasets[4].

In contrast to BoolODE and SERIGO, during training, GRouNdGAN's parameters are optimized to minimize any mismatch between real and simulated data distributions automatically. This enables it to replicate scRNA-seq data characteristics (e.g., technical and biological noise) without any user intervention or post-processing. Moreover, GRouNdGAN is trained such that each of its generated genes or TFs mimics the expression patterns of a corresponding gene or TF in the reference dataset, preserving gene/TF identities. In contrast, BoolODE and SERGIO simulate pseudo-genes whose identities cannot be readily mapped to their counterparts in the reference dataset. Gene identity preservation can be attained in these simulators by fine-tuning the SDE parameters until the expression patterns of a given generated gene aligns with a corresponding gene in the reference dataset. This, however, places further burden on the user. Finally,

GRouNdGAN learns complex non-linear co-regulatory patterns from the data, instead of relying on simplifying assumptions or user input. In contrast, SERGIO assumes that the combined effect of multiple regulatory TFs is simply the sum of their individual effects. Similarly, BoolODE requires the user to provide a truth table specifying combinatory TF co-regulation rules for each target gene. In contrast, GRouNdGAN's neural network-based generator enables it to implicitly learn complex non-linear co-regulatory patterns that may represent co-expression, co-inhibition, or co-activation of individual genes. This is achieved during the training of GRouNdGAN without user intervention.

Several key points need to be considered when using GRouNdGAN. First, our analyses showed that the architectural and hyperparameter choices of the presented model result in good performance across different datasets. We performed hyperparameter tuning on one dataset and used the same set of values for all datasets and for different choices of GRNs, obtaining state-of-the-art performance. However, when trained on a completely new dataset, or datasets containing significantly different number of genes and TFs, it is advisable to try different choices of the hyperparameters using a validation set. Second, the architecture of the model includes LSN layers and, as such, the model generates expression values library-size normalized across the considered genes and TFs. Our ablation study (Supplementary Notes) and our experiments showed that the LSN layer improves the GRN imposition, the realism of the generated samples, and stabilizes the training (an observation also made by the scGAN study). In the future iterations of the model, we are planning to remove this restriction to enable generating unnormalized count data. Third, an important consideration for any causal inference problem is that given an observational dataset, its underlying causal graph is not necessarily unique. When benchmarking GRN inference methods, this problem exists regardless of how the ground truth GRN is obtained (even using knockout/knockdown experiments), and a GRN inference algorithm may find a plausible causal GRN, yet not the one that generated the data. Despite this, our analysis using the positive/negative control GRNs showed that GRN inference methods could detect the imposed edges while ignoring others that were unimposed. Based on this observation, we posit that the imposed GRN by GRouNdGAN, while not the only causal GRN capable of generating the simulated dataset, is the most probable one. However, verifying this requires theoretical analyses that are well beyond the scope of this study. Fourth, GRouNdGAN is a generative model and is not a predictive model. As such, its goal is to simulate cells similar to a reference training set (while imposing a user-defined GRN), and not to predict the scRNA-seq profile of cells in a new context or an independent dataset. Given a new dataset, the model needs to be retrained to generate realistic samples.

Finally, although GRouNdGAN can generate data with any GRN, the resemblance of the data to the reference dataset deteriorates if the imposed TF-gene relationships are significantly different from those consistent with the patterns in the reference dataset. The reason is that in such a

scenario, generating simulated datapoints resembling the reference training dataset and imposing the GRN act as contradictory requirements. This is a fundamental problem that is not unique to GRouNdGAN and is applicable to any simulator with these two requirements. However, simulators that do not try to impose a causal graph (e.g., scGAN) can focus on generating realistic scRNA-seq samples, even at the expense of disrupting some of the interactions present in the reference dataset (as we observed in Fig. 3 with scGAN). As a result, for GRN inference method benchmarking, we recommend first finding a set of regulatory edges from the reference dataset (e.g., using GRNBoost2 or even other data modalities) and imposing a selected subset of those in the simulated data. This ensures that the imposed edges are encoded in the simulated data, the unimposed (or undetected) edges are disrupted (as shown in our analyses), and the simulated dataset resembles the reference dataset. It is important to note that the regulatory edges inferred from the reference dataset do not need to be causal (and most likely, some are not); some of them could represent spurious correlations that are observable in the data. In fact, most GRN inference methods do not identify causal edges, yet can be used to form the input GRN. When a subset of the (potentially) non-causal edges of the reference dataset are imposed by GRouNdGAN, they are imposed in a *causal* manner, representing the causal graph underlying the *simulated* data (and not the reference data).

The current version of GRouNdGAN only allows imposing a bipartite GRN and does not support a multi-layer GRN, an assumption that we plan to relax with the future versions of the model. Moreover, we intend to augment the model's architecture in the future to enable imposing a causal graph with three layers of nodes capturing TF-gene-phenotype relationships. This is particularly useful for development and evaluation of phenotype-relevant GRNs, a concept that we have introduced in the past[11]. Additionally, a conditional version of GRouNdGAN can be developed, allowing the user to generate cells of specific cell type or on a specific stage along a lineage trajectory. Finally, by incorporating DAG structural learning[53], we intend to enable GRouNdGAN to infer a GRN while simultaneously learning to mimic the reference dataset.

# Methods

## Datasets and preprocessing

We downloaded the human peripheral blood mononuclear cell (PBMC Donor A) dataset containing the single-cell gene expression profiles of 68579 PBMCs represented by UMI counts from the 10x Genomics (`https://support.10xgenomics.com/single-cell-gene-expression/datasets/1.1.0/fresh_68k_pbmc_donor_a`). This dataset contains a large number of well-annotated cells and has been used by other models to generate synthetic scRNA-seq

samples[24]. We also downloaded the scRNA-seq profile of 2730 cells corresponding to differentiation of hematopoietic stem cells to different lineages from mouse bone marrow[34] ("BoneMarrow" dataset) from Gene Expression Omnibus (GEO) (accession number: GSE72857). We also obtained another haematopoietic dataset corresponding to the scRNA-seq (10x Genomics) profiles of 44,802 mouse bone marrow hematopoietic stem and progenitor cells (HSPCs) differentiating towards different lineages from GEO (accession number: GSE107727)[46] (called Dahlin dataset here). Finally, we obtained the batch-corrected scRNA-seq (10x Genomics) profile of 136,147 cells corresponding to malignant cells as well as cells in the tumor microenvironment (called Tumor-All dataset here) from 20 fresh core needle biopsies of follicular lymphoma patients from (https://cellxgene.cziscience.com/collections/968834a0-1895-40df-8720-666029b3bbac)[47].

We followed pre-processing steps similar to those of scGAN and cscGAN[24] using scanpy version 1.8.2[45]. In each dataset, cells with nonzero counts in less than ten genes were removed. Similarly, genes that only had nonzero counts in less than three cells were discarded. Top 1000 highly variable genes were selected using the dispersion-based method described by Satija et al.[54]. Finally, library-size normalization was performed on the counts per cell with a library size equal to 20,000 in order to be consistent with previous studies[24]. See Supplementary Data 1 - Sheet 3 for the number of cells and highly variable genes and TFs present in each final dataset.

## GRouNdGAN's model architecture

GRouNdGAN's architecture consists of 5 components, each implemented using separately parameterized neural networks: a causal controller, target generators, a critic, a labeler, and an anti-labeler (Fig. 1C).

### Causal controller

The role of causal controller is to generate the expression of TFs that causally control the expression of their target genes based on a user-defined gene regulatory network (Fig. 1C). To achieve this, it is first pre-trained (Fig. 1B) as the generator of a Wasserstein GAN with gradient penalty (WGAN-GP) (see Supplementary Notes for the formulation of the Wasserstein distance). GANs are a class of deep learning models that can learn to simulate non-parametric distributions[55]. They typically involve simultaneously training a generative model (called the "generator") that produces new samples from noise, and its adversary, a discriminative model that tries to distinguish between real and generated samples (called the "discriminator"). The generator's goal is to generate samples so realistic that the discriminator cannot determine whether it is real or simulated (an accuracy value close to 0.5). Through adversarial training, the generator and

discriminator receive feedback, allowing them to co-evolve in a symbiotic manner.

The main difference between a WGAN and a traditional GAN is that in the former, a Wasserstein distance is used to quantify the similarity between the probability distribution of the real data and the generator's produced data (instead of Kullback–Leibler or Jensen–Shannon divergences). Wasserstein distance has been shown to stabilize the training of WGAN without causing mode collapse[32]. The detailed formulation of the Wasserstein distance used as the loss function in this study is provided in Supplementary Notes. In addition, instead of a discriminator, WGAN uses a "critic" that estimates the Wasserstein distance between real and generated data distributions. In our model, we added a gradient penalty term for the critic (proposed by Gulrajani et al.[56] as an alternative to weight clipping used in the original WGAN) in order to overcome vanishing/exploding gradients and capacity underuse issues.

In the pretraining step, we trained a WGAN-GP with a generator (containing an input layer, three hidden layers, and an output layer), a library-size normalization (LSN) layer[24], and a critic (containing an input layer, three hidden layers, and an output node). A noise vector of length 128, with independent and identically distributed elements following a standard Gaussian distribution, was used as the input to the generator. The output of the generator was then fed into the LSN layer to generate the gene and TF expression values. The details of hyperparameters and architectural choices of this WGAN-GP are provided in Supplementary Table 1. Although we were only interested in generating expression of TFs using the generator of this WGAN-GP (in the second step of pipeline), the model was trained using all genes and TFs to properly enforce the library-size normalization. Once trained, we discarded the critic and the LSN layer, froze the weights of the generator and used it as the "causal controller"[31] to generate expression of TFs (Fig. 1C).

### Target generators

The role of target generators is to generate the expression of genes causally regulated by TFs based on the topology of a GRN. Consider a target gene $G_j$ regulated by a set of $TFs = \{TF_1, TF_2, ..., TF_n\}$. Under the causal sufficiency assumption and as a result of the manner by which TFs' expressions are generated from independent noise variables, we can write $E_{G_j} = f_{G_j}(E_{TF_1}, E_{TF_2}, ..., E_{TF_n}, N_{G_j})$ and $E_{TF_i} = f_{TF_i}(N_{TF_i})$ for $i = 1, 2, ..., n$, where $E$ represents expression, $N$ represents a noise variable, and $f$ represents a function (to be approximated using neural networks). All noise variables are jointly independent. Following the theoretical and empirical results of CausalGAN[31], we can use feedforward neural networks to represent functions $f$ by making the generator inherit its neural network connections from the causal GRN. To achieve this, we generate each gene in the GRN by a separate generator such that target gene generators do not share any neural connections (Fig. 1C).

As input, the generator of each gene accepts a vector formed by concatenating a noise variable and a vector of non-library size normalized TF expressions from the causal controller, corresponding to the TFs that regulate the gene in the imposed GRN (i.e., its parents in the graph). The expression values of TFs and the generated values of target genes are arranged into a vector, which is passed to an LSN layer for normalization. We used target generators with three hidden layers of equal width. The width of the hidden layers of a generator is dynamically set as twice the number of its regulators (the noise variable and the set of regulating TFs). If the imposed GRN is relatively dense and contains more than 5000 edges, we set the depth to 2 and the width multiplier to 1 to be able to train on a single GPU. The details of hyperparameters and architectural choices are provided in Supplementary Table 2.

In practice, generating each target gene's expression using separate neural networks introduces excess overhead. This is because in every forward pass, all target genes' expressions must be first generated before collectively being sent into the LSN layer. As a result, instead of parallelizing the generation of each target gene's expression (which due to the bottleneck above does not provide a significant computational benefit), we implemented target generators using a single large sparse network. This allows us to reduce the overhead and the training time to train the model on a single GPU, and to benefit from GPU's large matrix multiplication. We mask weights and gradients to follow the causal graph, while keeping the generation of genes independent from each other. From a logical standpoint, our implementation has the same architecture described earlier, but is significantly more computationally efficient. See Supplementary Notes for details.

#### Critic

Similar to a traditional WGAN, the objective of GRouNdGAN's critic (Fig. 1C) is to estimate the Wasserstein distance between real and generated data distributions. We used the same critic architecture as the WGAN-GP trained in the first stage.

#### Labeler and anti-labeler

Although the main role of the target generators is to produce realistic cells to confuse the critic, it is crucial that they rely on the TFs' expression (in addition to noise) in doing so. One potential risk is that the target generators disregard the expression of TFs and solely rely on the noise variables. This is particularly probable when the imposed GRN does not conform to the underlying gene expression programs of real cells in the reference dataset; in such a scenario, and to make realistically looking simulated cells, it is more convenient for a WGAN to simply ignore the strict constraints of the GRN and solely rely on noise.

To overcome this issue, we used auxiliary tasks and neural networks known as "labeler" and "anti-

labeler"[31]. The task of these two networks is to estimate the causal controller's TF expressions (here called labels) from the target genes' expressions alone, by minimizing the squared L2 norm between each element's TF estimates and their true value. More specifically, the corresponding loss for a batch of size $N_{batch}$ is of the form $\frac{1}{N_{batch}} \sum_{i=1}^{N_{batch}} ||\hat{y}_i - y_i||_2^2$, where $\hat{y}_i$ is the estimated vector of TF expression values generated by the labeler or anti-labeler. For anti-labeler, $y_i$ corresponds to the TF expression values outputted by the causal controller; for labeler, this vector can also correspond to TF expression values from the real training data. This resembles the idea behind an autoencoder and ensures that the model will not disregard the expression of TFs in generating the expression of their target genes. The anti-labeler is trained solely based on the outputs of the target generators, while the labeler utilizes both the outputs of target generators and the expression of real cells (Fig. 1C). They are both implemented as fully connected networks with a width of 2000 and a depth of 3 and optimized using the AMSGrad[57] algorithm. Each layer, except the last one, utilizes a ReLU activation function and batch normalization. In addition to WGAN-GP losses, we add labeler and anti-labeler losses to the generator to minimize both. This is different from the approach used in CausalGAN, where the anti-labeler's loss is maximized in the early training stages to ensure that the generator doesn't fall into label-conditioned mode collapse. In GRouNdGAN, the causal controller is pretrained and generates continuous labels (TFs expression) and does not face a similar issue. As a result, we instead minimized the loss of the anti-labeler from the beginning and as such the labeler and anti-labeler both act as auxiliary tasks to ensure the generated gene expression values take advantage of TFs expression.

## Training procedure and hyperparameter tuning

We follow a two-step training procedure comprising of training two separate WGAN-GPs (Fig. 1) to train GRouNdGAN. The generators and critics in both GANs are implemented as fully connected neural networks with rectified linear unit (ReLU) activation functions in each layer, except for the last layer of the critic. The weights were initialized using He initialization[58] for layers containing ReLU activation and Xavier initialization[59] for other layers (containing linear activations). We used Batch Normalization[60] to normalize layer inputs for each training minibatch, except for the critic. This is since using it in the critic invalidates the gradient penalty's objective, as it penalizes the norm of the critic's gradient with respect to the entire batch rather than to inputs independently. An LSN layer[24] was used in both WGAN-GPs (Fig. 1B, C) to scale counts in each simulated cell to make it consistent with the library size of input reference dataset. This normalization results in a dramatic decrease in convergence time and smooths training by mitigating the inherent heterogeneity of scRNA-seq data.

We would like to point out that both steps of training (i.e., the pretraining of Fig. 1B and training of Fig. 1C) are performed on the exact same training set, and when data corresponding to a new

dataset is to be generated, these steps need to be repeated. The goal of the first step is to train the causal controller to learn the distribution of training set and generate realistic TF expression values (without imposing TF-gene relationships). The goal of the second step is to use the TF expression values generated by the trained causal controller to generate expression of target genes while imposing the TF-gene causal relationships.

GRouNdGAN solves a min-max game between the generator ($f_g$) and the critic ($f_c$), with the following objective function:

$$\min_{f_g} \max_{\|f_c\|_L \leq 1} \mathbb{E}_{x \sim \mathbb{P}_r} f_c(x) - \mathbb{E}_{x \sim f_g(\mathbb{P}_{\text{noise}})} f_c(x) \tag{1}$$

The training objective of the critic involves maximizing the difference between the average score assigned to real ($\mathbb{E}_{x \sim \mathbb{P}_r} f_c(x)$) and generated samples ($\mathbb{E}_{x \sim f_g(\mathbb{P}_{noise})} f_c(x)$) with respect to its parameters following Eq. (1). On the contrary, the generator attempts to minimize the average score that the critic assigns to real and generated samples. Through this adversarial game, both the critic and generator co-evolve and the generator learns a mapping $f_g$ from a simple standard Gaussian noise distribution ($\mathbb{P}_{noise}$) to a distribution $f_g\left(\mathbb{P}_{noise}\right)$ that approximates the real data distribution $\mathbb{P}_r$. An important point to consider is that the critic does not directly compute the Wasserstein distance (mathematically defined in Supplementary Notes - Details regarding the Wasserstein distance). Instead, it is encouraged to provide a meaningful estimate of the Wasserstein distance between the distribution of real data ($\mathbb{P}_r$) and the distribution of generated data ($f_g\left(\mathbb{P}_{noise}\right)$) through its training process.

We alternated between minimizing the generator loss for one iteration and maximizing the critic loss for five iterations. We employed the AMSGrad[57] optimizer with the weight decay parameters $\beta_1 = 0.5, \beta_2 = 0.9$ and employed an exponentially decaying learning rate for the optimizer of both the critic and generator.

The hyperparameters were tuned using a validation set consisting of 1000 cells from the PBMC-CTL dataset (Supplementary Tables 1 and 2) based on the Euclidean distance and the RF AUROC score, which were consistently in accord. The same hyperparameters were used for all other analyses and datasets.

## Causal GRN preparation

This section describes the creation of the causal graph inputted to GRouNdGAN used to impose a causal structure on the model. GRouNdGAN accepts a GRN in the form of a bipartite directed acyclic graph (DAG) as input, representing the relationship between TFs and their target genes. In this study, we created the causal graph using the 1000 most highly variable genes

and TFs identified in the preprocessing step (Supplementary Data 1 - Sheet3). First, the set of TFs among the highly variable genes were identified based on the AnimalTFDB3.0 database[61] and a GRN was inferred using GRNBoost2[8] (with the list of TFs provided) from the training reference dataset. It is important to note that the regulatory edges identified from the reference dataset using GRNBoost2 are not necessarily causal edges (and they do not need to be for the purpose of forming the input GRN), but they are consistent with the patterns of the data. However, when this (potentially non-causal) GRN is imposed by GRouNdGAN, it is imposed in a causal manner and represents the causal data generating graph of the simulated data (and not the reference data).

## Evaluation of the resemblance of real and simulated cells

We evaluated all models using held-out test sets containing randomly selected cells from each reference dataset (500 cells from BoneMarrow and 1000 cells for all other datasets) (see Supplementary Data 1 - Sheet 3 for other statistics about the datasets). To quantify the similarity between real and generated cells, we employed various metrics. For each cell represented as a datapoint in a low dimensional embedding (e.g., t-SNE or UMAP), the integration local inverse Simpson's Index (iLISI)[36] captures the effective number of datatypes (real or simulated) to which datapoints of its local neighborhood belong based on weights from a Gaussian kernel-based distributions of neighborhoods. The miLISI is the mean of all these scores and in our study ranges between 1 (poor mixing of real and simulated cells) and 2 (perfect mixing of real and simulated cells). Additionally, we calculated the cosine and Euclidean distances of the centroids of real cells and simulated cells, where the centroid was obtained by calculating the mean along the gene axis (across all simulated or real cells).

To estimate the proximity of high-dimensional distributions of real and simulated cells without creating centroids, we used the maximum mean discrepancy (MMD)[35]. Given two probability distributions $p$ and $q$ and a set of independently and identically distributed (i.i.d.) samples from them, denoted by $X$ and $Y$, $MMD$ with respect to a function class $\mathcal{F}$ is defined as:

$$\mathrm{MMD}\left[\mathcal{F}, X, Y\right] := \sup_{f \in \mathcal{F}} \left( \mathbb{E}_x \left[ f(x) \right] - \mathbb{E}_y \left[ f(y) \right] \right), \tag{2}$$

where *sup* refers to supremum and $\mathbb{E}$ denotes expectation. When the MMD function class $\mathcal{F}$ is a unit ball in a reproducing kernel Hilbert space (RKHS) $\mathcal{H}$ with kernel $k$, the population MMD takes a zero value if and only if $p = q$ and a positive unique value if $p \neq q$. The squared MMD can be written as the distance of mean embeddings $\mu_p, \mu_q$ of distributions $p$ and $q$, which

can be expressed in terms of kernel functions:

$$\begin{aligned} \mathrm{MMD}^2[F, p, q] &= \|\mu_p - \mu_q\|_H^2 \\ &= \mathbb{E}_{x,x'}\big[k(x, x')\big] - 2\mathbb{E}_{x,y}\big[k(x, y)\big] + \mathbb{E}_{y,y'}\big[k(y, y')\big] \end{aligned} \tag{3}$$

Following existing implementations of MMD in the single-cell domain[24,62], we chose a kernel that is the sum of three Gaussian kernels to increase sensitivity of the kernel to a wider range:

$$k\left(x, y\right) = \sum\nolimits_i \left(\frac{||x - y||^2}{\sigma_i^2}\right),\ i = 1, 2, 3 \tag{4}$$

where $_i$ denote standard deviations and were chosen to be the median of the average distance between a point to its 25 nearest neighbors divided by factors of 0.5, 1, and 2 in the three kernels, respectively.

We also used a random forests (RF) classifier and used its area under the receiver operating characteristic (AUROC) curve to determine whether the real and simulated cells can be distinguished from each other. Consistent with previous studies[24,63], we first performed a dimensionality reduction using principal component analysis (PCA) and used the top 50 PCs of each cell as the input features to the RF model, which improves the computational efficiency of this analysis. The RF model was composed of 1000 trees and the Gini impurity was used to measure the quality of a split.

## Baseline simulator models

We compared the performance of GRouNdGAN to scDESIGN2[25], SPARsim[26], and three GAN-based methods: scGAN[24], cscGAN with projection-based conditioning[24], and a conditional WGAN (cWGAN). The cWGAN method conditions by concatenation following the cGAN framework[64]. More specifically, it concatenates a one-hot encoded vector (representing the cluster number or cell type) to the noise vector input to the generator and cells forwarded to the discriminator. We did not train the cWGAN or cscGAN on the PBMC-CTL dataset, since it contains only one cell type. For the PBMC-All and the BoneMarrow dataset, we trained all models above. Additionally, we simulated data using scDESIGN2 and SPARsim with and without cell cluster information, as they allow providing such side information in their training.

To train models that utilized cell cluster information, we performed Louvain clustering and provided the cluster information and ratio of cells per cluster during training. Clustering was done by following the cell ranger pipeline[33], based on the raw unprocessed dataset (and independent of the pre-processing steps described earlier for training simulators). First, genes with no UMI count in any of the cells were removed. Then the gene expression profile of each

cell was normalized by the total UMI of all (remaining) genes, and highly variable genes were identified. The gene expression profile of each cell was then re-normalized by the total UMI of retained highly variable genes, and each gene vector (representing its expression across different cells) was z-score normalized. Given the normalized gene expression matrix above, we found top 50 principal components (PCs) using PCA analysis. These PCs were then used to compute a neighborhood graph with a local neighborhood size of 15, which was used in Louvain clustering. We ran the Louvain algorithm with a resolution of 0.15.

For SPARSim, we set all sample library sizes to 20000 and estimated gene expression level intensities and simulation parameters by providing it with both raw and normalized count matrices. When cell cluster information was provided, distinct SPARSim simulation parameters were estimated per cell for each cluster. scDESIGN2 accepts input matrices where entries are integer count values; we thus performed rounding on the expression matrix before fitting scDESIGN2. With cluster information provided, a scDESIGN2 model was fit separately for cells of each cluster, and similar to conditional GANs, the ratio of cells per cluster was provided to the method.

### In silico perturbation experiments using GRoundGAN

To perform perturbation experiments using GRoundGAN, we put the trained model in a deterministic mode of operation. This is necessary to ensure that the perturbation experiments are performed on the same batch (i.e., replicate) of generated cells to form matched case/control experiments. To do this, we performed a forward pass through the generator and then saved the input noise to the causal controller, the input noise to the target generators, the TF expression values generated by the causal controller, and the per-cell scaling factor of the LSN layer. Subsequent passes through the generators used the saved parameters so that ensuing runs always output the same batch of cells (instead of generating new unmatched cells).

### Trajectory inference and pseudo-time analysis

Following official PAGA tutorial for the BoneMarrow dataset (`https://github.com/scverse/scanpy-tutorials/blob/master/paga-paul15.ipynb`), we used (Partition-based graph abstraction) PAGA[38] for trajectory inference and analysis. We built force-directed graphs[65] (with ForceAtlas2[66]) using the top 20 principal components of the data (using principal component analysis or PCA) and a neighborhood graph of observations computed using UMAP (to estimate connectivities). We next denoised the graph by representing it in the diffusion map space and computed distances and neighbors as before using this new representation. After denoising, we then ran the Louvain clustering algorithm with a resolution of 0.6. Finally, we

ran the PAGA algorithm on the identified clusters and used the obtained graph to initialize and rebuild the force-directed graph.

### GRN inference methods

In our GRN benchmarking analysis, we focused on eight GRN inference algorithms: GENIE3[9], GRNBoost2[8], PPCOR[37], PIDC[10], LEAP[12], SCODE[13] and SINCERITIES[14], which were used in the BEELINE study[19], as well as CeSpGRN[48]. Of these methods, LEAP requires pseudo-time ordering of cells, while SCODE and SINCERITIES require both pseudo-time ordering and pseudo-time values. Since not all algorithms inferred the edge directionality or its sign (activatory or inhibitory nature), we did not consider these factors in our analysis to be consistent among different models.

For the methods available in the BEELINE study, we ran them as docker containers using the docker images provided by BEELINE's GitHub (`https://github.com/Murali-group/Beeline`)[19] with the default parameters used in BEELINE. These methods were applied to nine datasets simulated by GRouNdGAN and scGAN, and the original training real dataset corresponding to PBMC-CTL, PBMC-All, and BoneMarrow. To ensure consistency, the same number of cells as the real training set were simulated using GRouNdGAN and scGAN for each dataset: $n = 19773$ for PBMC-CTL, $n = 67579$ for PBMC-All, and $n = 2230$ for BoneMarrow. Number of genes and TFs present in the GRN for each dataset is provided in Supplementary Data 1 - Sheet 3. To benchmark algorithms requiring pseudo-time ordering of cells, we computed the pseudo-times of GRouNdGAN-simulated data (based on the BoneMarrow dataset) using PAGA[38] and diffusion pseudotime[39], following the methodology described earlier. In the GRN inference benchmark analysis, we did not provide the list of TFs to GRNBoost2 to make it consistent with other GRN inference methods.

We also included CeSpGRN, which is a cell-specific GRN inference method. Since this method first generates one GRN for each cell, it requires a high amount of memory to run. As a result, we were only able to benchmark it using a subset of data consisting of only $n = 1000$ cells (for any of the three datasets) and 100 genes (we did not change the number of TFs, or the GRN edges connecting them to the considered genes). Following the method described in the original study, we then averaged the total absolute edge weights across all cells to form a consensus GRN using CeSpGRN.

### Statistics and reproducibility

The sample sizes were selected by original studies producing datasets used for training GRouNdGAN. Preprocessing steps are described in the datasets and preprocessing section. Other than standard cell-level and gene-level filtering, there were no data exclusions. Statistical tests used for each analysis are described in the corresponding sections and include Wilcoxon signed rank and Mann Whitney U tests.

### Reporting summary

Further information on research design is available in the Nature Portfolio Reporting Summary linked to this article.

## Data availability

The PMBC reference dataset is available from the 10x Genomics repository (corresponding to healthy donor A) from the following link: https://support.10xgenomics.com/single-cell-gene-expression/datasets/1.1.0/fresh_68k_pbmc_donor_a? (related to PBMC-All, PBMC-CTL, and PBMC-NaiveT). The BoneMarrow reference dataset is available in the Gene Expression Omnibus (GEO) repository under accession number GSE72857. The Dahlin reference dataset is available in GEO (accession number: GSE107727). The Tumor-All (and Tumor-malignant) dataset is available from cellxgene (https://cellxgene.cziscience.com/collections/968834a0-1895-40df-8720-666029b3bbac). Simulated data corresponding to PBMC-All, PBMC-CTL, BoneMarrow, Dahlin, Tumor-All, and Tumor-malignant and their corresponding imposed GRNs are provided on GRouNdGAN's website (https://emad-combine-lab.github.io/GRouNdGAN/benchmarking) and can be used for benchmarking different GRN Inference methods. Source data are provided with this paper.

## Code availability

Our implementation and evaluation of GRouNdGAN in Python 3.9.6 using the PyTorch framework[67] along with a tutorial is freely available under the GNU Affero General Public License v3.0 on GitHub (https://github.com/Emad-COMBINE-lab/GRouNdGAN) which is archived in Zenodo under record number 11068246[68] (https://doi.org/10.5281/zenodo.11068246). The repository also contains a Docker image for reproducibility and our PyTorch implementation of scGAN, projection-based conditioning cscGAN[24] and a variation

of cscGAN that uses conditioning by concatenation (cWGAN). More information about the installation, a detailed tutorial, and simulated datasets that can be readily used are provided in (`https://emad-combine-lab.github.io/GRouNdGAN`).

## References


1. Lee, T. I. et al. Transcriptional regulatory networks in Saccharomyces cerevisiae. *Science* **298**, 799–804 (2002).

2. Che, D. et al. Dynamic and modular gene regulatory networks drive the development of gametogenesis. *Brief. Bioinform.* **18**, 712–721 (2017).

3. Olson, E. N. Gene regulatory networks in the evolution and development of the heart. *Science* **313**, 1922–1927 (2006).

4. Dibaeinia, P. & Sinha, S. SERGIO: a single-cell expression simulator guided by gene regulatory networks. *Cell Syst.* **11**, 252–271. e211 (2020).

5. Yang, Y. et al. Gene knockout inference with variational graph autoencoder learning single-cell gene regulatory networks. *Nucleic Acids Res.* **51**, 6578–6592 (2023).

6. Madhamshettiwar, P. B., Maetschke, S. R., Davis, M. J., Reverter, A. & Ragan, M. A. Gene regulatory network inference: evaluation and application to ovarian cancer allows the prioritization of drug targets. *Genome Med.* **4**, 1–16 (2012).

7. Manuel, A. M., Dai, Y., Jia, P., Freeman, L. A. & Zhao, Z. A gene regulatory network approach harmonizes genetic and epigenetic signals and reveals repurposable drug candidates for multiple sclerosis. *Hum. Mol. Genet.* **32**, 998–1009 (2023).

8. Moerman, T. et al. GRNBoost2 and Arboreto: efficient and scalable inference of gene regulatory networks. *Bioinformatics* **35**, 2159–2161 (2019).

9. Huynh-Thu, V. A., Irrthum, A., Wehenkel, L. & Geurts, P. Inferring regulatory networks from expression data using tree-based methods. *PloS one* **5**, e12776 (2010).

10. Chan, T. E., Stumpf, M. P. & Babtie, A. C. Gene regulatory network inference from single-cell data using multivariate information measures. *Cell Syst.* **5**, 251–267. e253 (2017).

11. Emad, A. & Sinha, S. Inference of phenotype-relevant transcriptional regulatory networks elucidates cancer type-specific regulatory mechanisms in a pan-cancer study. *NPJ Syst. Biol. Appl.* **7**, 9 (2021).

12. Specht, A. T. & Li, J. LEAP: constructing gene co-expression networks for single-cell RNA-sequencing data using pseudotime ordering. *Bioinformatics* **33**, 764–766 (2017).

13. Matsumoto, H. et al. SCODE: an efficient regulatory network inference algorithm from single-cell RNA-Seq during differentiation. *Bioinformatics* **33**, 2314–2321 (2017).

14. Papili Gao, N., Ud-Dean, S. M., Gandrillon, O. & Gunawan, R. SINCERITIES: inferring gene regulatory networks from time-stamped single cell transcriptional expression profiles. *Bioinformatics* **34**, 258–266 (2018).

15. Aibar, S. et al. SCENIC: single-cell regulatory network inference and clustering. *Nat. Methods* **14**, 1083–1086 (2017).

16. Bravo González-Blas, C. et al. SCENIC+: single-cell multiomic inference of enhancers and gene regulatory networks. *Nat. Methods* **20**, 1355–1367 (2023).

17. Kartha, V. K. et al. Functional inference of gene regulation using single-cell multi-omics. *Cell Genomics* **2**, 100166 (2022).

18. Badia-i-Mompel, P. et al. Gene regulatory network inference in the era of single-cell multi-omics. *Nat. Rev. Genet.* **24**, 739–754 (2023).

19. Pratapa, A., Jalihal, A. P., Law, J. N., Bharadwaj, A. & Murali, T. Benchmarking algorithms for gene regulatory network inference from single-cell transcriptomic data. *Nat. Methods* **17**, 147–154 (2020).

20. Emmert-Streib, F., Dehmer, M. & Haibe-Kains, B. Gene regulatory networks and their applications: understanding biological and medical problems in terms of networks. *Front. Cell Dev. Biol.* **2**, 38 (2014).

21. Kolmykov, S. et al. GTRD: an integrated view of transcription regulation. *Nucleic Acids Res.* **49**, D104–D111 (2021).

22. Xu, H. et al. ESCAPE: database for integrating high-content published data collected from human and mouse embryonic stem cells. *Database* **2013**, bat045 (2013).

23. Kanehisa, M. & Goto, S. KEGG: kyoto encyclopedia of genes and genomes. *Nucleic Acids Res.* **28**, 27–30 (2000).

24. Marouf, M. et al. Realistic in silico generation and augmentation of single-cell RNA-seq data using generative adversarial networks. *Nat. Commun.* **11**, 1–12 (2020).

25. Sun, T., Song, D., Li, W. V. & Li, J. J. scDesign2: a transparent simulator that generates high-fidelity single-cell gene expression count data with gene correlations captured. *Genome Biol.* **22**, 1–37 (2021).

26. Baruzzo, G., Patuzzi, I. & Di Camillo, B. SPARSim single cell: a count data simulator for scRNA-seq data. *Bioinformatics* **36**, 1468–1475 (2020).

27. Schaffter, T., Marbach, D. & Floreano, D. GeneNetWeaver: in silico benchmark generation and performance profiling of network inference methods. *Bioinformatics* **27**, 2263–2270 (2011).

28. Marbach, D. et al. Revealing strengths and weaknesses of methods for gene network inference. *Proc. Natl Acad. Sci.* **107**, 6286–6291 (2010).

29. Marbach, D. et al. Wisdom of crowds for robust gene network inference. *Nat. Methods* **9**, 796–804 (2012).

30. Chen, S. & Mar, J. C. Evaluating methods of inferring gene regulatory networks highlights their lack of performance for single cell gene expression data. *BMC Bioinforma.* **19**, 1–21 (2018).

31. Kocaoglu, M., Snyder, C., Dimakis, A. G. & Vishwanath, S. CausalGAN: Learning causal implicit generative models with adversarial training. Preprint at https://arxiv.org/abs/1709.02023 (2017).

32. Arjovsky, M., Chintala, S. & Bottou, L. Wasserstein generative adversarial networks. In *International conference on machine learning*. 214–223 (PMLR, 2017).

33. Zheng, G. X. et al. Massively parallel digital transcriptional profiling of single cells. *Nat. Commun.* **8**, 1–12 (2017).

34. Paul, F. et al. Transcriptional heterogeneity and lineage commitment in myeloid progenitors. *Cell* **163**, 1663–1677 (2015).

35. Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B. & Smola, A. A kernel two-sample test. *J. Mach. Learn. Res.* **13**, 723–773 (2012).

36. Korsunsky, I. et al. Fast, sensitive and accurate integration of single-cell data with Harmony. *Nat. Methods* **16**, 1289–1296 (2019).

37. Kim, S. ppcor: an R package for a fast calculation to semi-partial correlation coefficients. *Commun. Stat. Appl. Methods* **22**, 665 (2015).

38. Wolf, F. A. et al. PAGA: graph abstraction reconciles clustering with trajectory inference through a topology preserving map of single cells. *Genome Biol.* **20**, 1–9 (2019).

39. Haghverdi, L., Büttner, M., Wolf, F. A., Buettner, F. & Theis, F. J. Diffusion pseudotime robustly reconstructs lineage branching. *Nat. Methods* **13**, 845–848 (2016).

40. Trapnell, C. et al. Pseudo-temporal ordering of individual cells reveals dynamics and regulators of cell fate decisions. *Nat. Biotechnol.* **32**, 381 (2014).

41. Kumar, P., Tan, Y. & Cahan, P. Understanding development and stem cells using single cell-based analyses of gene expression. *Development* **144**, 17–32 (2017).

42. Saelens, W., Cannoodt, R., Todorov, H. & Saeys, Y. A comparison of single-cell trajectory inference methods. *Nat. Biotechnol.* **37**, 547–554 (2019).

43. Street, K. et al. Slingshot: cell lineage and pseudotime inference for single-cell transcriptomics. *BMC Genom.* **19**, 1–16 (2018).

44. Hwang, B., Lee, J. H. & Bang, D. Single-cell RNA sequencing technologies and bioinformatics pipelines. *Exp. Mol. Med.* **50**, 1–14 (2018).

45. Wolf, F. A., Angerer, P. & Theis, F. J. SCANPY: large-scale single-cell gene expression data analysis. *Genome Biol.* **19**, 1–5 (2018).

46. Dahlin, J. S. et al. A single-cell hematopoietic landscape resolves 8 lineage trajectories and defects in Kit mutant mice. *Blood J. Am. Soc. Hematol.* **131**, e1–e11 (2018).

47. Han, G. et al. Follicular lymphoma microenvironment characteristics associated with tumor cell mutations and MHC class II expression. *Blood Cancer Discov.* **3**, 428–443 (2022).

48. Zhang, Z., Han, J., Song, L. & Zhang, X. Inferring cell-specific gene regulatory networks from single cell gene expression data. Preprint at https://www.biorxiv.org/content/10.1101/2022.03.03.482887v3.full (2022).

49. Yuan, Y. & Bar-Joseph, Z. Deep learning for inferring gene relationships from single-cell expression data. *Proc. Natl Acad. Sci.* **116**, 27151–27158 (2019).

50. Chen, J. et al. DeepDRIM: a deep neural network to reconstruct cell-type-specific gene regulatory network using single-cell RNA-seq data. *Brief. Bioinforma.* **22**, bbab325 (2021).

51. Zhang, S. Y. & Stumpf, M. P. Learning cell-specific networks from dynamical single cell data. Preprint at https://www.biorxiv.org/content/10.1101/2023.01.08.523176v2 (2023).

52. Crowell, H. L., Morillo Leonardo, S. X., Soneson, C. & Robinson, M. D. The shaky foundations of simulating single-cell RNA sequencing data. *Genome Biol.* **24**, 1–19 (2023).

53. Gao, Y., Shen, L. & Xia, S.-T. DAG-GAN: Causal structure learning with generative adversarial nets. In *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 3320-3324 (IEEE, 2021).

54. Satija, R., Farrell, J. A., Gennert, D., Schier, A. F. & Regev, A. Spatial reconstruction of single-cell gene expression data. *Nat. Biotechnol.* **33**, 495–502 (2015).

55. Goodfellow, I. et al. Generative adversarial networks. *Commun. ACM* **63**, 139–144 (2020).

56. Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V. & Courville, A. C. Improved training of wasserstein gans. In *Advances in neural information processing systems*, 30 (NeurIPS, 2017).

57. Reddi, S. J., Kale, S. & Kumar, S. On the convergence of adam and beyond. Preprint at https://arxiv.org/abs/1904.09237 (2019).

58. He, K., Zhang, X., Ren, S. & Sun, J. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*. 1026–1034 (IEEE, 2015).

59. Glorot, X. & Bengio, Y. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. 249-256 (JMLR Workshop and Conference Proceedings, 2010).

60. Ioffe, S. & Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*. 448-456 (PMLR, 2015).

61. Hu, H. et al. AnimalTFDB 3.0: a comprehensive resource for annotation and prediction of animal transcription factors. *Nucleic Acids Res.* **47**, D33–D38 (2019).

62. Shaham, U. et al. Removal of batch effects using distribution-matching residual networks. *Bioinformatics* **33**, 2539–2546 (2017).

63. Shu, H. et al. Modeling gene regulatory networks using neural network architectures. *Nat. Comput. Sci.* **1**, 491–501 (2021).

64. Mirza, M. & Osindero, S. Conditional generative adversarial nets. Preprint at https://arxiv.org/abs/1411.1784 (2014).

65. Islam, S. et al. Characterization of the single-cell transcriptional landscape by highly multiplex RNA-seq. *Genome Res.* **21**, 1160–1167 (2011).

66. Jacomy, M., Venturini, T., Heymann, S. & Bastian, M. ForceAtlas2, a continuous graph layout algorithm for handy network visualization designed for the Gephi software. *PloS One* **9**, e98679 (2014).

67. Paszke, A. et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in neural information processing systems* 32 (NeurIPS, 2019).

68. Zinati, Y., Takiddeen, A. & Emad, A. GRouNdGAN. Zenodo https://doi.org/10.5281/zenodo.11068246 (2024). article-v1.0.0.

## Acknowledgements

This work was supported by grants from Natural Sciences and Engineering Research Council of Canada (NSERC) [RGPIN-2019-04460] (A.E.), Government of Canada's New Frontiers in Research Fund (NFRF) [NFRFE-2019-01290] (A.E.), Canada Foundation for Innovation (CFI) JELF [project 40781], and McGill Initiative in Computational Medicine (A.E.). This research was enabled in part by support provided by the Digital Research Alliance of Canada (`https://alliancecan.ca`) and Calcul Québec (`www.calculquebec.ca`). *This publication is part of the Human Cell Atlas* –`www.humancellatlas.org/publications`.

## Author contributions

All authors contributed to the design of the study and the algorithm, the analyses, and the writing of the manuscript. A.E. supervised the study. Y.Z. implemented the model and performed the statistical analyses. Y.Z., A.T., and A.E. read and approved the final manuscript.

## Competing interests

The authors declare no competing interests.

## Additional information

**Supplementary information** The online version contains supplementary material available at https://doi.org/10.1038/s41467-024-48516-6.

**Correspondence** and requests for materials should be addressed to Amin Emad.

**Peer review information** Nature Communications thanks Eric Lu Zhang, who co-reviewed with Xu Yuand Hao Li for their contribution to the peer review of this work. A peer review file is available.

**Reprints and permissions information** is available at http://www.nature.com/reprints

**Publisher's note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

**Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/ licenses/by/4.0/.

© The Author(s) 2024

# Background and Context for "*Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit*"

While RAPPPID represented a meaningful advance in our ability to make accurate predictions on datasets which controlled for data leakage, generalization remained (and continues to remain) a challenge for PPI inference models. Following the publication of RAPPPID, we began to look to specific areas in which PPI inference models fail to generalize. Predicting interactions between proteins of different organism, particularly those of very evolutionary distance species, had been previously identified as a challenge for PPI inference models [81, 155]. What's more, these "cross-species" interactions are of great value to researchers. The vast majority of PPIs studied belong to a small number of model organisms representing a small fraction of all species, representing less than a third of taxa at the class rank as measured by the STRING core dataset [9]. We've named this discrepancy in elucidated PPI's the "species gap". It follows, then, that it would be greatly advantageous to construct computational methods for expanding our knowledge of PPIs in species other than a select few model organisms.

The cross-species PPI inference models of which I was aware at the time, however, conducted their evaluations on datasets which suffered from data leakage [81, 155]. Further, when evaluated on a strict, C3 dataset which controlled for data leakage, performance severely deteriorated [9].

This is the context from which our method INTREPPPID arose [9]. INTREPPPID builds upon the lessons and architecture of RAPPPID, leaving unchanged its AWD-LSTM encoder, expanding the model such that it now also modelled the orthology between proteins. Orthology here can be thought of similarities in protein sequence across species due them sharing a common ancestor [165]. Modelling orthology proved to have many synergistic effects with the cross-species task, lending itself particularly well to both theoretical and practical concerns. Theoretically, homology is a natural place to begin when comparing the interactions of two species, and forms the basis of many early analyses such as Date and Marcotte's work as well as that of Sharan *et al.* [74, 166].

Practically, the orthology modelling task can be argued to provide a regularizing effect when viewed through the lens of contrastive self-supervised learning [9, 167, 168]. Due to our novel "quintuplet" architecture, whereby an encoders weights are shared across a twin network task as well as as a triplet contrastive network, the number of parameters used in the INTREPPPID network were ostensibly identical to that of RAPPPID.

INTREPPPID successfully improved on RAPPPID's predictive performance on testing pairs between proteins of the same and differing organisms [9]. Experiments also demonstrated that the improvements in predictive performance that INTREPPPID enjoyed were due to having successfully modelled orthology, rather than any spurious or unintended cause resulting from the novel "quintuplet" architecture. INTREPPPID's efficient architecture and resulting low computational requirements has allowed us to create an online service, PPI.bio, for using INTREPPPID and RAPPPID to infer PPI interactions. Together, the work presented in the INTREPPPID manuscript represent a concrete step in the direction of narrowing the species gap.

The contents of Chapter 4 were first published on the 22nd of August, 2024 as an article of the same title in Volume 25, Issue 5, of Oxford University Press' *Briefings in Bioinformatics* journal and is made available under the Creative Commons Attribution-NonCommercial License, © Joseph Szymborski and Amin Emad, 2024. DOI: 10.1093/bib/bbae405

# Chapter 4

# Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit


## Abstract

**Background** Rapid advances in high-throughput single-cell sequencing technologies, coupled with the development of computational methods capable of leveraging large datasets, have led to the emergence of numerous approaches for deciphering regulatory interactions in the form of Gene Regulatory Networks (GRNs). However, in the absence of context-specific gold-standard ground truths, particularly those containing causal interactions, systematically benchmarking GRN inference methods remains a challenge. Thus, we previously developed GRouNdGAN, a causal implicit generative model capable of simulating realistic observational and interventional scRNA-seq data following a user-defined GRN on any biological system of interest. Importantly, we demonstrated that GRouNdGAN generates datasets that bridge the gap between experimentation and simulation for GRN inference benchmarking.

**Method** Building upon the GRouNdGAN framework, we developed an extended toolkit that offers additional features, including interactive model visualization, training monitoring, more customizable GRN creation options, synthetic data similarity and GRN inference benchmarking metrics, and an intuitive TF knockout prediction module. Here, we provide a step-by-step procedure for implementing the protocol from start to finish and introduce alternative variations to adapt GRouNdGAN to studies with different experimental setups. GRouNdGAN-Toolkit is publicly available as a python code repository and containerized application and is accompanied by a tutorial website featuring a collection of simulated datasets. Model training largely depends on graphic hardware and the size and density of the input GRN, and typically takes around 75h to complete on a single GPU. Excluding model training, this protocol typically takes less than

25min to complete.

**Discussion** GRouNdGAN-Toolkit is a versatile simulator with user friendly interface that does not assume advanced computational genomics expertise, enhancing its usability and accessibility across a wide range of users.

**Keywords**: GRouNdGAN, Gene Regulatory Networks, GRN Inference, GRN Inference Benchmarking, scRNA-seq, Data Augmentation, In-Silico Perturbation.

## Introduction and Background

Understanding how the gene expression is regulated within cells, including when, where, and to what extent a gene is transcribed and translated into a functional product like a protein, is fundamental to molecular biology[1, 2]. These regulatory processes, collectively known as gene regulation, are often conceptualized as transcriptional or gene regulatory networks (GRNs), which capture the dynamic interactions among genes and their regulators. To allow for computational analysis, GRNs are often modeled as directed acyclic bipartite graphs, linking transcription factors (TFs) to their target genes[3]. Inferring GRNs from bulk or single-cell RNA sequencing data[4-7] alone or in combination with other data modalities[8-10], has long been a topic of significant interest. This is due to the fact that GRNs play a critical role in our understanding of biological processes involved in cellular development, function, and disease, with applications in identifying therapeutic targets and predicting the outcomes of perturbation experiments[11]. However, systematically benchmarking GRN inference tools remains a challenge due to the absence of context-specific gold-standard ground truth networks and their associated datasets, particularly those that contain causal interactions[12]. While a ground truth GRN could be constructed from literature and curated biological databases of interactions, it may not accurately reflect the context- and cell type-specific regulatory programs of the biological system under study as they are aggregated across various datasets. Experimental validation through perturbation experiments could provide definitive evidence for the presence or absence of individual interactions, but such approaches are costly, time-consuming, and laborintensive[11, 13, 14]. To address this, multiple GRN-guided simulators have been developed, offering an alternative means of generating benchmarking datasets[12, 15-17]. However, existing simulators often fail to impose regulatory relationships causally within their generated data and have key limitations (see our previous study for a detailed discussion[18]). Moreover, reported mismatches between benchmarking results obtained from experimental and simulated datasets raise concerns about their reliability[12].

## Development of the protocol

We developed GRouNdGAN[18], a generative deep learning model capable of simulating realistic single cell data, performing in-silico perturbation experiments, and systematically benchmarking GRN inference methods. GRouNdGAN combines and builds upon elements from previous studies on generative adversarial networks (GANs) namely scGAN[19] and causalGAN[20], to generate synthetic single cells mimicking those within a provided reference dataset while imposing causal relationships between TFs and genes as defined by a user-provided GRN.

Our approach follows a two-step training process. First, we train a Wasserstein GAN[21] with gradient penalty (WGAN-GP)[22] to generate realistic single cells without the regulatory constraints of the GRN. Once trained, we freeze the parameters of the generator to use as a "causal controller" in training a second WGAN-GP. In the second step, random Gaussian noise, along with TF expression values coming from the causal controller, are provided to a generator (called the "target generator") which derives its neural network connections from the provided GRN. The masking of neural connections in the target generator ensures that target genes are causally expressed under the influence of their regulating TFs and some unobserved/unknown confounders (modelled by the inputted random noise unit). Both WGAN-GPs include a library size normalization layer as their last layer. Since only unnormalized TF expression values are needed from the causal controller, the LSN layer of the first WGAN-GP is discarded after it is trained in the first step. These TF expression values, along with the target gene expression values regenerated by the target generator, are then fed into the LSN layer of the target generator for final normalization.

Both WGAN-GPs are trained in an adversarial manner, with a "Critic" that evaluates generated samples by estimating the Wasserstein distance between the reference and simulated data distributions, and a generator that continuously refines its output to minimize this distance. Through this minimax game, both components co-evolve in a dynamic feedback loop, each improving in response to the other.

Unlike traditional GANs, GRouNdGAN employs two auxiliary networks, the labeler and antilabeler, to ensure the target generator incorporates TF expression when generating target gene expression values, rather than relying solely on the input noise. These networks predict TF expression from target gene expression by minimizing the L2 loss between estimated and true TF values. The anti-labeler is trained exclusively on generated data, while the labeler is trained on both real and generated data to make further connection to the biology of the reference dataset. This approach reinforces causal TF-gene relationships within the GAN framework ensuring that regulatory consistency and realistic single-cell generation are optimized in tandem.

## Applications of the method

In our original study[18], we demonstrated that GRouNdGAN possesses essential properties for systematically benchmarking of GRN inference algorithms. This was achieved by generating benchmarking datasets based on six experimental datasets, and various causal GRN configurations. In summary, GRouNdGAN generates realistic scRNA-seq data while simultaneously imposing the causal relationships defined by the user (provided as an input GRN). It ensures that imposed edges are emphasized, making them recoverable by a high-performing GRN inference algorithm. Conversely, regulatory relationships that were intrinsic to the reference biological data, yet were not part of the input GRN are disrupted in the simulated dataset, limiting the potential of bias and ensuring a fair evaluation of GRN inference methods. We observed that benchmarking results of seven GRN inference methods using GRouNdGAN were consistent with insights previously found by BEELINE[12] from curated and experimental datasets, suggesting that GRouNdGAN effectively bridges the gap between experimental and simulated data benchmarks. Since then, GRouNdGAN has facilitated the development and evaluation of new GRN inference methods, such as LLM4GRN[3] and scRegulate[23], enabling researchers to carry out a systematic comparative evaluation of their method compared to the state-of-the-art. Figure 1 demonstrates an overview of the GRouNdGAN pipeline.

In addition to sampling from observational distributions to generate realistic benchmarking datasets, GRouNdGAN can also sample from interventional distributions and generate cells that do not naturally occur in the dataset, since it is a causal generative model. We previously showcased this application by performing in silico knock out experiments on top differentially expressed TFs of specific cell types, and observing their effect on the generated cells. The in silico perturbations can be performed during inference by manipulating the TF expressions generated by the causal controller, which are then fed to the target generator to produce the regulated target genes. In GRouNdGAN-Toolkit, we provide a user-friendly interface that allows users to generate cells under various perturbation conditions such as knockout. This application is not only suitable for predicting the outcome of specific perturbations but also for creating case/control experiments for studies that require such setups. Since our in silico perturbation is applied to the same batch of cells, keeping all generation parameters such as noise constant, it is deterministic, allowing to observe the effect of perturbations on the exact set of cells, providing matched case/control simulated datasets. This is not easily possible on experimental scRNA-seq data since most sequencing technologies are destructive, preventing the same cell from being sequenced twice (before and after perturbation).

As a specialized extension of general-purpose simulators, GRouNdGAN can fulfill all the same functions of non-causal simulators, when a GRN that accurately represents the reference dataset is imposed. Interestingly, we previously observed that general-purpose simulators that do not

explicitly impose a causal graph, focus on generating realistic cells at the expense of compromising TF-gene relationships of the reference experimental dataset[18]. In this regard, GRouNdGAN may be a better choice even for tackling use cases typically handled by generalpurpose simulators, as we have previously demonstrated that it outperforms state-of-the-art simulators in generating realistic data[18]. Here, we will briefly describe some use-cases of simulated data that GRouNdGAN can also support. Simulated data can be used for data augmentation to supplement limited sample availability caused by biological constraints, such as rare cell types, specific contexts, or populations. It can also help overcome technical limitations, like low-quality or low-yield sequencing data, and address ethical concerns related to human or animal experiments. The ability to generate synthetic data can significantly reduce the need for wet-lab experiments, which leads to considerable time and cost savings, particularly when testing biological hypotheses. Moreover, since GRouNdGAN can generate an unlimited number of cells without duplicates, it helps overcome challenges related to small sample sizes, which might not accurately reflect the broader population and can lead to poor robustness and reduced reproducibility of experimental results, though further validation is needed to confirm its effectiveness in this context.

In our original study, we showed that GRoundGAN-generated datasets effectively preserve key biological features such as cellular trajectories, pseudo-time ordering of cells, and marker gene expression. As a result, GRouNdGAN-generated data has the potential to improve downstream analyses, such as marker gene detection, classification of sparse cells, and trajectory inference, as previously demonstrated by scGAN[24] on their own data. Moreover, augmented data can enhance denoising and dropout imputation algorithms. Traditional imputation methods typically rely on borrowing information from similar cells within the dataset, but this approach can lead to oversmoothing, particularly for rarer cell populations, ultimately suppressing natural cell-tocell variability. sciGAN[25] for instance, has demonstrated that synthetic cells can be used for information borrowing, rather than relying solely on observed cells, thus preserving the inherent stochasticity of the data. Finally, simulated data has the potential to be studied as means of sharing transcriptomic datasets at a reduced risk of exposing sensitive information, all while preserving the statistical and biological integrity of the original experimental reference data. However, demonstrating the effectiveness of this approach in addressing privacy concerns remains an ongoing challenge and is beyond the scope of our current work.

## Comparison with other methods

Several tools have been developed for simulation of scRNA-seq data over the years[24, 26-29]. However, most are general-purpose simulators that mainly focus on generating high-fidelity data that adheres to the statistical properties of experimental datasets. Using such simulators, one can augment data, improve downstream analysis, and benchmark some computational tools

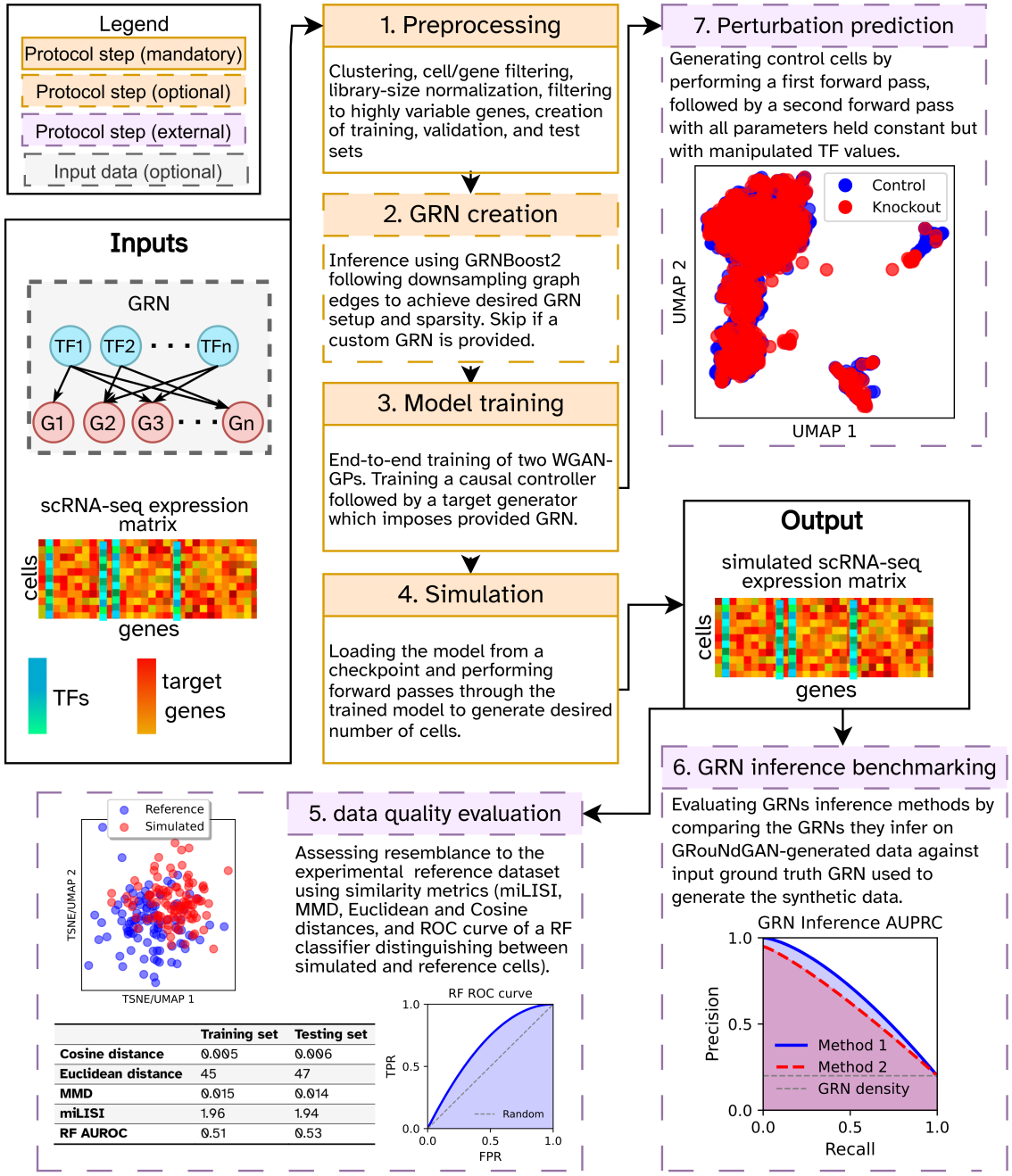



**Figure 1:** Schematic overview of the GRouNdGAN pipeline. Left: GRouNdGAN's input data consists of a scRNA-seq dataset and an optional user-provided GRN. A list of TF names must be provided to differentiate TFs from target genes. Middle (light orange): Main steps in the GRouNdGAN protocol. The GRN creation step can be skipped only if a GRN is provided by the user. Right and bottom (in light purple): External protocol steps that involve using GRouNdGAN's trained model or simulated data for secondary analysis and downstream tasks.

but cannot benchmark GRN inference algorithms. These simulators have shown to capture genegene correlations, but do not explicitly simulate data following a regulatory program that is either user-provided or extracted from the dataset, making them unsuitable for benchmarking GRN inference algorithms. Only a small number of simulators are explicitly guided by a user-defined GRN. One of the earliest models is GeneNetWeaver (GNW)[17] that was used to simulate bulk RNA-seq for benchmarking multiple GRN inference methods during Dialogue for Reverse Engineering Assessment and Methods (DREAM) challenges[17, 30, 31]. However, since it was originally designed for bulk RNA-seq data generation, its simulated datasets do not usually exhibit the statistical properties of single-cell data. Although there have been multiple attempts at adapting GNW to single-cell data, it falls short of modern simulators and cannot simulate data with multiple cell types, contexts, or trajectories[15].

SERGIO[15] and BoolODE[12] are single-cell GRN-guided simulators that use stochastic differential equations (SDEs) to model gene expression dynamics. BoolODE is a de novo simulator, meaning that it generates data entirely from scratch based solely on user-defined parameters and rules, without relying on a reference biological dataset. As a result, it cannot be used to benchmark GRN inference methods under specific contexts and conditions that are unique to an experimental dataset. Instead of a reference dataset, BoolODE relies on carefully selected user-defined parameters such as noise, dropout rate, and kinetic parameters to impose properties such as dropout, batch effect, variation across samples, population, and conditions. Although this gives the end-users greater control over the data generating process, it also places the responsibility of fully harnessing this flexibility on them, making it more challenging to generate new datasets. In other words, without extensive knowledge of single-cell transcriptomics and BoolODE's inner workings, one may risk creating simulations that do not reflect experimental datasets. SERGIO[15] is a hybrid de novo/reference based simulator that first generates a "clean" dataset, strictly following a GRN without aligning to a reference dataset. The user then refines this simulated data by iteratively adjusting three parameters (controlling technical noise) to match a reference dataset, by assessing similarity based on five statistical measures: library sizes, mean gene expression, gene expression variance, and zero counts per gene and per cell. This approach introducessubjectivity that affects reproducibility of the results. In our experiments, even after the addition of technical noise, data quality remained subpar compared to other simulators. Furthermore, since technical noise is introduced after the causal relationships have already been encoded, it may unintentionally alter or disrupt interactions that should remain unchanged for a GRN benchmarking experiment. This can make relationships unidentifiable in the final dataset, potentially explaining the significant drop in the performance of all GRN inference methods tested by the authors in the original study[15].

On the other hand, GRouNdGAN optimizes its parameters to simultaneously impose causal TFgene relationships while minimizing the distance between simulated and reference data distri-

butions. This process is fully automated, requiring no user intervention or post-processing to preserve technical and biological noise, and cellular dynamics (such as pseudotime and trajectories). Unlike SERGIO and BoolODE, which generate pseudo-genes without clear mappings to the reference dataset, GRouNdGAN ensures a direct one-to-one correspondence between genes and TFs in both simulated and reference data. Each generated gene and TF mimic the expression patterns of their real counterparts. In contrast, preserving gene/TF identities in BoolODE and SERGIO requires manual fine-tuning of SDE parameters to align generated gene expressions with reference data, placing a significant burden on users. Finally, GRouNdGAN learns elaborate non-linear co-regulatory dynamics through implicitly parameterized functions implemented as neural networks from the data and without making simplifying assumptions. This is in contrast to the simplifying assumption of SERGIO that assumes the combinatorial effect of multiple regulators is the sum of their individual effects, or BoolODE that relies on a userprovided Boolean truth table for TF co-regulation rules.

## Experimental design

Besides configurations, GRouNdGAN requires only two inputs: (1) a scRNA-seq dataset (as the reference) and (2) a gene regulatory network (GRN). It produces a single output: a simulated scRNA-seq dataset that mimics the input reference dataset, while encoding the regulatory information from the input GRN within the simulated data. By preparing the inputs and using the output in a specific way (described in this manuscript), one can carry out different types of experiments, which we will explore in this section.

### Input reference data and GRN considerations

GRouNdGAN's architecture and optimization function ensures that data simulation is guided by a structural causal model defined by the input GRN. Unlike general-purpose simulators that prioritize generating realistic data often at the cost of losing underlying regulatory information, GRouNdGAN optimizes for both GRN imposition and the realism of the simulated data. One should however note that while GRouNdGAN can faithfully encode any GRN into the data it simulates, the resemblance of its output to the reference dataset deteriorates when the input GRN deviates considerably from the patterns present in the reference. In such cases, optimizing for both objectives (realistic data generation and GRN imposition) becomes contradictory rather than complementary. For this reason, we suggest users to impose GRNs that are generally in line with the reference dataset, though strict matching is not required for most applications. Having access to regulatory programs underlying the data is especially important for predicting the result of interventions. GRouNdGAN's default approach involves building a scaffold GRN

by inferring regulatory edges using GRNBoost2[5], then imposing a subset by retaining the top k interactions for each gene. The value of k can be adjusted to control the sparsity of the graph, with a denser GRN typically providing more information and leading to higher-quality simulations.

Instead of the default approach, one can use a custom GRN. Custom GRNs can be constructed by selecting a subset of edges other than top k (for example top $1^{st}$, $3^{rd}$, $5^{th}$, ...) or those inferred by an algorithm other than the default option. Additionally, they can be constructed using data sourced from literature, structured curated biological databases (e.g., KEEG[32], I2D[33], IID[34], ESCAPE[35], and GTRD[36]) or other modalities(e.g., TF ChIP-seq or ATAC-seq). These sources can also be combined to create a more complete GRN. The edges in the input GRN do not have to reflect the true causal relationships in the reference dataset. They could arise from spurious correlations or other observed associations. However, once imposed by GRouNdGAN, these edges become the underlying causal edges of the simulated dataset.

In our original study[18], we imposed a single shared GRN on a highly heterogeneous dataset of human peripheral blood mononuclear cells (PBMC) and obtained state-of-the-art performance. However, since regulatory programs are cell type- and context-specific, it may be beneficial to impose different GRNs on subpopulations of the dataset, rather than using a single shared GRN, depending on the application. This can be easily achieved by training multiple population-specific GRouNdGAN models, each using a subpopulation of the experimental dataset along with a GRN specific to that subpopulation. To demonstrate this experimental setup, here we trained five GRouNdGAN models using cell-type-specific GRNs for the five most populated cell types in the PBMC dataset[37]. We followed the procedure outlined in Figure 2-a where preprocessing is done on the entire dataset after which it is separated based on cell type and finally used to construct cell type-specific GRNs. These cell type-specific GRNs depict distinct regulatory programs as shown by Figure 2-b, while still sharing some regulatory edges. Table 1 highlights strong performance of cell type-specific models across five metrics in generating realistic data. While this approach offers some advantages, it cannot leverage the potential benefits of information sharing, nor the performance improvements that come from using a larger training sample. Nonetheless, the cell type-specific modelsstill achieved performance comparable to that of the non-cell-type specific model (PBMC-ALL in Table 1) despite being trained on fewer samples (See Table S1). By aggregating the outputs from cell type-specific models in proportion to the relative abundance of each cell type, one can effectively reconstruct the overall dataset simulation, similar to a non-cell-type specific model (see Figure 2-c).

When there is no need to integrate the simulation across cell types, the dataset may first be partitioned by cell type, followed by independent preprocessing and GRN construction for each subset. This approach may offer some advantages: for example, one can separately preprocess and identify highly variable genes specific to each cell type. As a result, selected genes and TFs

are more biologically relevant to specific cell types, leading to the inference and construction of more representative GRNs. Table S2 summarizes the performance of cell type-specific models under this alternative experimental setup.

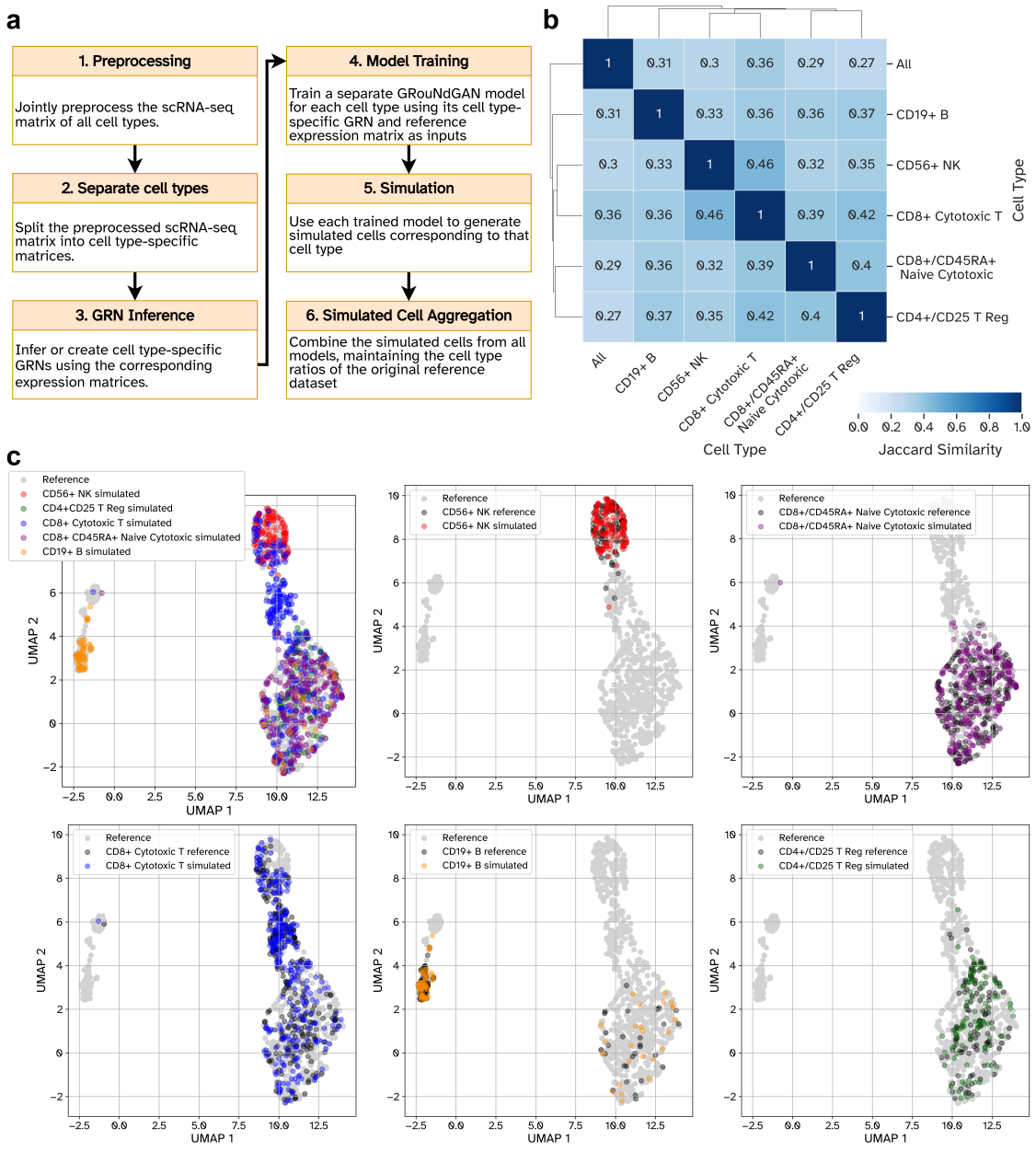



**Figure 2: Cell type-specific simulation of cells using GRouNdGAN.** Panel **a** illustrates an experimental setup designed for running simulations with cell type-specific GRNs. **b** Clustered heatmap showing pairwise Jaccard similarity between GRNs constructed for each cell type, based on shared TF-gene regulatory edges. Hierarchical clustering was done using average linkage and Euclidean distance. **c** Subplots show the UMAP embedding of cell type-specific simulations overlaid on reference (real) cells. The top left subfigure displays an aggregated UMAP where simulated cells from all cell type-specific models are combined in proportions matching the reference dataset (see Table S1 for relative abundance of cell types). The overlaid reference cells serve as a visual benchmark for simulation data quality.

For comprehensive benchmarking studies, we recommend that users generate multiple simu-

**Table 1:** Performance of GRouNdGAN in generating realistic scRNA-seq data using cell-typespecific GRNs using the PBMC dataset. The metrics are calculated by comparing a held-out test set of 1000 real cells and a simulated dataset of the same size. In the imposed GRN each gene is regulated by a maximum of 15 TFs. The default algorithm GRNBoost2 was used with the training experimental data to form the imposed GRN. For the cosine distance, Euclidean distance, and MMD, values closer to zero are desired. For RF AUROC a value closer to 0.5 and for miLISI a value closer to 2 reflect a more realistic simulation.

| Simulator | Cell Type | Cosine distance | Euclidean distance | MMD | RF AUROC | miLISI |
|---|---|---|---|---|---|---|
| GRouNdGAN | CD8+ Cytotoxic T | 0.00023 | 175 | 0.0137 | 0.53 | 1.889 |
| Control | CD8+ Cytotoxic T | 0.00017 | 129 | 0.0136 | 0.50 | 1.902 |
| GRouNdGAN | CD8+/CD45RA+/Naïve Cytotoxic | 0.00012 | 134 | 0.0161 | 0.51 | 1.902 |
| Control | CD8+/CD45RA+/Naïve Cytotoxic | 0.00011 | 135 | 0.0145 | 0.50 | 1.906 |
| GRouNdGAN | CD56+ NK | 0.00027 | 126 | 0.0181 | 0.59 | 1.885 |
| Control | CD56+ NK | 0.00022 | 116 | 0.0154 | 0.50 | 1.908 |
| GRouNdGAN | CD4+/CD25 T Reg | 0.00011 | 126 | 0.0113 | 0.54 | 1.895 |
| Control | CD4+/CD25 T Reg | 0.00011 | 126 | 0.0113 | 0.50 | 1.914 |
| GRouNdGAN | CD19+ B | 0.00018 | 117 | 0.0116 | 0.55 | 1.897 |
| Control | CD19+ B | 0.00023 | 125 | 0.0116 | 0.50 | 1.906 |
| GRouNdGAN | PBMC-ALL | 0.00018 | 175 | 0.0157 | 0.50 | 1.914 |
| Control | PBMC-ALL | 0.00019 | 138 | 0.0160 | 0.50 | 1.905 |

lated datasets with varying characteristics such as different GRN densities, different topologies, different number of TFs, and different number of target genes. Another approach that can enhance the rigor of a benchmarking study (previously demonstrated in our original study) involves creating two distinct GRNs. Each GRN would consist of half of the top k regulating TFs for each gene, with one serving as an unimposed control and the other used for simulation. The unimposed control GRN can serve as baseline for computing performance metrics.

### Output data considerations

GRouNdGAN includes built-in functionalities to assess both data quality and GRN inference performance. Data quality is evaluated using the Area Under the Receiver Operating Characteristic Curve (AUROC) of a Random Forest (RF) distinguishing experimental from simulated cells, mean integration local inverse Simpson's index (miLISI)[38], Maximum Mean Discrepancy (MMD), and the Euclidean and Cosine distances between the centroids of experimental and simulated cells. GRN inference performance is measured with the Area Under the Precision-Recall Curve (AUPRC) and precision. For detailed explanations of each metric, please refer to our original publication.

A key advantage of using simulation for benchmarking GRN inference methods is the ability to generate an unlimited number of samples, which offers significant benefits over subsampling a finite experimental dataset. This enables comprehensive benchmarking by assessing GRN

inference algorithms in terms of computational resource usage (e.g., time and memory efficiency on large datasets), and enables quantifying their stability across multiple runs, evaluating their robustness to noise, and optimizing their performance for specific applications. To demonstrate this, we assessed the performance of PIDC[6] and PPCOR[7] on simulated datasets ranging from 100 to 100,000 cells, with the results presented in Figure 3. The reference dataset (PBMC-CTL) for this consists of the most abundant cell type of the PBMC dataset[37] (i.e., CD8+ Cytotoxic T cells). Figure 3-a, b, c show that GRouNdGAN-generated cells closely resemble real cells across multiple quality metrics and in low-dimensional t-SNE space. These results are based on 100 replicate comparisons, comparing a held-out test of 1000 real cells and 100 sets of simulated cells (each containing 1000 cells for a total of 100,000), demonstrating consistent and high data quality of GRouNdGAN simulations.

As expected, we observed an improvement in AUPRC for GRN recovery as the number of cells increased (Figure 3-d) with PIDC outperforming PPCOR. PPCOR fails on very small datasets (100– 500 cells) and PIDC becomes computationally prohibitive (taking longer than 12 hours to run) on large datasets (>80,000 cells). These results underscore that GRouNdGAN can not only be used for data augmentation to increase the stability of downstream computational tools, but also to systematically evaluate their robustness and scalability.

## Expertise needed to implement the protocol

Some familiarity with the command line and basic understanding of the Python programming language are required to execute this protocol. Our protocol assumes no advanced bioinformatics expertise; though, such expertise may be necessary for downstream tasks and analysis involving the output of this protocol.

## Discussion and Limitations

While GRouNdGAN is a powerful generative model, it has inherent limitations that need to be considered. Through adversarial training, GRouNdGAN learns to approximate the distribution of training samples. Once trained, it samples simulated cells from the learned probability distribution. As such, like most reference-based simulators, its ability to generalize and produce realistic outputs depends not only on the number but also the quality and diversity of training examples. Despite that, our previous study revealed that GRouNdGAN can introduce causality and capture properties such as pseudo-time ordering and cellular trajectories on relatively small datasets and excessively large datasets do not substantially improve performance beyond a certain point.

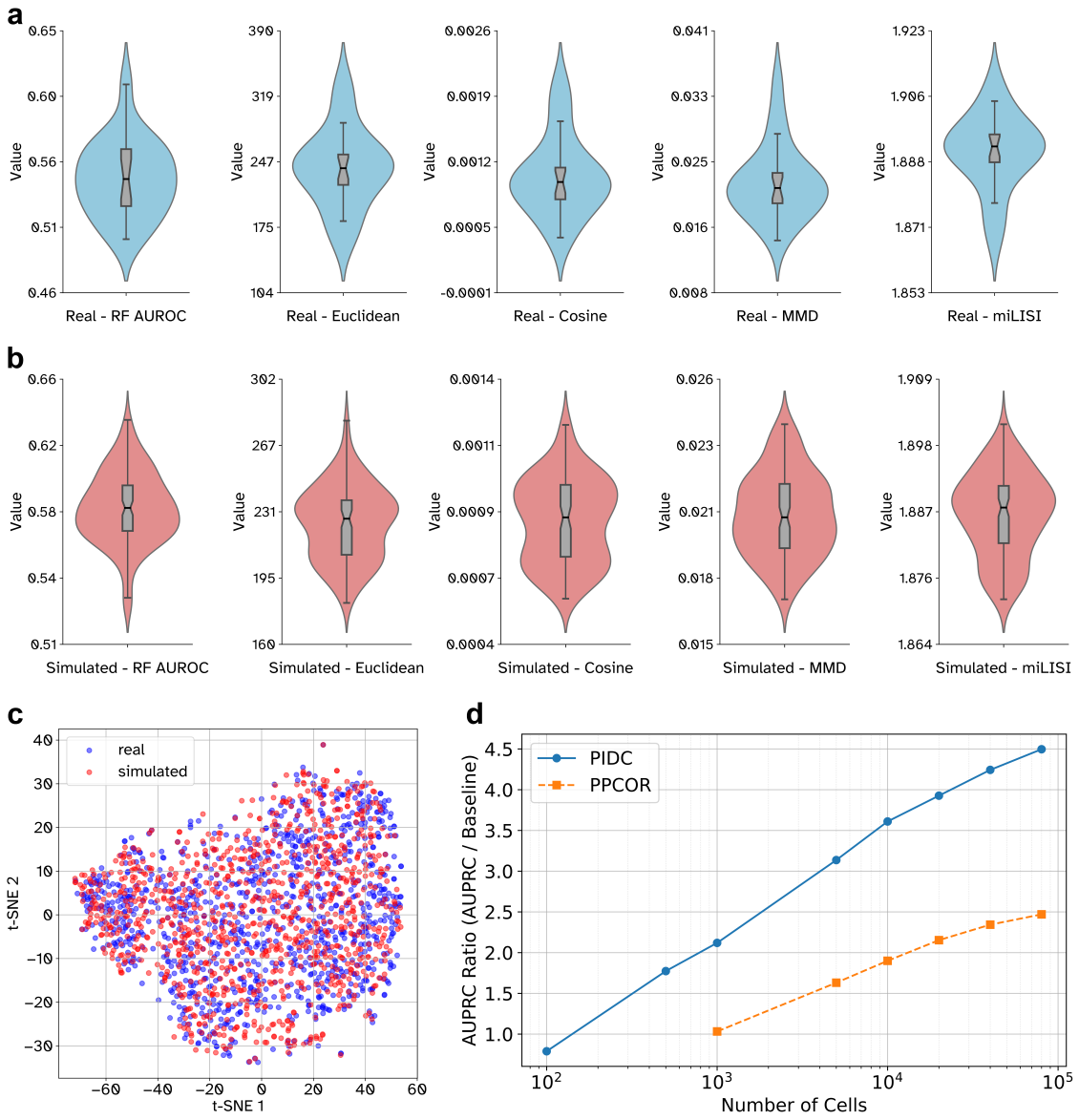



**Figure 3: Robustness analysis of simulated data.** **a** Violin plots showing five data quality metrics computed between partitions of 19000 reference training experimental cells (1000 cells each) and a held-out test set of 1000 experimental PBMC-CTL cells, serving as real vs real control benchmark. **b** Similar to **a** but computed between 100 distinct simulated datasets (each containing 1000 cells) and the same experimental test set. **c** t-SNE plot of 1000 simulated cells and the same 1000-cell test showing a visual assessment of the distributional similarity between generated and experimental cells. **d** AUPRC ratio (compared against the GRN density) for PIDC and PPCOR in inferring GRNs from simulated GRouNdGAN scRNA-seq data. The x-axis (log scale) represents the number of cells provided. The imposed GRN contains the top 15 regulating TFs for each target gene. Performance improves when increasing the number of provided cells, with PIDC outperforming PPCOR. PIDC takes over 12 hours to run on more than 80000 cells and PPCOR fails to output a GRN when provided with 100 and 500 cells.

Throughout all our analyses, we consistently used the same model architecture and hyperparameter settings, achieving state-of-the-art performance with different choices of GRN across diverse datasets spanning various cell-types, conditions, species, and sequencing technologies.

However, when training GRouNdGAN to a completely new dataset, especially one with a significantly different number of genes or transcription factors, we recommend fine-tuning hyperparameters using a validation set. GRouNdGAN's current architecture incorporates a library-size normalization (LSN) function as the last layer of its causal generator. The LSN layer ensures that all generated cells maintain a consistent fixed library size across selected genes and TFs. Our ablation study[18] confirmed that this layer strengthens GRN imposition and enhances the quality of generated cells. Additionally, our observations indicate that it accelerates model convergence and increases training stability, findings that align with previous GAN-based research in scRNA-seq[24]. While we offer the option to use GRouNdGAN without the LSN layer for flexibility in specific biological analyses requiring unnormalized count data, we advise against this unless necessary as it reduces the quality of the simulated data.

GRouNdGAN's main objective is two-fold: to generate realistic scRNA-seq data that aligns with the reference dataset and to impose causal relationships of a given GRN. If the TF-gene regulatory relationships described by the GRN are overly inconsistent with the patterns in the reference data, the two objectives become contradictory and impossible to optimize simultaneously. Conversely, providing a GRN consistent with the reference dataset can serve as additional knowledge, improving the quality of simulated cells compared to simulators that do not incorporate regulatory information. Hence, we suggest creating the input GRN from the dataset itself or using other modalities pertaining to the same biological system under study.

The model is currently confined to capturing gene regulation through a bipartite GRN (TF to gene). Gene regulation is inherently more complex, involving gene-gene interactions, multi-level regulation, feedback loops, and self-regulation—elements this model doesn't fully capture. However, we should mention that this simplified GRN model is widely used by GRN inference algorithms alike and avoids overcomplicating computations. We plan to relax this limitation as much as possible in future versions of GRouNdGAN.

## Materials and Methods

### Equipment

#### Hardware

This protocol was run on a high-performance compute cluster with 2 cores of an AMD EPYC 7413 (@ 2.65 GHz, 128M cache L3) processor, 16GB of RAM, and a single NVidia A100SXM4 (40 GB memory) GPU. Additionally, we conducted tests of the protocol on a desktop workstation equipped with an 11th Gen Intel Core i7-11700 processor (@ 2.50 GHz, 8 cores), 32GB

of RAM, and a GeForce RTX 3070 GPU (24GB memory). At minimum, we recommend a dual-core computer with 16GB of RAM, and a GPU with 24GB of video memory. However, it is still possible to run GRouNdGAN on devices with smaller VRAM than 24GB by making the input GRN sparser or reducing the model size through hyperparameters. An internet connection is required for setting up the compute environment and downloading GRouNdGAN and input data files.

#### Software

Operating system: Linux, Windows, or MacOS

- Python version 3.9.6
- Docker or Singularity, if running GRouNdGAN via containerized environments.
- GRouNdGAN: a free and open-source implementation maintained on GitHub at `https://github.com/Emad-COMBINE-lab/GRouNdGAN`.
- Basic command-line tools such as `curl`, `tar`, `git`, etc.
- (Optional) Jupyter interactive computing environment for secondary analysis
- (Optional) `pyenv` to install and switch between GRouNdGAN's required Python version and other installed versions, along with the `pyenv-virtualenv` plugin to manage Python virtual environments.
- (Optional) `SCANPY`, `Seurat`, or other single-cell processing pipelines for converting raw single-cell expression data into appropriate input formats.

#### Configuration file

Each GRouNdGAN command requires providing the path to a configuration file which specifies the protocol's hyperparameters and inputs. This configuration file (with a `.cfg` extension) employs a structure closely similar to that of Microsoft Windows INI files, which can be easily customized by the users. Aside from configurations regarding input and output parameters (discussed in each procedure step), the content of the base configuration file can remain unchanged. GRouNdGAN has consistently well-performed across datasets with the same set of hyperparameters specified in the base configuration file. Although GRouNdGAN is not overly sensitive to the choice of hyperparameters, we still advise to test different hyperparameters on a validation set when training on a new dataset with significantly different dimensions in terms of

number of TFs or target genes. A default base configuration file is available in Supplementary Notes and our code repository (path: `configs/causal_gan.cfg`) but can also be directly downloaded from the command line by running the following command:

```
curl -O https://raw.githubusercontent.com/Emad-COMBINElab/GRouNdGAN/
master/configs/causal_gan.cfg
```

#### Input data files

The protocol requires two user data inputs: raw single-cell RNA-seq count data and a list of known TFs belonging to the species being studied. GRouNdGAN accepts the scRNA-seq matrix as an AnnData object stored in the standard hierarchical `.h5ad` format. The AnnData object, at a minimum, requires the `X` and `var` fields to contain the expression data matrix and gene identifiers (e.g., Ensembl Gene ID, HGNC (HUGO Gene Nomenclature Committee) Symbol, or NCBI Entrez Gene ID), respectively. Note that the expression matrix needs to be stored as a dense (and not sparse) matrix. Alternatively, GRouNdGAN can accept 10x Genomics-formatted directories that include `matrix.mtx` (gene expression matrix in Matrix Market Exchange format), `genes.tsv` (list of features to be used as gene identifiers as discussed), and `barcodes.tsv` (list of cell barcodes representing unique cells) files. Users can optionally supplement that input with a `.tsv` file for additional cellular annotations and metadata such as cell type and cluster. To match TFs to their expression data, the list of known TFs must use the same identifier type that were chosen to uniquely name genes in the count matrix. Users can decide to omit GRouNdGAN's default causal GRN preparation procedure and instead define a custom causal graph. In that case, a causal graph should be supplied in lieu of the TF list. See Procedure – Step 2 for the causal graph's format.

### Equipment setup

#### Software Installation

Depending on user preferences and requirements, GRouNdGAN can either be locally installed or run as a containerized application. Local installation is recommended for users that need greater control over GRouNdGAN's environment or decide to build new projects upon its framework as a foundation for new bioinformatics work. To reproduce our results or run the protocol using a pre-configured environment where installation is not needed, we suggest using our Docker or Singularity (where Docker is not allowed; e.g, HPC clusters) images, both of which are configured with CUDA support for GPU acceleration. GRouNdGAN's latest version

can be downloaded from our GitHub repository's release page: https://github.com/Emad-COMBINElab/GRouNdGAN/releases/.

#### Local Installation

- **TIMING ~12 min**

For local installation, python 3.9.6 is required. We recommend isolating GRouNdGAN's environment from the base environment to avoid breaking package dependencies either using python's virtualenv module or pyenv coupled with pyenv-virtualenv to manage multiple python versions. Here, we show how to set up GRouNdGAN with Python's own virtual environment module (venv) and package management system (pip).

1. Start with cloning GRouNdGAN's GitHub repository to a directory of your choosing.

   ```
   $ git clone https://github.com/Emad-COMBINE-lab/GRouNdGAN.git
   ```

   To fully reproduce our primary study[41], one can additionally include specific versions of scGAN, BEELINE[12], scDESIGN2[42], and SPARSIM[26] projects as submodules when cloning the repository.

   ```
   $ git clone –recurse-submodules https://github.com/Emad-COMBINElab/GRouNdGAN.git
   ```

2. Navigate to the cloned directory.

   ```
   $ cd GRouNdGAN
   ```

3. Initialize a new virtual environment for the project (here named `groundgan_env`).

   ```
   $ python -m venv groundgan_env
   ```

   If you chose to use pyenv and pyenv-virtualenv, visit their tutorial and installation guides for your operating system at https://github.com/pyenv/pyenv and https://github.com/pyenv/pyenvvirtualenv.

4. Activate the created virtual environment. Linus or MacOS:

   ```
   $ source groundgan_env/bin/activate
   ```

   Windows:

```
$ groundgan_env
```

5. Project dependencies can now be installed in the created virtual environment through pip from the Python package index (PyPi) using the `requirements.txt` file available at the root of the cloned directory.

```
(groundgan_env)$ pip install -r requirements.txt
```

#### Using the Docker Image

- **TIMING ~6 min**

GRouNdGAN's docker image is available on the DockerHub at https://hub.docker.com/r/yazdanz/groundgan.

1. Run the following command to pull the latest image from DockerHub:

```
$ docker pull yazdanz/groundgan:GRouNdGAN-toolkit
```

2. To run the Docker container, execute the following:

```
$ docker run –gpus all -it yazdanz/groundgan:latest /bin/bash
```

The `--gpus all` and `–it` flags enable GPU support within the container and start an interactive terminal session respectively.

#### Building Docker Image from Dockerfile

- **TIMING ~13 min**

We only recommend this option if you wish to further finetune the provided containerized environment to your specific requirements and maintain reproducibility.

1. Clone GRouNdGAN's GitHub repository to a directory of your choosing.

```
$ git clone https://github.com/Emad-COMBINE-lab/GRouNdGAN.git
```

2. Navigate to the project directory.

```
$ cd GRouNdGAN
```

3. Make intended updates and modifications to the Dockerfile (path: `docker/Dockerfile`)

4. Build the Docker image using the provided Dockerfile. The command below will build a docker image with the tag `yourusername/groundgan:custom` which you can change.

```
$ docker build -t yourusername/groundgan:custom -f
docker/Dockerfile .
```

5. Once the image is built, start the docker container interactively and pass GPU devices with the following command. You can also decide to publish it to DockerHub.

```
$ docker run -itd –name yourusername/groundgan:custom –gpus all
groundgan /bin/bash
```

#### Using Singularity

- **TIMING ~7 min**

1. Pull and convert the container image from Dockerhub into the Singularity Image Format (SIF). The following command creates a singularity image with the `groundgan.sif` filename.

```
$ singularity pull groundgan.sif docker://yazdanz/groundgan
:GRouNdGAN-toolkit
```

2. Start an interactive shell session within the groundgan.sif container. The –nv flag enables NVIDIA GPU hardware support and exposes the host system's drivers and libraries to the container.

```
$ singularity shell –nv groundgan.sif
```

   You can now execute shell commands inside the running container.

#### Verifying GPU Acceleration

- **TIMING ~<1 s**

You can ensure that your device's GPUs are recognized inside Docker and Singularity containers by running the `nvidia-smi` command. You should see detailed information regarding your GPU and installed drivers. See an example of the expected output below.

```
Tue May 27 16:06:35 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.163.01             Driver Version: 550.163.01     CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA A100-SXM4-40GB          Off |   00000000:C1:00.0 Off |                    0 |
| N/A   29C    P0             55W /  400W |       1MiB /  40960MiB |      0%      Default |
|                                         |                        |             Disabled |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

#### Example input data

Example input data comprising of the PBMC[37] and BoneMarrow[43] expression data and list of Homo sapiens and Mus musculus TFs (from AnimalTFDB3.0[44]) can downloaded from our webserver at https://nextcloud.computecanada.ca/index.php/s/pXKQ2isr47AwKEX. Alternatively, one can run the below command in the terminal to download, extract, and copy the raw input data files in the `data/raw` project directory. Make sure to run this command in the GRouNdGAN project directory after having cloned the repository.

```
$ curl https://nextcloud.computecanada.ca/index.php/s/WqrCqkH5zjYYMw9/download –output demo_data.tar &&
tar -xvf demo_data.tar -C data/raw/ &&
mv data/raw/demo/* data/raw &&
rm demo_data.tar &&
rm -rf data/raw/demo/
```

▲ **CRITICAL** Docker and Singularity images come prepackaged with the said example input data files and this step should be skipped.

## Procedure

GRouNdGAN can be executed through a command-line interface (CLI) by running the `main.py` file in the `src/` directory via the Python interpreter. Running `main.py` with the `–help` (or the abbreviated `-h`) flag outputs a summary of the tool and a list of required and optional arguments. Commands are listed as optional arguments and the path to the configuration file, which needs to be inputted alongside each command, is a required argument. Each of the seven available commands is a step in the procedure: `–preprocess`, `–create_grn`, `–train`,

–generate, –evaluate, –benchmark_grn, and –perturb. Below is the output of the –help command

```
$ python src/main.py –help
usage: GRouNdGAN [-h] –config CONFIG [–preprocess] [–create_grn] [–train] [–generate]
[–evaluate] [–benchmark_grn] [–perturb]

GRouNdGAN is a gene regulatory network (GRN)-guided causal implicit generative model
for simulating single-cell RNA-seq data, in-silico perturbation experiments, and
benchmarking GRN inference methods. This program also contains cWGAN and unofficial
implementations of scGAN and cscGAN (with projection conditioning)

required arguments:
–config CONFIG Path to the configuration file

optional arguments:
–preprocess Preprocess raw data for GAN training
–create_grn Infer a GRN from preprocessed data using GRNBoost2 and appropriately
format as causal graph
–train Start or resume model training
–generate Simulate single-cells RNA-seq data in-silico
–evaluate Evaluate the data quality of the simulated dataset
–benchmark_grn Evaluate the performance of a GRN inference method in inferring the
ground truth GRN
–perturb Perform a perturbation experiment using a trained GRouNdGAN model
```

It is possible to execute the whole pipeline in one go through a single command by chaining multiple steps of the pipeline, as shown below.

```
$ python src/main.py –config configs/causal_gan.cfg –preprocess –
create_grn –train –generate –evaluate
```

Alternatively, one can run each step separately. For example, the below command only runs the preprocessing step.

```
$ python src/main.py –config configs/causal_gan.cfg –preprocess
```

▲ **CRITICAL** If you decide to chain two or more commands, the supplied configuration file must include required arguments of all invoked commands. We discuss each step of the pipeline and relevant configuration file arguments in the following sections.

▲ **CRITICAL** Avoid chaining commands that are not meant to be run consecutively (e.g., running –preprocess with –generate since this jumps over the –create_GRN and –train commands).

▲ **CRITICAL** Substitute python with python3.9 if you are running GRouNdGAN in a Docker or Singularity container.

**Table 2:** Arguments relevant to the preprocessing command in the configuration file.

| Configuration File Section | Argument | Explanation |
|---|---|---|
| `[EXPERIMENT]/[Preprocessing]` | `10x`<br>`[type: boolean]` | "True" if you are supplying 10x-formatted data files (`matrix.mtx`, `genes.tsv`, and `barcodes.tsv`). "False" if you are providing annotated data (anndata) objects in the `.h5ad` format. |
| `[EXPERIMENT]/[Preprocessing]` | `Raw`<br>`[type: file path/directory]` | If 10x is set to "True", path to the directory containing 10x-formatted data files. Otherwise, path to the raw (unprocessed) `.h5ad` file. |
| `[EXPERIMENT]/[Preprocessing]` | `validation set size`<br>`[type: integer]` | Number of cells to include in the validation set. |
| `[EXPERIMENT]/[Preprocessing]` | `test set size`<br>`[type: integer]` | Number of cells to include in the test set. |
| `[EXPERIMENT]/[Preprocessing]` | `Annotations`<br>`[type: directory, Optional]` | Path to additional annotations (such as cell-type, clusters, etc. for cells/samples). |
| `[EXPERIMENT]/[Preprocessing]` | `min cells`<br>`[type: integer]` | Filter genes that are expressed in less than the specified "min cells" threshold of cells. |
| `[EXPERIMENT]/[Preprocessing]` | `min genes`<br>`[type: integer]` | Filter cells with less than the specified threshold of "min genes" expressed. |
| `[EXPERIMENT]/[Preprocessing]` | `library size`<br>`[type: integer]` | Library size used for library-size normalization. |
| `[EXPERIMENT]/[Preprocessing]` | `louvain res`<br>`[type: float]` | Louvain clustering resolution (higher resolution means finding more and smaller clusters). |
| `[EXPERIMENT]/[Preprocessing]` | `highly variable number`<br>`[type: integer]` | Number of highly variable genes to identify and keep for the rest of the pipeline. |
| `[EXPERIMENT]/[Data]` | `train`<br>`[type: file path]` | Path to a `.h5ad` file to output the training set. |
| `[EXPERIMENT]/[Data]` | `validation`<br>`[type: file path]` | Path to a `.h5ad` file to output the validation set. |
| `[EXPERIMENT]/[Data]` | `test`<br>`[type: file path]` | Path to a `.h5ad` file to output the test set. |
| `[EXPERIMENT]/[Data]` | `number of genes`<br>`[type: integer]` | Number of genes in the dataset. |

## Step 1. Preprocessing

## • TIMING <1 min

Before running the preprocessing command, ensure that the configuration file contains the required arguments to the Preprocessing and Data sections of the configuration file, as outlined in Table 2.

Once the configuration file is ready, run the program with the `–preprocess` flag, supplying the path to the configuration file with the `–config` flag.

```
$ python src/main.py –config configs/causal_gan.cfg –preprocess
```

♦ **TROUBLESHOOTING**

Once completed, you will be shown a success message and the preprocessed train, validation, and test sets will be saved as separate `.h5ad` files in the paths defined under the `[Data]` section of the configuration file.

*Example of the expected output:*

```
Successfully preprocessed and saved dataset.
Train set: data/processed/PBMC/PBMC68k_train.h5ad
Validation set: data/processed/PBMC/PBMC68k_validation.h5ad
Test set: data/processed/PBMC/PBMC68k_test.h5ad
```

▲ **CRITICAL** Do not replace GRouNdGAN's preprocessing with a custom preprocessing step. GRouNdGAN specifically requires library-size normalized count data.

## Step 2. GRN Creation

### A) Imposing a GRN Inferred from Data

**• TIMING ~8 min for a dataset of around 67000 cells with 4 CPU threads**

GRouNdGAN by default uses GRNBoost2[5] to infer a GRN on the preprocessed training set. It then uses the inferred GRN as a scaffold to subsample edges in order to create a causal bipartite directed acyclic graph (DAG). This DAG is then converted and saved as a pickle file in a custom format (see Step 2 - Imposing Custom GRNs) readable to the program. The `––create_grn` command requires the arguments listed in Table 3, in addition to those previously discussed for Step 1 - Preprocessing. GRouNdGAN provides two ways to subsample the GRN. The first uses `[EXPERIMENT]/[GRN Preparation]/strategy = top`, which includes the top k most important TFs of each gene. The second involves generating two separate GRNs, each containing half of these top k TFs per gene. By setting `[EXPERIMENT]/[GRN Preparation]/strategy = pos ctr` and then `[EXPERIMENT]/[GRN Preparation]/strategy = neg ctr`, the TFs are split based on their parity in the list of each gene's ranked list of important TFs. Since this alternative method produces two GRNs of identical densities and relatively similar importances, it is useful for comparative and controlled analysis.

After completing the relevant sections of the configuration file, execute the following command

**Table 3:** Arguments relevant to the GRN creation command in the configuration file.

| Configuration File Section | Argument | Explanation |
|---|---|---|
| `[EXPERIMENT]/[GRN Preparation]` | `TFs` `[type: file path]` | Path to the file containing TF names (accepts the AnimalTFDB `.csv` format). |
| `[EXPERIMENT]/[GRN Preparation]` | `k` `[type: integer]` | The number of most important TFs per gene to include in the GRN. |
| `[EXPERIMENT]/[GRN Preparation]` | `strategy` `[type: string]` | Defines the procedure used to subsample from the inferred GRN. The following options are available:<br>"top": keep the top most important "k" TFs for every gene.<br>"pos ctr:" keep the TFs with even parity in the list of top "k" TFs per gene (even indices 0, 2, 4... = top 1, 3, 5, ...).<br>"neg ctr": keep the TFs with odd parity in the list of top "k" TFs per gene (odd indices 1, 3, 5... = top 2, 4, 6,...). |
| `[EXPERIMENT]/[GRN Preparation]` | `Inferred GRN` `[type: file path]` | File path to store the GRN inferred by GRNBoost2 (`.csv` file). |
| `[EXPERIMENT]/[Data]` | `causal graph` `[type: file path]` | File path to store the GRouNdGAN subsampled GRN (`.pkl` file). |

to run this step:

```
$ python src/main.py –config configs/causal_gan.cfg –create_grn
```

## ♦ TROUBLESHOOTING

Once finished, you will be shown an output detailing the properties of the subsampled GRN and the causal graph will be written in the location specified by `[EXPERIMENT]/[Data]/causal graph` in the configuration file.

*Example of the expected output:*

```
Using 63 TFs for GRN inference.
preparing dask client
parsing input
creating dask graph
4 partitions
computing dask graph
shutting down client and local cluster
finished
Successfully saved GRN inferred by GRNBoost2 GRN to
data/processed/PBMC/inferred_grnboost2.csv

Causal Graph
_______ _____
TFs 63
Targets 937
Genes 1000
Possible Edges 59031
Imposed Edges 14055
GRN density Edges 0.238095
_______ _____
Successfully saved GRouNdGAN causal graph to
data/processed/PBMC/causal_graph.pkl
```

#### B) Imposing a Custom GRN

▲ **CRITICAL STEP** Instead of creating causal graphs through the described subsampling process of a GRNBoost2 inferred GRN, it is possible to impose custom GRNs for greater control over exact edges to include and exclude from the causal graph. Nonetheless, we still advise users to impose causal edges that are biologically meaningful to the reference dataset (through evidence found from literature, computational methods, ChiP-seq datasets, etc.).

▲ **CRITICAL** Since GRouNdGAN's two-fold objective is to impose a causal structure while generating realistic data points, drastic inconsistencies between the reference data and the imposed causal graph can deteriorate the quality of simulated cells.

GRouNdGAN reads the causal graph from a pickle file storing a python dictionary. In this dictionary, each key corresponds to a gene index, and each value is a Python set containing the indices of its regulating TFs.

Before saving the dictionary, gene and TF names must be converted to their corresponding indices based on their order in the input dataset (`.var_names` attribute in the Anndata file). For example, a dictionary structured with gene/TF names of the form

```
causal_graph = {
 "G2": {"TF2", "TFn"},
 "G1": {"TF1"},
 "Gn": {"TF2", "TF1"},
 "G3": {"TFn", "TF1"}
}
```

must first be converted into index-based format after resolving each name to its datasetspecific index below:

```
causal_graph = {
 1: {4, 5},
 3: {0},
 6: {4, 0},
 2: {5, 0}
}
```

Finally, the causal graph dictionary can be saved as a pickle file in python as shown below.

```
import pickle

with open("path/to/write/causal_graph.pkl", "wb") as fp:
pickle.dump(causal_graph, fp, protocol=pickle.HIGHEST_PROTOCOL)
```

▲ **CRITICAL** If you're imposing a custom GRN, skip the `–create_grn` command.

▲ **CRITICAL** The GRN must be a directed bipartite graph. It cannot include cycles.

▲ **CRITICAL** The python dictionary must include all TFs and genes present in the preprocessed dataset. Every target gene must appear as a key in the dictionary, and every TF must appear at least once in the list of values associated with those keys.

▲ **CRITICAL** Make sure the `[EXPERIMENT]/[Data]/causal graph]` argument in the configuration file points to the newly-created pickle file.

### Step 3. Model Training

#### • TIMING ~75 h on a single NVidia A100SXM4 GPU

When running training command, three folders will be created in the output directory specified by the configuration file (`[EXPERIMENT]/output directory`). The configuration file will also be copied over into this directory.

**Table 4:** Arguments relevant to the training command in the configuration file.

| Configuration File Section | Argument | Explanation |
|---|---|---|
| `[EXPERIMENT]/[Training]/[Logging]` | `summary frequency [type: integer]` | Specifies the frequency at which TensorBoard logs are generated for the GRouNdGAN model. |
| `[EXPERIMENT]/[Training]/[Logging]` | `plot frequency [type: integer]` | Specifies the frequency at which t-SNE plots are generated for the GRouNdGAN model. |
| `[EXPERIMENT]/[Training]/[Logging]` | `save frequency [type: integer]` | Specifies the frequency at which checkpoints are saved for the GRouNdGAN model. |
| `[EXPERIMENT]/[CC Training]/[Logging]` | `summary frequency [type: integer]` | Specifies the frequency at which TensorBoard logs are generated for the causal controller. |
| `[EXPERIMENT]/[CC Training]/[Logging]` | `plot frequency [type: integer]` | Specifies the frequency at which t-SNE plots are generated for the causal controller. |
| `[EXPERIMENT]/[CC Training]/[Logging]` | `save frequency [type: integer]` | Specifies the frequency at which checkpoints are saved for the causal controller. |

- `checkpoints/`: contains the `.pth` state dictionaries, including model's weights, biases, and other parameters.
- `TensorBoard/`: contains stored TensorBoard logs (.tfevent files) for visualizing the model's training progress and neural graph.
- `TSNE/`: contains t-SNE plots of reference and simulated data.

The frequency at which these directories are updated can be configured through parameters specified in the configuration file, as detailed in Table 4.

To begin training, use the `–train` flag and along with the path to the configuration file as shown below.

```
$ python src/main.py –config configs/causal_gan.cfg –train
```

## ♦ TROUBLESHOOTING

*Example of the expected output:*

```
Done training causal controller step 0
Done training causal controller step 1
Done training causal controller step 2
...
Done training causal controller step 10000
Saved logs
Saved t-SNE plot
Done training causal controller step 10001
...
Done training causal controller step 200000
Saved logs
Saved t-SNE plot
Saved checkpoint
Done training GRouNdGAN step 0
Done training GRouNdGAN step 1
Done training GRouNdGAN step 2
...
Done training GRouNdGAN step 10000
Saved logs
Saved t-SNE plot
Done training GRouNdGAN step 10001
...
Done training GRouNdGAN step 100000
Saved logs
Saved t-SNE plot
Saved checkpoint
...
Done training GRouNdGAN step 1000000
Saved logs
Saved t-SNE plot
Saved checkpoint
Finished training
```

▲ **CRITICAL** While GRouNdGAN supports muti-GPU training, we recommend using a single GPU to minimize overhead. By default, GRouNdGAN trains for 1000000 steps. Changing this setting in the configuration file is not recommended.

▲ **CRITICAL** GRouNdGAN also provides two slurm scripts for training and monitoring on HPC clusters in the `scripts/` directory of its code repository.

□ **PAUSE POINT** Users can stop and resume training from a checkpoint by setting `[EXPERIMENT]/checkpoint` in the configuration file to the latest saved checkpoint (`.pth` file in `[EXPERIMENT]/output directory/checkpoints/`) and rerunning the `–train` command.

***Training Monitoring using TensorBoard***

TensorBoard is a visualization tool that GRouNdGAN uses to provide its users with an intuitive monitoring and visualization platform. It keeps track of several key scalar values such as generator and critic losses, learning rates, and gradient penalty (Figure 4-b). At regular intervals, it also logs t-SNE plots comparing cells from the validation set with an equal number of simulated cells (Figure 4-a). Furthermore, TensorBoard provides a detailed view of GRouNdGAN's architecture by rendering the computational graphs of the Wasserstein critic and generator (comprised of causal controller and target generators) at varying levels of complexity as shown in Figure 4-c,d.

GRouNdGAN saves TensorBoard event files in `[EXPERIMENT]/output directory_CC/Tensorboard` for the causal controller training and `[EXPERIMENT]/output directory/Tensorboard` for the second training step.

Run the following command to start TensorBoard.

```
tensorboard –logdir="Tensorboard Event Directory " –host 0.0.0.0
–load_fast false
```

▲ **CRITICAL** Users can replace the `Tensorboard Event Directory` in the above command with a parent directory to compare different training runs of the model.

Once Tensorboard is running, it will output a local URL (by default: `http://localhost:6006/`) that can be opened in a web browser to access the TensorBoard dashboard. In the case you are running GRouNdGAN on a remote server, you might need to set up port forwarding to access TensorBoard on your local machine. The command below forwards the remote machine's port 6006 (where TensorBoard is running) to port 6007 on your local machine.

```
ssh -N -f -L localhost:6007:computenode:6006
userid@remote_machine_ip
```

▲ **CRITICAL** The above command might need to be modified depending on your remote server setup, network configuration, and port availability. Ensure that the port on which TensorBoard is running is not blocked by a firewall on the remote server. Some servers might be configured to restrict port forwarding.

### Step 4. Simulation

- **TIMING <30 s**

Once training is finished, set the `[EXPERIMENT]/checkpoint` path in the configuration file to the desired model checkpoint (`.pth` file in `[EXPERIMENT]/output directory/checkpoints/`).

**Figure 4: TensorBoard visualizations during GRouNdGAN training. a** The "images" tab displays a t-SNE plot of reference and generated cells. **b** The "scalars tab" displays key training metrics over time, including gradient penalty, learning rate, and losses for both the critic and generator. **c** and **d** The "graphs" tab visualise the computational graph for the critic and generator networks of GRouNdGAN, showing the model architecture and the flow of gradients during backpropagation.

In most cases, you would want to use the latest checkpoint corresponding to the greatest number of steps. Furthermore, you can control the generation path and number of cells to generate through the configuration file (refer to Table 5).

Running the following command will save the generated cells as a `.h5ad` file (`simulated.h5ad`).

**Table 5:** Arguments relevant to the generation command in the configuration file.

| Configuration File Section | Argument | Explanation |
|---|---|---|
| `[EXPERIMENT]/checkpoint` | `model checkpoint [type: file path]` | Path to the checkpoint from which to load the trained model. |
| `[EXPERIMENT]/[generation]` | `number of cells to generate [type: integer]` | Number of cells to simulate. |
| `[EXPERIMENT]/[generation]` | `generation path [type: directory, Optional]` | Path where to save the simulated dataset (`simulated.h5ad`). If left undefined, `simulated.h5ad` will be saved to `[Experiment]/output directory/`. |

```
$ python src/main.py –config configs/causal_gan.cfg ––generate
```

*Example of the expected output:*

```
Simulated cells saved to results/GRouNdGAN/simulated.h5ad
```

□ **PAUSE POINT** Remaining steps are optional. Users can stop here and use the saved simulated dataset for benchmarking, data augmentation, and other external analyses.

## Step 5. Assessing simulated data quality (Optional)

## • TIMING <30 s

This step provides a simple command for assessing the quality of simulated scRNA-seq data compared to its reference experimental dataset using the following qualitative and quantitative metrics:

- t-SNE plot visualization of jointly embedded experimental and simulated cells. Greater overlap indicates better alignment.
- Euclidean distance between the mean expression profiles of real and simulated cells. A lower value indicates higher similarity.
- Cosine distance between the mean expression profiles of real and simulated cells. A lower value indicates higher directional similarity and better alignment.
- AUROC of a random forest classifier trained to distinguish simulated cells from experimental cells. An AUROC closer to 0.5 indicates that the classifier fails to tell the two classes of cells apart, implying more realistic simulation.

**Table 6:** Arguments relevant to the evaluation command in the configuration file.

| Configuration File Section | Argument | Explanation |
|---|---|---|
| `[EXPERIMENT]/[Evaluation]` | `simulated data path` `[type: directory, Optional]` | Path to the simulated data you wish to assess. Defaults to `[EXPERIMENT]/[Generation]/generation path` if left undefined. |
| `[EXPERIMENT]/[Evaluation]` | `plot tsne` `[type: boolean]` | Whether to plot the t-SNE plot. |
| `[EXPERIMENT]/[Evaluation]` | `compute euclidean distance` `[type: boolean]` | Whether to compute the Euclidean distance metric. |
| `[EXPERIMENT]/[Evaluation]` | `compute cosine distance` `[type: boolean]` | Whether to compute the Cosine distance metric. |
| `[EXPERIMENT]/[Evaluation]` | `compute rf auroc` `[type: boolean]` | Whether to compute the RF AUROC metric and plot the ROC curve. |
| `[EXPERIMENT]/[Evaluation]` | `compute MMD` `[type: boolean]` | Whether to compute the MMD metric. |
| `[EXPERIMENT]/[Evaluation]` | `compute miLISI` `[type: boolean]` | Whether to compute the miLISI metric. |

- Maximum Mean Discrepancy (MMD) to quantify distributional similarity of experimental and simulated cells. A lower value is desired.
- Mean integration local inverse Simpson's index (miLISI) to measure local mixing of real and simulated cells in the t-SNE space. A value closer to 2 indicates good integration and mixing.

All metrics are computed by comparing the test set to an equal number of cells sampled from the simulated data. As a control, we also compute the Euclidean distance, Cosine distance, MMD, and miLISI between two halves of the reference test set to establish a baseline for comparison. You can specify which evaluation metrics to compute by setting the corresponding flags in the configuration file (see Table 6). Additionally, you can use GRouNdGAN's evaluation framework to assess simulated datasets generated by external simulators by updating `[EXPERIMENT]/ [Evaluation]/simulated data path` in the configuration file to point to the desired dataset.

To run the evaluation pipeline with the specified configuration, execute the following command.

```
$ python src/main.py –config configs/causal_gan.cfg –evaluate
```

Output plots (following the format of Figure 5-a,b) will be saved to [EXPERIMENT]/output directory/

*Example of the expected output:*

```
t-SNE plot saved to results/GRouNdGAN/tSNE.png

Euclidean distance (real vs fake): 171.218017578125
Euclidean distance (control): 242.84738159179688

Cosine distance (real vs fake): 0.0003756880760192871
Cosine distance (control): 0.000784814357757568 4

RF ROC plot saved to results/GRouNdGAN/RF.png
RF AUROC: 0.5211342726343393

MMD (real vs fake): 0.03008744777663175
MMD (control): 0.06244054076970418

miLISI (real vs fake): 1.89692891595357
miLISI (control): 1.8900128524031679
```

### Step 6. GRN inference benchmarking (Optional)

### • TIMING <20 s

One of GRouNdGAN's main goals is to provide tools and ground truth for benchmarking GRN inference methods. This command evaluates how faithful GRNs inferred on GRouNdGAN-generated data are to the one imposed onto the model and used for simulation. To run this command, you must provide the path to the inferred GRN through the configuration file using the `[EXPERIMENT]/[GRN Benchmarking]/grn to benchmark` field (refer to Table 7). The GRN should be a tab-separated `.csv` file with three columns in the following order:

1. TF
2. Gene
3. Importance

Headers are optional; only the column order matters.

*Example format:*

```
Gata1 Tbb-bs 0.89
Tal1 Tbb-bs 0.74
Gata2 Gfi1b 0.52
```

**Table 7:** Arguments relevant to the benchmarking command in the configuration file.

| Configuration File Section | Argument | Explanation |
|---|---|---|
| `[EXPERIMENT]/[GRN Benchmarking]` | `grn to benchmark` `[type: file path]` | Path to the inferred GRN you wish to benchmark. |
| `[EXPERIMENT]/[GRN Benchmarking]` | `ground truth save path` `[type: directory, Optional]` | If specified, the ground truth GRN (GRouNdGAN's causal graph) will be exported to the provided path. |
| `[EXPERIMENT]/[GRN Benchmarking]` | `plots save path` `[type: path]` | Folder in which to save generated plots. |
| `[EXPERIMENT]/[GRN Benchmarking]` | `compute precision at k` `[type: boolean]` | Whether to compute precision at k. |
| `[EXPERIMENT]/[GRN Benchmarking]` | `k` `[type: integer]` | How many of the top k regulators to consider when computing precision at k. |
| `[EXPERIMENT]/[GRN Benchmarking]` | `compute pr` `[type: boolean]` | Whether to plot the PR curve and compute AUPRC of GRN recovery. |

When enabled in the configuration file, the GRN benchmarking command generates one or both of the following plots, which are saved to the directory specified in `[EXPERIMENT]/[GRN Benchmarking]/plots save path`:

- Precision at k: Evaluates the proportion of the top-k inferred TFs for each gene are true regulators. The value of k is configurable.
- Precision-Recall (PR) curve: Computed across all predicted edges in the inferred GRN.

Run this step using the command below.

```
$ python src/main.py –config configs/causal_gan.cfg –benchmark_grn
```

Both plots (see Figure 5-c, d) are saved as `.png` files, and relevant evaluation metrics such as precision at k scores and the area under the precision recall curve (AUPRC) are printed to the console.

*Example of the expected output:*

**Table 8:** Arguments relevant to the perturbation command in the configuration file.

| Configuration File Section | Argument | Explanation |
|---|---|---|
| `[EXPERIMENT]/[Perturbation]` | `save dir` `[type: directory]` | Directory in which to save cells before and after perturbation, as well as generated plot. |
| `[EXPERIMENT]/[Perturbation]` | `tfs to perturb` `[type: list[string]]` | List of space-separated TF names to perturb. Example: `IRF8 CEBPA`. |
| `[EXPERIMENT]/[Perturbation]` | `perturbation values` `[type: list[float]]` | List of space-separated float values. |

```
Saved ground truth GRN to data/generated/ground truth GRN.csv

Inferred GRN precision at k: [1.0, 0.9983425414364641,
0.9918968692449356, 0.9820441988950276, 0.9703867403314917]
Baseline precision at k: [0.010526315789473684, 0.021052631578947368,
0.031578947368421054, 0.042105263157894736, 0.05263157894736842]
Saved precision at k plot at results/GRouNdGAN/Top_TF_precision.png

Inferred GRN AURPC: 0.71
Baseline AUPRC (random predictor): 0.15789473684210525
Saved PR curve at results/GRouNdGAN/PR_curve.png
```

□ **PAUSE POINT** The ground truth GRN can be exported as a `.csv` file to be used for additional analysis or use in external benchmarking tools. Simply specify `[EXPERIMENT]/[GRN Benchmarking]/ground truth save path` in the configuration file before running the command.

## Step 7. Perturbation prediction (Optional)

## • TIMING <1 min

Once trained, GRouNdGAN can sample from interventional distributions and perform in silico perturbation experiments at inference time. It enables the creation of matched case/control comparison as perturbations are applied to the same batch of cells (batch size of 1024 by default). To simulate a perturbation, provide a list of TFs to perturb using `[EXPERIMENT]/[Perturbation]/tfs to perturb`, along with the corresponding target values in `[EXPERIMENT]/[Perturbation]/perturbation values`. Each TF is matched to a value by its position in the list, with entries separated by spaces (as shown in Table 8).

▲ **CRITICAL** Ensure all TFs specified in `[EXPERIMENT]/[Perturbation]/tfs to perturb` exist in the causal graph and that their count matches the number of entries in `[EXPERIMENT]/[Perturbation]/perturbation`.

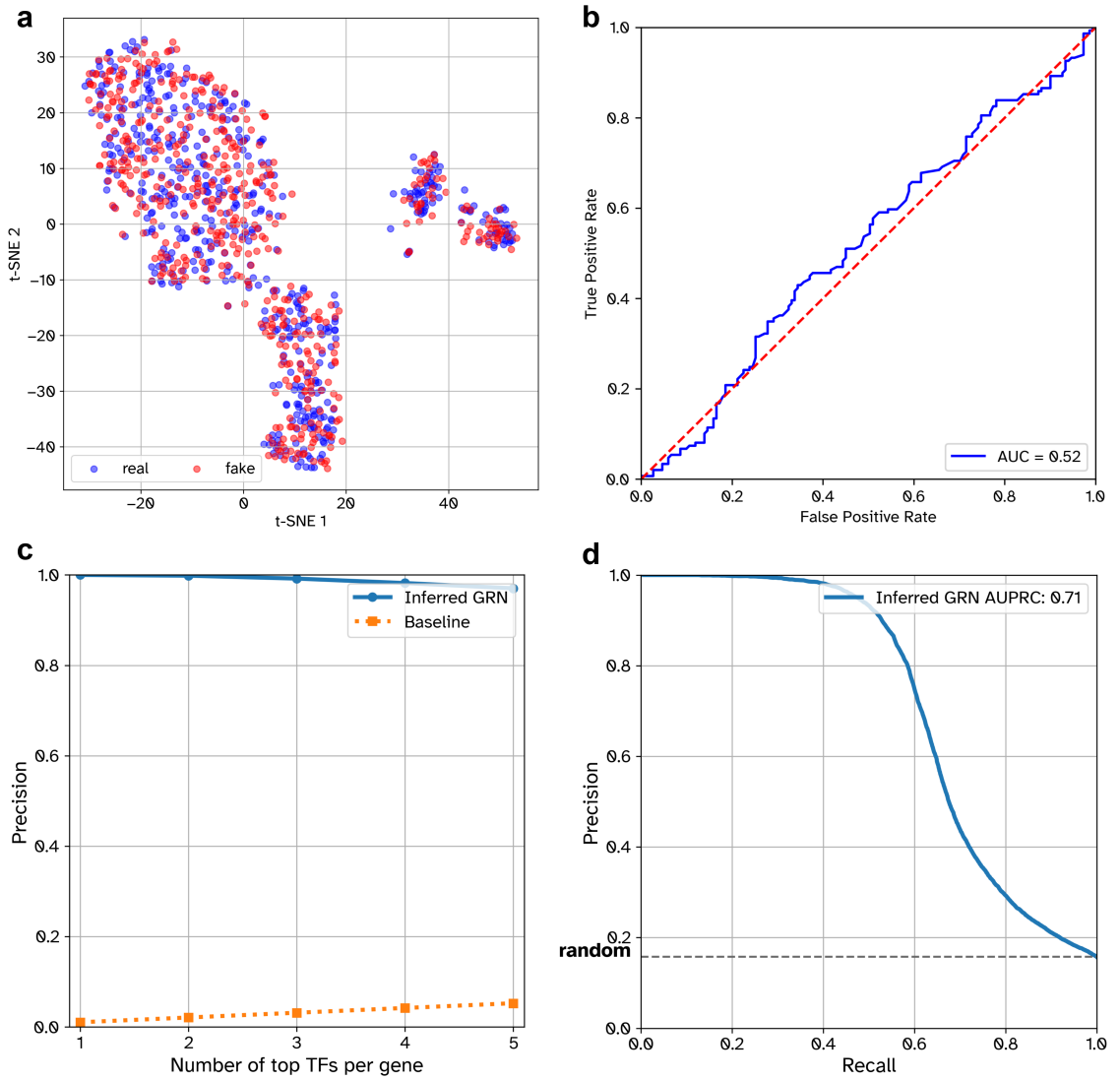



**Figure 5: Example of expected plot outputs saved by the evaluation and benchmarking commands.** **a** t-SNE plot of simulated and test set experimental cells. **b** Receiver Operating Characteristic (ROC) curve of a random forest classifier distinguishing between simulated cells and test set experimental cells. **c** Precision at k plot showing the proportion of true regulators among the top-k predicted transcription factors for each gene. **d** Precision-Recall (PR) curve illustrating global GRN reconstruction performance. The baseline corresponds to the performance of a random predictor which also corresponds to the density of the ground truth GRN.

Once the experiment parameters are defined, you can run the perturbation experiment with the following command.

```
$ python src/main.py -config configs/causal_gan.cfg -perturb
```

## ♦ TROUBLESHOOTING

The cell states before and after perturbation will be saved as `before_perturbation.h5ad` and `after_perturbation.h5ad`, respectively, in the specified output directory. There is a

one-to-one correspondence between cells in the two simulated datasets (row and column indices match). These files can be used for downstream or secondary analysis. In addition to the data files, a series of plots, similar to Figure 6, will be generated and saved to visualize the effects of the perturbation.

*Example of the expected output:*

```
Loaded GAN
Using checkpoint at results/GRouNdGAN/checkpoints/step_1000000.pth
Saved cells before and after perturbation to data/generated/
```

## Timing

The reported execution times correspond to running GRouNdGAN on the PBMC dataset, which comprises 67579 cells, using the hardware configuration detailed in the Equipment section. The duration of step 1 (Preprocessing) and step 2 (GRN creation) primarily scale with the dataset size. In contrast, the execution of step 3 (Model training) is largely dependent on the density of the imposed GRN and GPU specification. The remaining steps exhibit comparatively lower sensitivity to these factors. Installation times are predominantly affected by download speed and server loads.

Software installation - local: ~12 min
Software installation - running through the Docker image: ~6 min
Software installation - building Docker image from Dockerfile: ~13 min
Software installation - using Singularity: ~7 min

Procedure - Step 1, Preprocessing: <1 min
Procedure - Step 2, GRN creation: ~8 min
Procedure - Step 3, Model training: ~75 h
Procedure - Step 4, Simulation: <30 s
Procedure - Step 5, Data quality evaluation: <30 s
Procedure - Step 6, GRN inference benchmarking: <20 s
Procedure - Step 7: Perturbation prediction: <1 min
Procedure - End to end (Steps 1-7): ~75 h, 12 min

### General notes and troubleshooting

Table 9 contains common issues and how to troubleshoot them.

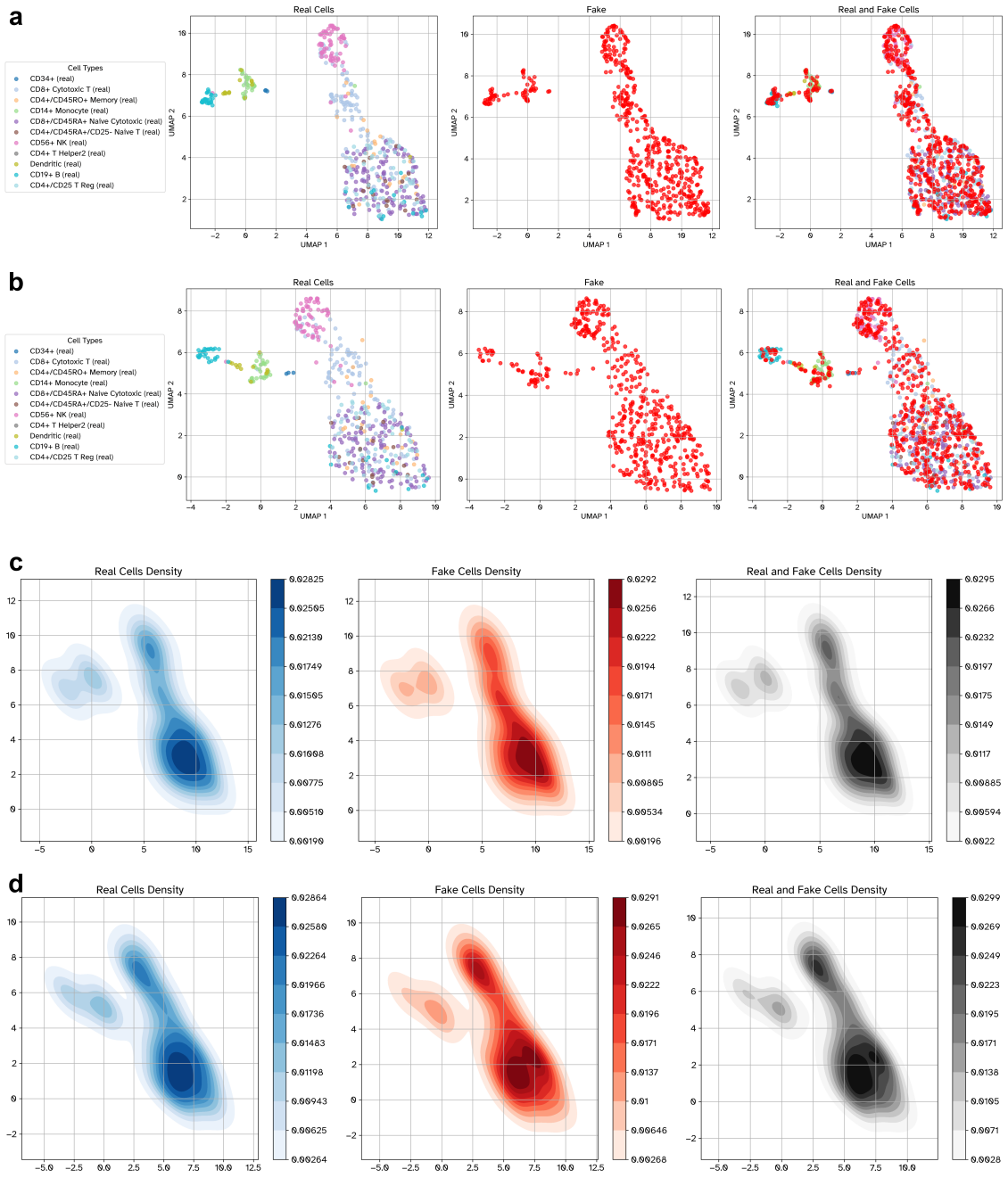



**Figure 6: Example of expected plot outputs saved by the perturbation command.** **a** UMAP embedding of experimental cells (left), unperturbed simulated cells (middle), and overlayed on top of each other (right). **b** similar to a but simulated cells were generated under perturbation. **c** Cell density in the UMAP embedding for experimental cells (left), unperturbed simulated cells (middle), and together (right). **d** similar to **c** but simulated cells are perturbed.

## Expected results

Running GRouNdGAN's GRN-guided simulation procedure (Steps 1 to 4) produces a single final output: a `simulated.h5ad` file containing simulated scRNA-seq data. The file preserves the original gene annotations from the input data and assigns sequential labels to generated

**Table 9:** Troubleshooting table.

| Step | Problem | Possible reason | Solution |
|---|---|---|---|
| 1, 2 | Error message: `check_values_indices_shape_match` | Mismatch between the number of genes in `train`, `validation`, or `test` sets and configuration parameters or `GRN`. | Verify `[EXPERIMENT]/[Preprocessing]/highly variable number` and `[EXPERIMENT]/[Data]/number of genes` parameters in the configuration file against the input dataset. |
| | | The `.X` attribute in the `anndata` input is a sparse expression matrix. | If the input expression matrix is sparse, convert it into a dense matrix using `adata.X.toarray()`. |
| 3 | Error message: `torch.cuda.OutOfMemoryError: CUDA out of memory` | Insufficient video memory on `GPU`. | Switching to a `GPU` with a larger `VRAM`. |
| | | | Making the input `GRN` sparser. If the `–create_grn` command is used to construct the input `GRN`, making the hyperparameter `[GRN Preparation]/k` in the configuration file smaller will result in a sparser `GRN`. Reducing `GRN` size might result in a lower quality simulation. |
| | | | Reducing the `[EXPERIMENT]/[Model]/depth per gene` and `[EXPERIMENT]/[Model]/width per gene]` hyperparameters in the configuration file to 2 and 1, respectively. This will make `GRouNdGAN's` neural network smaller to fit in a smaller `VRAM`. This option might result in a lower quality simulation. |
| 3 | Error message: `what(): mat1 and mat2 shapes cannot be multiplied` | The specified number of genes is inconsistent with the `GRN`. | Ensure that the number of nodes in the `GRN` correspond to the `[EXPERIMENT]/[Data]/number of genes` parameter in the configuration file. |
| 7 | Error message: `ValueError: 'TF' is not in list` | The `TF` targeted for perturbation does not exist in the `GRN`. | Ensure that all the `TFs` listed in the `[EXPERIMENT]/[Perturbation]/tfs to perturb` parameter of the configuration file exist in the dataset and are regulators (not targets) in the `GRN`. |

cells (e.g., Cell1, Cell2, ...). The simulated expression matrix is stored in the `.X` attribute, where rows correspond to cells and columns to genes. Furthermore, `.var` and `.obs` store gene and cell metadata, respectively. This structured output format ensures compatibility with downstream python and R single-cell analysis workflows. A collection of such output datasets paired with the GRN used for simulation can be found on GRouNdGAN's website at https://emad-combinelab.github.io/GRouNdGAN/benchmarking.html.

The perturbation prediction procedure generates two .h5ad files: `before_perturbation.h5ad` and `after_perturbation.h5ad`. They contain expression matrices (in the `.X` attribute) of identical dimensions, representing cellular states before and after perturbation, respectively. Both include the same set of genes and annotations. Additionally, cells are matched one-to-one between the two files, meaning that each cell (e.g., "Cell 1") in `before_perturbation.h5ad` corresponds directly to its perturbed counterpart in `after_perturbation.h5ad`.

Given a reasonably sized training set and a sufficiently representative GRN, we expect the simulated output data to be realistic (i.e., mimicking experimental training data) and true to the imposed GRN. Under the same conditions, we also expect that the perturbation results will align with biological expectations, though perturbation prediction relies more heavily on the

quality of the input GRN. We provide guidelines for evaluating the quality of the output data using five metrics (see the Procedure – Step 5) and comparing it to a control baseline, where the control is derived from the data itself. Furthermore, it is expected that the generator and critic losses, which can be monitored using TensorBoard (see Procedure – Step3), will decrease as the model undergoes training, showing better performance over time.

## Supplementary Information

The online version contains supplementary material available in Supplementary File S1.

## Authors' contributions

All authors contributed to the design of the protocol, data analyses, and writing the manuscript. A.E. supervised the study. Y.Z. implemented the protocol, conducted computational analyses, developed the website, and managed releases across various formats.

## Funding

This work was supported by a Discovery grant from Natural Sciences and Engineering Research Council of Canada (NSERC) [RGPIN-2019-04460] (AE), a NOVA grant funded by NSERC and Fonds de recherche du Québec – Nature et technologies (FRQNT) [2024-NOVA-344140], and by a grant from Canada Foundation for Innovation (CFI) JELF [project 40781] (AE). This work was enabled in part by support provided by Calcul Québec (www.calculquebec.ca) and the Digital Research Alliance of Canada (alliancecan.ca)

## Data Availability

The dataset used to demonstrate the protocol is publicly available. The PBMC-ALL scRNA-seq dataset (healthy donor A) can be downloaded through the 10x Genomics website at https://support.10xgenomics.com/single-cell-geneexpression/datasets/1.1.0/fresh_68k_pbmc_donor_a?. The GRouNdGAN-simulated PBMC-CTL dataset (100k cells by 1000 genes ,including TFs), along with the imposed ground truth GRN, is available on GRouNdGAN's website at https://emad-combinelab.github.io/GRouNdGAN/benchmarking.html#pbmc-ctl.

## Code Availability

An implementation of GRouNdGAN is freely available under the terms of the GNU Affero General Public License v3.0 on GitHub at https://github.com/Emad-COMBINE-lab/GRouNdGAN. The software is written in Python 3.9.6 using the PyTorch deep learning framework (v. 1.13.1). To ensure ease of use and reproducibility, a pre-build container image is also available on DockerHub (https://hub.docker.com/r/yazdanz/groundgan) and an archived version can be found on Zenodo (https://doi.org/10.5281/zenodo.11068245). Visit the GRouNdGAN website (https://emad-combine-lab.github.io/GRouNdGAN) to explore an expanding collection of readily available simulated datasets for benchmarking GRN inference algorithms, along with the latest updates on running the protocol.

## Competing interests

The authors have no conflict of interest to declare.

## References

1. Pardee AB: **Regulatory molecular biology.** *Cell Cycle* 2006, **5**(8):846-852.

2. Jacob F, Monod J: **Genetic regulatory mechanisms in the synthesis of proteins.** *Journal of molecular biology* 1961, **3**(3):318-356.

3. Afonja T, Sheth I, Binkyte R, Hanif W, Ulas T, Becker M, Fritz M: **LLM4GRN: Discovering Causal Gene Regulatory Networks with LLMs–Evaluation through Synthetic Data Generation.** *arXiv preprint arXiv:241015828* 2024.

4. Emad A, Sinha S: **Inference of phenotype-relevant transcriptional regulatory networks elucidates cancer type-specific regulatory mechanisms in a pan-cancer study.** *NPJ systems biology and applications* 2021, **7**(1):9.

5. Moerman T, Aibar Santos S, Bravo González-Blas C, Simm J, Moreau Y, Aerts J, Aerts S: **GRNBoost2 and Arboreto: efficient and scalable inference of gene regulatory networks.** *Bioinformatics* 2019, **35**(12):2159-2161.

6. Chan TE, Stumpf MP, Babtie AC: **Gene regulatory network inference from single-cell data using multivariate information measures.** *Cell systems* 2017, **5**(3):251-267. e253.

7. Kim S: **ppcor: an R package for a fast calculation to semi-partial correlation coefficients.** *Communications for statistical applications and methods* 2015, **22**(6):665.

8. Su C, Pastor WA, Emad A: **Deciphering lineage-relevant gene regulatory networks during endoderm formation by InPheRNo-ChIP.** *Briefings in Bioinformatics* 2024, **25**(6):bbae592.

9. Kamimoto K, Stringa B, Hoffmann CM, Jindal K, Solnica-Krezel L, Morris SA: **Dissecting cell identity via network inference and in silico gene perturbation.** *Nature* 2023, **614**(7949):742-751.

10. Bravo González-Blas C, De Winter S, Hulselmans G, Hecker N, Matetovici I, Christiaens V, Poovathingal S, Wouters J, Aibar S, Aerts S: **SCENIC+: single-cell multiomic inference of enhancers and gene regulatory networks.** *Nature Methods* 2023:1-13.

11. Emmert-Streib F, Dehmer M, Haibe-Kains B: **Gene regulatory networks and their applications: understanding biological and medical problems in terms of networks.** *Frontiers in cell and developmental biology* 2014, **2**:38.

12. Pratapa A, Jalihal AP, Law JN, Bharadwaj A, Murali T: **Benchmarking algorithms for gene regulatory network inference from single-cell transcriptomic data.** *Nature methods* 2020, **17**(2):147-154.

13. McCall MN: **Estimation of gene regulatory networks.** *Postdoc journal: a journal of postdoctoral research and postdoctoral affairs* 2013, **1**(1):60.

14. Erbe R, Gore J, Gemmill K, Gaykalova DA, Fertig EJ: **The use of machine learning to discover regulatory networks controlling biological systems.** *Molecular cell* 2022, **82**(2):260-273.

15. Dibaeinia P, Sinha S: **SERGIO: a single-cell expression simulator guided by gene regulatory networks.** *Cell systems* 2020, **11**(3):252-271. e211.

16. Li H, Zhang Z, Squires M, Chen X, Zhang X: **scMultiSim: simulation of multimodality single cell data guided by cell-cell interactions and gene regulatory networks.** *Research Square* 2023:rs. 3. rs-2675530.

17. Schaffter T, Marbach D, Floreano D: **GeneNetWeaver: in silico benchmark generation and performance profiling of network inference methods.** *Bioinformatics* 2011, **27**(16):2263-2270.

18. Zinati Y, Takiddeen A, Emad A: **GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks.** *Nature Communications* 2024, **15**(1):4055.

19. Marouf M, Machart P, Bansal V, Kilian C, Magruder DS, Krebs CF, Bonn S: **Realistic in silico generation and augmentation of single-cell RNA-seq data using generative adversarial networks.** *Nature communications* 2020, **11**(1):166.

20. Kocaoglu M, Snyder C, Dimakis AG, Vishwanath S: **Causalgan: Learning causal implicit generative models with adversarial training.** *arXiv preprint arXiv:170902023* 2017.

21. Arjovsky M, Chintala S, Bottou L: **Wasserstein generative adversarial networks.** In: *International conference on machine learning: 2017.* PMLR: 214-223.

22. Gulrajani I, Ahmed F, Arjovsky M, Dumoulin V, Courville AC: **Improved training of wasserstein gans.** *Advances in neural information processing systems* 2017, **30**.

23. Zandigohar M, Rehman J, Dai Y: **scRegulate: Single-Cell Regulatory-Embedded Variational Inference of Transcription Factor Activity from Gene Expression.** *bioRxiv* 2025:2025.2004.2017.649372.

24. Marouf M, Machart P, Bansal V, Kilian C, Magruder DS, Krebs CF, Bonn S: **Realistic in silico generation and augmentation of single-cell RNA-seq data using generative adversarial networks.** *Nature communications* 2020, **11**(1):1-12.

25. Xu Y, Zhang Z, You L, Liu J, Fan Z, Zhou X: **scIGANs: single-cell RNA-seq imputation using generative adversarial networks.** *Nucleic acids research* 2020, **48**(15):e85-e85.

26. Baruzzo G, Patuzzi I, Di Camillo B: **SPARSim single cell: a count data simulator for scRNA-seq data.** *Bioinformatics* 2020, **36**(5):1468-1475.

27. Song D, Wang Q, Yan G, Liu T, Sun T, Li JJ: **scDesign3 generates realistic in silico data for multimodal single-cell and spatial omics.** *Nature Biotechnology* 2024, **42**(2):247-252.

28. Zappia L, Phipson B, Oshlack A: **Splatter: simulation of single-cell RNA sequencing data.** *Genome biology* 2017, **18**(1):174.

29. Zhang X, Xu C, Yosef N: **Simulating multiple faceted variability in single cell RNA sequencing.** *Nature communications* 2019, **10**(1):2611.

30. Marbach D, Prill RJ, Schaffter T, Mattiussi C, Floreano D, Stolovitzky G: **Revealing strengths and weaknesses of methods for gene network inference.** *Proceedings of the national academy of sciences* 2010, **107**(14):6286-6291.

31. Marbach D, Costello JC, Küffner R, Vega NM, Prill RJ, Camacho DM, Allison KR, Kellis M, Collins JJ: **Wisdom of crowds for robust gene network inference.** *Nature methods* 2012, **9**(8):796-804.

32. Kanehisa M, Goto S: **KEGG: kyoto encyclopedia of genes and genomes.** *Nucleic acids research* 2000, **28**(1):27-30.

33. Brown KR, Jurisica I: **Unequal evolutionary conservation of human protein interactions in interologous networks.** *Genome biology* 2007, **8**:1-11.

34. Kotlyar M, Pastrello C, Sheahan N, Jurisica I: **Integrated interactions database: tissue-specific view of the human and model organism interactomes.** *Nucleic acids research* 2016, **44**(D1):D536-D541.

35. Xu H, Baroukh C, Dannenfelser R, Chen EY, Tan CM, Kou Y, Kim YE, Lemischka IR, Ma'ayan A: **ESCAPE: database for integrating high-content published data collected from human and mouse embryonic stem cells.** *Database* 2013, **2013**:bat045.

36. Kolmykov S, Yevshin I, Kulyashov M, Sharipov R, Kondrakhin Y, Makeev VJ, Kulakovskiy IV, Kel A, Kolpakov F: **GTRD: an integrated view of transcription regulation.** *Nucleic acids research* 2021, **49**(D1):D104-D111.

37. Zheng GX, Terry JM, Belgrader P, Ryvkin P, Bent ZW, Wilson R, Ziraldo SB, Wheeler TD, McDermott GP, Zhu J: **Massively parallel digital transcriptional profiling of single cells.** *Nature communications* 2017, **8**(1):1-12.

38. Korsunsky I, Millard N, Fan J, Slowikowski K, Zhang F, Wei K, Baglaenko Y, Brenner M, Loh P-r, Raychaudhuri S: **Fast, sensitive and accurate integration of single-cell data with Harmony.** *Nature methods* 2019, **16**(12):1289-1296.

39. Wolf FA, Angerer P, Theis FJ: **SCANPY: large-scale single-cell gene expression data analysis.** *Genome biology* 2018, **19**:1-5.

40. Satija R, Farrell JA, Gennert D, Schier AF, Regev A: **Spatial reconstruction of single-cell gene expression data.** *Nature biotechnology* 2015, **33**(5):495-502.

41. Zinati Y, Takiddeen A, Emad A: **GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks.** *bioRxiv* 2023:2023.2007. 2025.550225.

42. Sun T, Song D, Li WV, Li JJ: **scDesign2: a transparent simulator that generates high-fidelity single-cell gene expression count data with gene correlations captured.** *Genome biology* 2021, **22**(1):1-37.

43. Paul F, Arkin Ya, Giladi A, Jaitin DA, Kenigsberg E, Keren-Shaul H, Winter D, Lara-Astiaso D, Gury M, Weiner A: **Transcriptional heterogeneity and lineage commitment in myeloid progenitors.** *Cell* 2015, **163**(7):1663-1677.

44. Hu H, Miao Y-R, Jia L-H, Yu Q-Y, Zhang Q, Guo A-Y: **AnimalTFDB 3.0: a comprehensive resource for annotation and prediction of animal transcription factors.** *Nucleic acids research* 2019, **47**(D1):D33-D38.

# Chapter 5

# Discussion

The studies which comprise this thesis leave its author with a several key insights into PPI inference methods that underscore their current limitations and paths towards addressing them. These insights are outlined here in detail.

## Appropriate regularization methods

The application of regularization is a central component to the out-of-distribution performance of both RAPPPID and INTREPPPID, as demonstrated by ablation experiments. To be effective, however, the method of regularization must be chosen to appropriately reflect the architecture of the model and the underlying data. RAPPPID and INTREPPPID offer an interesting look into how progressively overlapping, specifically-targeted regularization can result in gains in model robustness and generalization.

### Regularization in RAPPPID

Much of the regularization applied in the training of RAPPPID emerged from work by Stephen Merity *et al.* on adapting existing methods of regularization for use with LSTMs [164]. Among the methods Merity *et al.* describe, the one that resulted in the greatest improvement in their testing is employing weight-dropped LSTMs. Weight-dropping, also known as DropConnect, is a method of regularization similar to Dropout, but operates at the level of model weights rather than network activations [170, 171]. Comparisons to other architectures show that dropping out the hidden activations of LSTMs greatly underperforms weight-dropping [164]. Indeed, applying dropout to LSTM activations has been identified for quite some as a point of frustration when training recurrent models time, resulting in "dramatic" changes in model dynamics [172].

In our own ablation experiments conducted as part of the development of RAPPPID, we found that tokenizing amino acid sequences had the greatest effect on model performance on an out-of-distribution testing set. Tokenization improves RAPPPIDs performance by simultaneously reducing the number of timesteps, thereby mitigating the risk of vanishing gradients. Tokenization increases the pool of possible inputs at every timestep, as they no longer are limited to 22 amino acids, but any number of their permutations. Increasing the vocabulary like so can exacerbate the risk of overfitting. Luckily in the case of RAPPPID, the regularization opportunities afforded by tokenization greatly outweighed the risk of overfitting. The Unigram tokenization algorithm via the SentencePiece library was used to tokenize amino acid sequences. This algorithm allows for subword regularization, a process by which multiple segmentations for a sequence are sampled. For instance, the first 10 residues of the sequence for Human insulin (`MALWMRLLPL`) can be represented by the following sequence of tokens:

- `M, AL, W, M, R, L, L, PL`
- `M, A, L, W, M, R, LL, P, L`
- `M, A, L, W, M, RL, L, P, L`

and *etc*. One of all possible segmentations of the sequence are then chosen at random and used as the input to the model in question. The noise introduced by this process is regularizing and helps avoid the problem of rare tokens, whereby some tokens are under sampled at training time. Additionally, as part of the recommendations of the Merity *et al.* study, "embedding dropout" is employed, which is a process by which the embeddings of certain tokens are randomly zeroed out. This regularizes the network in a similar way to random dropout on the activations or weights of a network, but particularly targets the problem of overfitting to the large vocabulary that was created using the Unigram algorithm.

## Regularization in INTREPPPID

The focus on choosing regularization methods for RAPPPID centred on their appropriateness for the overall architecture. INTREPPPID inherited the architecture-focused regularization of RAPPPID, but also explored regularization methods that were informed by biological expert knowledge and inductive biases. Specifically, we sought to leverage our knowledge of orthologous protein sequences to improve the generalization of the model on testing examples from species other than those it was trained.

We took inspiration from a common regularization technique used in the field of computer vision where images are subtly augmented such that many pixels are altered, but their semantic meaning

is unchanged [173]. Orthologous proteins are similarly altered subtly by evolution, but typically retain their function and purpose. Incidentally, the application of augmentation techniques to proteins was simultaneously explored by Sun *et al.*, albeit by directly and randomly altering sequences rather than by using orthologous proteins [174]. This direct mutation approach scales much more easily than our orthologous method, as you are not limited by the number of orthologous sequences found in curated knowledge-bases. Unlike using orthologous protein sequences, randomly altering sequences can however result in proteins which lose their semantic meaning — much as an image of the number 9, when rotated 180°, loses its semantic meaning and becomes a 6.

The final result of adopting the orthologous locality loss introduced in the INTREPPPID study is a regularized latent space where proteins which are similar in function across species share similar vector embeddings.

## Questions regarding sequence identity and its role in data leakage

The three studies reproduced in this thesis are greatly indebted to the work of Park and Marcotte, which identified their strictest dataset type (C3) as having "test pairs sharing neither protein with the training set" [13]. Membership of a protein in either the testing, validation, set can be defined a number of ways. For instance, membership can be determined by shared UniprotKB identifiers of proteins. The trouble, however, in using protein names and most identifiers is that many protein names, while unique, have sequences that are identical, or nearly so. For instance, the January 2025 release of Uniprot has two entries for the human protein "Cytochrome c oxidase subunit 3" with identifiers `A0A023Q4K2` and `Q4EZD1` which differ by one additional serine residue in the former.

To help identify proteins that are identical in all but name, clustered identifiers such as those pre-computed in the UnirRf100 database, are commonly used. The two Uniprot identifiers for human Cytochrome c oxidase subunit 3 map to a single UniRef100 identifier (`UniRef100_A0A023Q4K2`). By mapping Uniprot identifiers to UniRef100 identifiers, it is possible to avoid considering proteins with identical sequences as distinct proteins.

A consequence of the high degree of homology between proteins is that many proteins approach, but fall short of 100% sequence identity. These homologous proteins often share the same function and interact with similar proteins, and therefore likely are a source of data leakage and warrant removal. Highly-conserved proteins like insulin (`P01308`) are prolific examples of this phenomena, sharing at least 90% sequence identity with 14 other proteins in Uniprot. Matters

get much worse when one expands the similarity threshold to at least 50%, revealing 53 other homologous proteins in Uniprot from species as diverse as *Rattus norvegicus* (Rat) and *Gorilla gorilla gorilla* (Western lowland gorilla). Once more, the UniRef database allows us to map similar proteins to the same identifiers. For example, proteins with at least 90% sequence identity and 50% sequence identity to insulin can be mapped to the identifiers `UniRef90_P01308` and `UniRef50_P01308`, respectively.

Park and Marcotte used, without a stated justification, a threshold of 40% sequence identity to determine protein similarity [13]. This threshold is used by many others when establishing a similarly strict training/testing split without much or often any discussion as to how it was selected. For instance, Bernett *et al.* used the CD-HIT clustering software to establish validation folds with no more than 40% sequence identity between them [15, 175]. Prior to that study, Hamp and Rost used a sequence-structure alignment database (HSSP) "...correspond[ing] to 40% pairwise sequence identity for 250 aligned residues" [14, 176, 177].

While the rationale for these seemingly arbitrary thresholds is not provided in those studies, one begins to emerge after a thorough review of the relevant literature, though there is no guarantee it is shared with the original authors. It is possible that these thresholds, particularly in the case of Hamp and Rost, were influenced by a 1991 study by Sander and Schneider which found that protein sequences with more than 30% sequence identity often share similar structure [176]. Subsequent work by Burkhard Rost found that proteins identified as having between 20-35% sequence identity, a so-called protein-sequence alignment "twilight zone", are far more likely to be "false negatives" [178]. In this context, false negatives are taken to mean proteins which, despite falling below the sequence identity threshold, are structurally similar and false positive are proteins above the threshold which do not share structural similarity. Rost reports:

> The number of false positives rose by a factor of 5 000 (Figure 2B). Hence, separating true and false positives switched from a trivial task (above 35% [pairwise sequence identity]) to the problem of finding needles in a haystack (20–30%). [178]

In light of these findings, a threshold of 40% sequence identity appears to be a fair compromise between identifying as many structurally similar proteins as possible while greatly reducing the number of proteins false-positives.

While we have identified a possible rationale for why the 40% sequence identity threshold has been identified — it is a proxy for the presence of structural similarity — we have yet to examine any evidence as to whether it is reasonable or useful. I argue here that eliminating nonindependence between the training and testing set must be balanced with creating unduly biased datasets.

Supervised machine learning algorithms rely on example input-output pairs to learn a generalized mapping between the two. These training examples are required to be representative of the

variation observed in the universe of input-output pairs that are expected to be correctly inferred. To understand whether a trained model is performing at the level intended, a set of evaluation input-output pairs must be chosen. If it is to give us an accurate representation of the model's performance, the evaluation set must simultaneously (1) be representative of the problem space and (2) avoid data leakage. Data leakage is a topic well discussed throughout this thesis, however it is useful here to use the taxonomy outlined by Kapoor and Narayanan to identify some of its common subtypes [160]. In the framework of this taxonomy, the Park and Marcotte study identified examples of L3.2 and L1.4 errors — *i.e.*, "nonindependence between training and test samples" and "duplicates in datasets", respectively [13, 160].

The L1.4, or duplication, error describes instances where duplicate data is present in the dataset increasing the risk that "the same data could exist in the training set and the test set" [160]. As explored previously, homologous sequences result in nearly identical sequences, and therefore PPI datasets which do not account for homology will almost certainly observe an L1.4 data leakage. By applying a maximum sequence identity threshold between testing and training datasets of 40%, Park and Marcotte side-step the issue of duplicates. A 100% threshold, however, would also eliminate the presence of duplicate sequences. A further 5-10% allowance for small variations corresponding to threshold between 95-90% would be appropriately sufficient to eliminate near-duplicates.

The L3.2, or nonindependence, error describes instances where testing samples are not drawn independently from the distribution of the problem space, but rather from one present in the training set that is not central to the scientific claim of the model. Examples provided by Kapoor and Narayanan include histology classification models which are evaluated on datasets of patients also present in the training set, or protein function classification models where a given protein family is present in both training and testing sets [179, 180]. In the case of PPI inference models, it is uncontroversial to assert that Park and Marcotte identified the L3.2 error caused by shared proteins in testing and training sets and remediated it by removing the dependence between the training and testing examples. It remains to be explored, however, which precise dependency was eliminated. Was it a dependency between shared *proteins* in the testing and training set, or rather a dependency between *homologous* proteins? Depending on one's interpretation of the role of the chosen sequence identity threshold, perhaps even a dependency on shared protein families is being avoided. If the former, most threshold values would be sufficient to mitigate the error. If the latter, then 40% would appear to be somewhat of an upper-bound among threshold values that would capture structural similarity, as was discussed earlier in reviewing the works of Sander and Schneider and Rost [176, 178].

The murkiness of this question is to be anticipated, according to Kapoor and Narayanan, who

warn:

> Handling nonindependence between the training and test sets in general (*i.e.*, without any assumptions about independence in the data) is a hard problem, because we might not know the underlying dependency structure of the task in many cases.[160]

Indeed, it is not easy to disentangle which aspects of the training and evaluation set distribution is intrinsic to learning a generalized model, and which serve only to artificially inflate the testing performance of the model via data leakage. While it may be tempting at first to resort to lower thresholds in the range of 30-50%, as many studies have done, there is a risk that models are being needlessly deprived of training data causing gaps in the coverage protein families which have representative samples in the training set. An over-correction could result in exacerbating the risk of biasing the training data, reducing its magnitude and variance, and ultimately resulting in overfitting.

Additionally, there is a question of whether stringent sequence identity thresholds are effective methods for excluding proteins on the basis of their protein family membership. Rost reminds us that his findings echo those of the earlier Sander and Schneider study: "...high levels of sequence similarity or identity do *not* ascertain structural similarity..." [178].

Further study into the extent of dependence between the training and testing splits of PPI datasets, at what level it occurs, and possible mitigation strategies, is gravely needed. Until then, interpretation of the findings in the works presented herein offer a suggestion of what such a study might confirm.

In the study in which it was introduced, RAPPPID was trained and tested on a C3-style dataset, however protein homology was not accounted for [10]. This is to say, proteins were excluded solely on the basis of shared STRING protein identifiers across training and testing datasets. The INTREPPPID study, by contrast, saw a RAPPPID model trained on a C3-style dataset with a 90% maximum sequence identity threshold between training and testing splits [9]. Introducing the 90% maximum sequence identity threshold results in PPI inference performance, as measured by AUROC, dropping by $\approx$ 8%. The last study presented in this thesis includes an analysis in the change of performance of PLLM models when trained on PPI datasets with maximum sequence identity threshold between their training and testing sets ranging between 50% to 90% [181]. Over that range, very little variation is observed. While further studies are required to draw any conclusions, we are left with the suggestion that, while it is pertinent to account for protein homology, little if any data leakage is mitigated by varying the maximum sequence identity threshold over the range of 50% to 90%.

## Distal impacts of efficient architectures

### As a means to improve generalization

RAPPPID's encoder, which make up the majority of the network by neuron count, is constructed using a twin neural network architecture. By sharing weights between the encoders of each protein, RAPPPID effectively halves the number of weights needed to infer PPIs. While the forward pass requires the same number of floating point operations (FLOPS), the number of FLOPS required for the backward-pass is halved. Seeing as the back-propagation is far more computational-intensive than the forward pass, the backward-pass is the more relevant target for optimization. A study by Richoux *et al.* confirms that twin neural network constructions observe far less overfitting than a twin construction [138]. This finding exemplifies the so-called "bias/variance dilemma" that relates the increase in neurons of a neural network to an increase in variance, leading to overfitting [182].

The INTREPPPID expands this concept by adding an additional three passes to compute an orthology-related task, forming what we've come to call a "quintuplet" network. This allows us to leave the number of effective weights in the network unchanged, increase the amount of data the network can learn from, and add an additional source of inductive bias. The result is a network that generalizes better than RAPPPID, despite maintaining the same number of weights, requires the same number of FLOPS for PPI inference, and requires a comparable amount of time to train.

High-quality, unbiased sources of biological data are difficult to come by due to the cost associated with generating such data, and PPI pairs are no exception. After filtering out low confidence interaction pairs and creating a strict validation dataset, only 39 229 training PPI pairs were used to train INTREPPPID. This stands in comparison to the 220 000 parameters that make-up INTREPPPID's encoder, alone. By increasing the amount of data the model is able to learn from (in the form of the orthologous locality task) while leaving the number of parameters unchanged, INTREPPPID reduces the degree of over-parameterization that is common among contemporaneous PPI inference methods.

It's important to note, however, that recent studies have found that over-parameterization in deep neural networks, past some (ill-defined) point, can cease to cause overfitting and result in well-calibrated, generalized models [183]. Further, it has been found that the variance-bias trade-off is not universally applicable [184]. These developments are, in part, leveraged in the study presented in Chapter 5. Briefly, PLLM's offer an interesting solution to the over-parameterization problem that plagues PPI inference models, along with many other protein-related tasks which suffer from constrained training datasets.

## Lowering barriers to PPI inference research

> *"Necessity is the mother of invention."*
> — Proverb

Computational resources are not equally distributed amongst researchers, not at the global scale nor most regional scales. An indication of this is that the top 10 super-computers by performance on the LINPACK benchmark all belong to countries in the Global North, according to TOP500 as of June 2025 [185, 186]. While not a perfect measure of the accessibility of computational resources, it follows that access is greater in wealthy countries that can and do allocate more funds to public and private research in the form of grants and tax credits.

Access to computational hardware is an important factor in being able to explore and successfully study research questions in the field of deep learning. In her influential essay "The Hardware Lottery", Sara Hooker argues that "[i]n the field of artificial intelligence research ... it is our tooling which has played a disproportionate role in deciding what ideas succeed (and which fail)" [187]. Over the course of my doctoral studies, I've had the good fortune to have had far greater access to computational resources than the majority of individuals globally. Despite this, I had access to many orders of magnitude fewer GPU hours and video random-access memory (VRAM) than leading research groups, particularly in the private sector. To narrow this gap, I applied a great focus on efficient deep learning architectures to make the most of the resources made available to me. This approach is advocated by Hooker as a way to mitigate the disparity in access to computational resources:

> Software needs to do more work, but it is also well positioned to do so. We have neglected efficient software throughout the era of Moore's law, trusting that predictable gains in compute would compensate for inefficiencies in the software stack. This means there are many low hanging fruit as we begin to optimize for more efficient code... [187]

A out-sized contributor to this hardware inequity on the side of software is the overwhelming popularity of the compute-hungry transformer architecture [123]. An oft-cited concern regarding the computational intensity of transformers is the self-attention mechanism at their heart, whose complexity scales quadratically with the length of its input sequence [188, 189]. As a result, the attention mechanism of the transformer architecture is often the target of improvement and optimization [190, 191, 192, 193].

Despite this, Iandola *et al.* found that the fully-connected layers in the feedforward network is responsible for the overwhelming share of FLOPS (73%) and latency on mobile devices (69.4%) in traditional transformer architectures [194]. They proceed to develop a convolution-based

alternative to the fully-connected layers in the feedforward modules of the transformer architecture [194]. Using this construction, we were capable of training SqueezeProt, a pLLM with less than 7% of the number of parameters of leading pLLM ProtBERT, computes tokens 5× faster, and achieves comparable PPI inference accuracy. VRAM is often the primary hardware bottleneck when training large neural networks and is commonly the main price differentiator between GPUs of the same hardware architecture. While SqueezeProt-U50 was trained on four Nvidia RTX 3090 GPUs, the relatively-low number of parameters allows for SqueezeProt to be trained on one such GPU. The RTX 3090 is a consumer GPU which had an MSRP of $1 499 USD at launch [195]. By comparison, ProtBERT was trained on two supercomputers — one of which is, as its authors describe, "the world's second fastest computer" comprised of 27 708 Nvidia Tesla V100 GPUs [148]. The Nvidia Tesla V100 is a data center GPU with an MSRP of $10 000 USD at launch [196].

Through the early adoption of efficient neural network architectures, we were able to conduct research on topics which are dominated by laboratories with asymmetric access to computational resources. Despite this victory, the inequities of the hardware lottery are unlikely to be optimized away by efficient architectures like the ones presented in this thesis. Hooker concludes:

> ...the hardware lottery has not gone away, and the gap between the winners and losers will grow increasingly larger. In order to avoid future hardware lotteries, we need to make it easier to quantify the opportunity cost of settling for the hardware and software we have. [187]

### Environmental impacts

> *"...invention is the mother of necessity."*
> — Thorstein Veblen
> The Instinct of Workmanship and the State of the Industrial Arts

As established in the previous subsection, deep learning is a power-hungry endeavour that relies on specialized hardware. Both the high energy and semiconductor demands of deep learning algorithms have associated environmental footprint. In this subsection, I explore briefly what the relevance of the efficient architectures adopted in this thesis may have on their relative environmental impact.

On the topic of energy consumption, both SqueezeProt and the top-ranking pLLMs analysed herein cause an Nvidia RTX 3090 GPU to consume just under its Thermal Design Power (TDP) of 350 watts. SqueezeProt can compute embeddings at a far faster rate than the top-ranking pLLMs, however, which results in energy requirements for embedding computation that is orders of magnitude lower for the same amino acid sequence. Specifically, SqueezeProt requires

approximately 8.33 millijoules (mJ) per amino acid embedded whereas ProtBERT, ESM2, and ProtT5 require 37.4 mJ, 52.3 mJ, and 67.5 mJ, respectively. As a practical example, it would require the same energy to power the average Canadian home for approximately 4 days to compute embeddings for all 253 061 696 proteins currently in UniProt using SqueezeProt[1]. This number balloons to over two weeks (16 days) in the case of ProtBERT and just under a month (29 days) for ProtT5. By eschewing transformers entirely, power consumption could be greatly reduced. RAPPPID causes an RTX 3090 to consume about half of its TDP according to my measurements, resulting in the use of a mere 0.543 mJ per amino acid embedded. In our UniProt-wide embedding scenario, RAPPPID would require the same energy as the typical Canadian household uses in 5 hours and 36 minutes. Notably, all network training in this study was conducted in the province of Québec where 99% of power delivered to the energy grid is produced by non-emitting sources [199].

Energy consumption is but one of the environmental consequences of deep-learning-based PPI inference methods. Another important ecological concern is the life-cycle impact of the manufacturing of the specialized semiconductors required to train and operate these models. According to Jennifer Gabrys, Chair in Media, Culture and Environment at the University of Cambridge, "[t]o produce a two-gram memory microchip, 1.3 kilograms of fossil fuels and materials are required" [200]. The aforementioned materials are severely injurious to human and animal health, with sweeping implications for individuals born in regions contaminated by electronic waste — including, but not limited to: "...suppressed immune response, hearing loss, altered hepatic and renal function, decline in oral anti-inflammatory ability, and abnormal [fasting blood glucose level]" as well as adverse birth outcomes [201]. By reducing the dependence on such specialized semiconductor hardware, we can hope to lessen the ecological and health impacts caused by their manufacturing and disposal. Gabrys writes:

> While the substances used in the manufacture of microchips contribute to pollution at both bodily and environmental levels, this condition intensifies with the sheer quantity of microchips manufactured. [200]

As discussed in the previous sub-section, VRAM is often the most relevant bottleneck when considering GPUs. Models, particularly LLMs, often require more VRAM than afforded by one GPU. For this reason, multiple GPUs are often required. By adopting an efficient architecture which requires a small fraction of the number of parameters of other leading pLLMs, it is possible to train SqueezeProt on a single consumer-class GPU rather than many.

It may be tempting to see these economies in the amount of energy and number of microchips required to train and operate the models described in this thesis and declare that there has

[1] Assuming an average amino acid length of 400 amino acids and a yearly household energy consumption of 85.4 gigajoules [197, 198].

been an overall improvement in the environmental impact of the model — that the ecological consequences have been partly mitigated. To do so would be premature. Merely observing gains of efficiency is not sufficient to come to such a conclusion. Gains in efficiency of a process can indeed increase the demand and usage of its inputs. William S. Jevons famously observed that the demand, and consequently the usage, of iron and coal increased as machines who required them became more efficient. Jevons' paradox, as it so became known, can be imagined in the context of this work: if pLLM-based PPI inference methods become more accurate and less costly, researchers and industry may adopt them where they had not before. This would result in energy and environment costs that would not have been incurred if pLLM-based PPI inference methods were inefficient, and therefore less desirable. Jevons' paradox, and the so called "rebound" debate which has arisen from it, is far from a settled matter and lacks broad expert consensus [202]. Also to be considered are the environmental impacts of the waste and pollution caused by the experimental techniques used to discover PPIs that inference methods might replace [203, 204, 205, 206].

Whether pLLM-based PPI inference, even efficient ones as I've described, are an improvement over their alternatives requires systematic study. Frameworks, such as Chistopher Becker's five factors of just sustainability design (JSD), are a useful basis for such a study [207]. Regrettably, performing such a thorough analysis could demand another doctoral degree's worth of study, which I have deemed out of scope.

# Chapter 6

# Conclusion

A robust model with the ability to reliably infer the interactions of two proteins given only their sequences in real-world, out-of-distribution would be deeply impactful — accelerating discoveries that currently take weeks or months to potentially mere minutes or seconds. Despite three decades of developing computational methods for PPI inference, the persistent failure of these approaches to generalize to real-world use cases is a challenge the scientific community continues to grapple with. This is partly due to the many years in which researchers were unaware of or neglected this challenge.

This thesis catalogues the work I've conducted over the last half decade to identify and mitigate obstacles which are at the root of this failure, while anticipating new obstacles to generalizability. A great deal of progress is needed before these sources of bias are sufficiently addressed and PPI inference models become useful for real-world tasks. The three studies presented here represent substantive developments towards that goal.

The development of RAPPPID demonstrated that a deep learning technique could be made to generalize to PPI pairs whose members don't appear in the training dataset. To my knowledge, the study which introduced RAPPPID was the first to do so. RAPPPID out-performed all of the contemporaneous leading PPI inference methods against which it was compared.

In our subsequent study, I expanded on the generalization of RAPPPID to include PPIs between unseen proteins originating from organisms other than that which the model was trained on. To do so, we devised an "orthologous locality" task whose formulation was motivated by both the biological underpinnings of our data as much as it was by practical considerations of model building.

Finally, our third study identified challenges in the nascent area of PLM-based models. It is in this study which I identify an emerging source of data leakage, before uncharacterized, which

already implicates a number of methods. It is also here that I demonstrate ways in which PPI inference models continue to fail to generalize to common, practical use-cases.

The studies in this thesis are humble but tangible contributions to a relatively new field which still holds many mysteries to unravel and challenges to overcome. I know not what the next three decades will uncover for PPI inference, but I am confident that a focus on methodological rigour and generalization lights the way forward.

# Appendix A

# Supplementary Material for "GRouNdGAN: GRN-guided simulation of single-cell RNA-seq data using causal generative adversarial networks"

**Table S1: Ranges of hyperparameters used for training of RAPPPID.** This table shows the range of hyperparameters and their increments considered when training RAPPPID. The final values of hyperparameters were chosen based on the performance on a randomly selected subset of the training set (i.e., a validation set).

| Hyperparameter | Number of LSTM Layers | Embedding Dropout Rate | LSTM Dropconnect Rate | Classifier Dropout Rate | Learning Rate |
|---|---|---|---|---|---|
| Range | 2-3 | 0.1-0.4 | 0.1-0.4 | 0.1-0.4 | $10^{-3}$-$10^{-2}$ |
| Increment | 1 | 0.1 | 0.1 | 0.1 | 0.009 |

**Table S2: The final values of hyperparameters selected for the analysis reported in Table 1 of the manuscript for different datasets.** The hyperparameters are selected based on the performance of a validation set (separate from the testing set).

| Experiment | Number of LSTM Layers | Embedding Dropout Rate | LSTM Dropconnect Rate | Classifier Dropout Rate | Learning Rate |
|---|---|---|---|---|---|
| C1 | 3 | 0.1 | 0.1 | 0.1 | $10^{-2}$ |
| C2 | 2 | 0.3 | 0.3 | 0.2 | $10^{-2}$ |
| C3 | 2 | 0.3 | 0.3 | 0.2 | $10^{-2}$ |

**Table S3: Statistics for the STRING C1, C2, and C3 datasets.**

| | C1 | C2 | C3 |
|---|---|---|---|
| Number of Proteins | 9 340 | 9 340 | 9 279 |
| Number of Edges | 263 130 | 263 130 | 173 914 |
| Training Split Size (%) | 79.9% | 64.1% | 97.0% |
| Validation Split Size (%) | 10.0% | 19.0% | 1.6% |
| Testing Split Size (%) | 10.1% | 16.9% | 1.4% |
| Train/Validation Protein Overlap (%) | 100% | 83.2% | 0% |
| Train/Test Protein Overlap (%) | 100% | 84.3% | 0% |
| Train Edges w/ Test Proteins | 9 137 | 3 758 | 0 |
| Train Edges w/ Validation Proteins | 9 170 | 3 663 | 0 |

**Table S4: Repeatability of results across randomly generated C3 datasets.**

| Dataset # | Validation Loss | Validation AUROC | Validation AUPR | Testing AUROC | Testing AUPR |
|---|---|---|---|---|---|
| 1 | 0.555 | 0.804 | 0.803 | 0.804 | 0.809 |
| 2 | 0.560 | 0.804 | 0.807 | 0.806 | 0.803 |
| 3 | 0.559 | 0.780 | 0.776 | 0.782 | 0.789 |

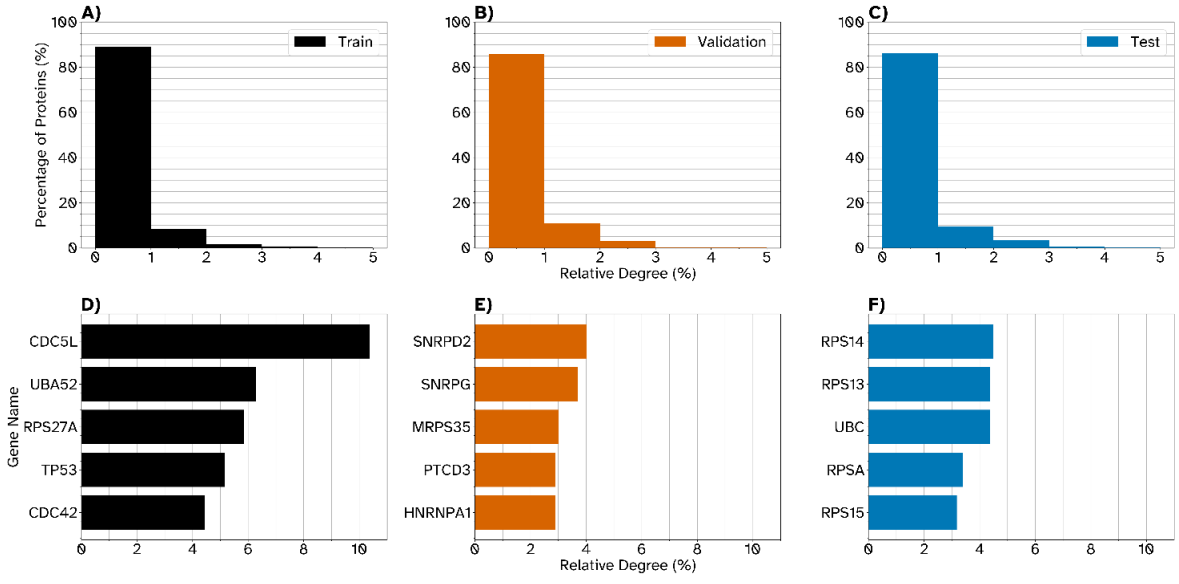



**Figure S1: Relative Degree of Proteins in the *H. sapiens* STRING C3 dataset.** The distribution of the relative degree of proteins between the three splits of the *H. sapiens* STRING C3 datasets reveal that the vast majority of proteins have a relative degree of less than 1% **(A-C)**. The top five proteins with the highest relative degrees are plotted in bar graphs D through F. *CDC5L* has a relative degree of 10.37%, the highest of the entire dataset. Many of the proteins of high relative degree belong to the families of ribosomal or ribonucleoproteins.

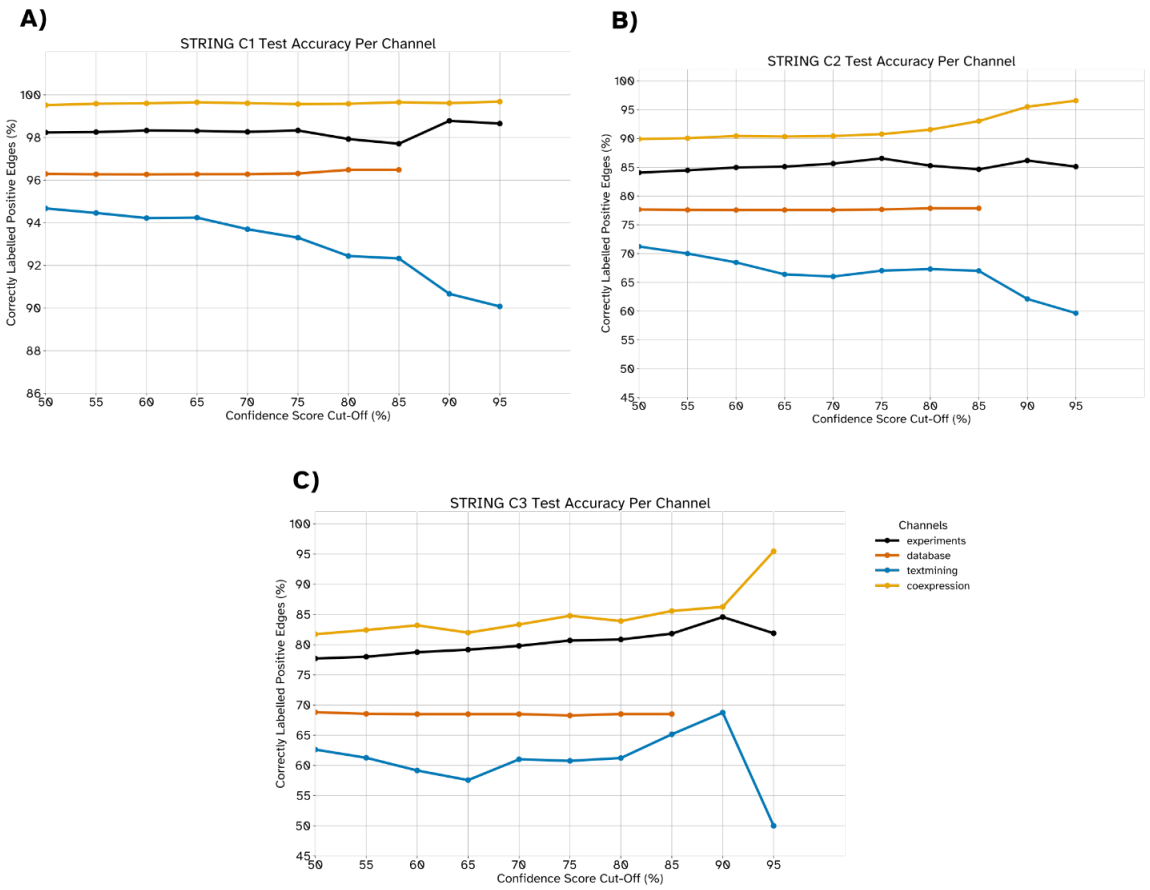



**Figure S2: Accuracy of positive edges across edge confidence stratified by STRING channels.** The percentage of correctly labelled positive edges are plotted for each major STRING channel in the C1 **(A)**, C2 **(B)**, and C3 **(C)** datasets. The x-axis denotes the channel edge confidence cut-off score for each curve's respective channel.

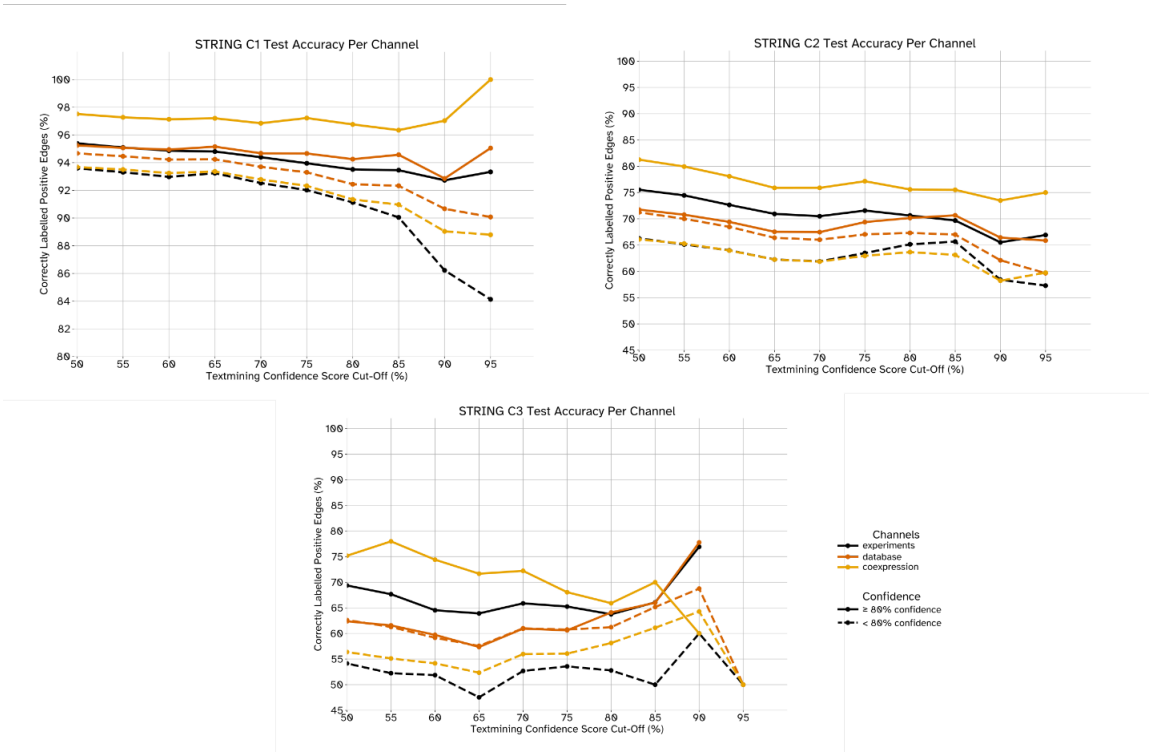



**Figure S3: Accuracy of positive edges across edge confidence stratified by STRING channels.** The percentage of correctly labelled positive edges are plotted for each major STRING channel, excluding text-mining. Edges are excluded according to the text-mining confidence threshold (x-axis). The solid curves include edges which have a channel confidence ≤ 80% for the channel indicated by the curve's colour. Dashed curves conversely include edges whose channel confidence is 80% for the channel indicated by the curve's colour. Data shown reflects the C1 model/dataset in panel **(A)**, the C2 model/dataset in panel **(B)**, and the C3 model/dataset in panel **(C)**.

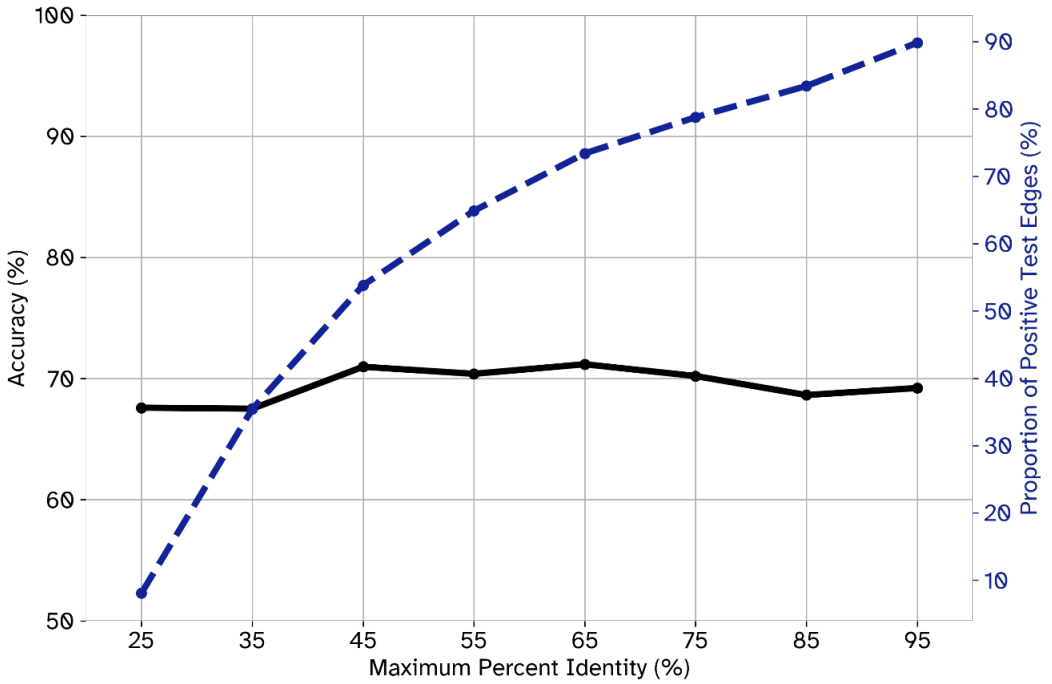



**Figure S4: Accuracy of positive edges as a function of similarity between testing and training proteins in C3.** The similarity between testing and training proteins was measured using their percent identity as computed by NCBI's PSI-BLAST software. The highest percent identity between any training protein and a testing protein in a testing edge was considered to be that testing edge's "maximum percent identity". The percentage of accurately labelled positive edges (black curve, left y-axis) is reported for edges with maximum percent identities lower than the threshold reported on the x-axis. The proportion of testing edges for each threshold values is reported by the dashed blue curve and the right y-axis.

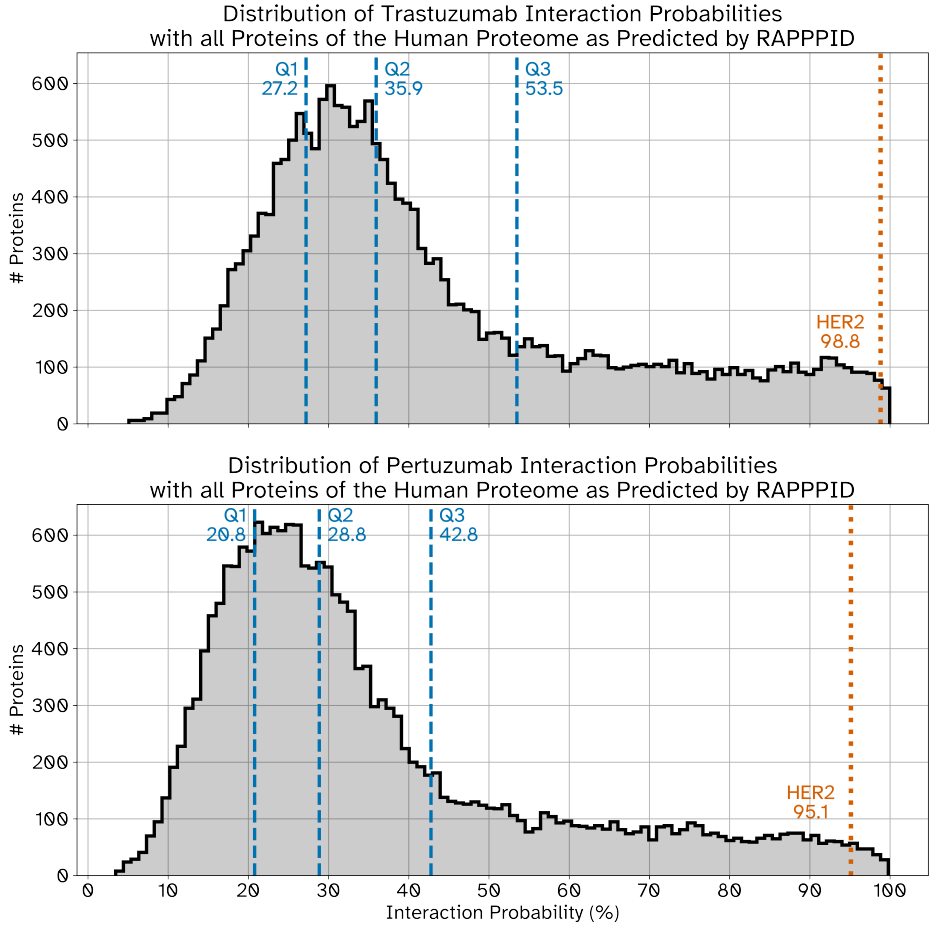



**Figure S5: Distribution of Interaction Probabilities of Trastuzumab and Pertuzumab with all Proteins of the Human Proteome as Predicted by rapppid.** The probability of interaction between the antibodies (Trastuzumab and Pertuzumab) and each protein in the human proteome (according to STRING) were individually predicted using RAPPPID. The distribution of these probabilities is here shown for interactions between Trastuzumab (top) and Pertuzumab (bottom). The quartiles are marked by blue dashed lines, and illustrate that the majority of proteins have a probability of interacting less than 35.9% and 28.8% for Trastuzumab and Pertuzumab, respectively. The dotted orange line demarks where HER2 is positioned in the distribution, holding a probability of interaction of 98.8% and 95.1% with Trastuzumab and Pertuzumab, respectively.

# Appendix B

# Supplementary Material for "Causal single-cell RNA-seq simulation, in silico perturbation, and GRN inference benchmarking using GRouNdGAN-Toolkit"

## Supplementary Methods

### Preparing strict PPI datasets

A PPI dataset $\mathscr{D}$ is a set of interacting protein-pairs reflecting binary interactions $D_i \in \mathscr{D}$. $\mathscr{D}$ is partitioned into three mutually disjoint subsets: a training subset ($\mathscr{D}_{\text{Tr}}$), a testing subset ($\mathscr{D}_{\text{Te}}$), and a validation subset ($\mathscr{D}_{\text{V}}$). Let $P_{\text{Tr}}$, $P_{\text{Te}}$, and $P_{\text{V}}$ be the set of proteins present in $\mathscr{D}_{\text{Tr}}$, $\mathscr{D}_{\text{Te}}$, and $\mathscr{D}_{\text{V}}$, respectively, and let $\mathscr{P}$ be the collection of protein sets $P_{\text{Tr}}, P_{\text{Te}}, P_{\text{V}}$.

We define a strict PPI dataset as a dataset $\mathscr{D}$ (with its collection of protein sets $\mathscr{P}$) that fulfills the following two criteria.

*Criterion 1 — Distinct Protein Identity*
The resulting protein sets $P_{\text{Tr}}, P_{\text{Te}}, P_{\text{V}}$ of the dataset must be mutually disjoint:

$$\forall Q, R \in \mathscr{P}\ Q \cap R = \emptyset \text{ if } Q \neq R.$$

*Criterion 2 — Distinct Sequence Identity*
The amino acid sequence of any pair of proteins belonging to different protein subsets must have a sequence identity no greater than 90%:

$$\forall Q, R \in \mathcal{P},\ \forall q \in Q\ \forall r \in R\ f(q,r) \leq 90\% \text{ if } Q \neq R$$

where $f(\cdot,\cdot)$ is some protein sequence identity function.

For the datasets used in the experiments carried out herein, the UniRef90 identifiers provided by UniProt were used to determine sequence identity, and as such we adopt the same definition of sequence identity [1,2].

*High-confidence examples of PPIs*

To minimize our exposure to falsely labelled PPIs, we required a database that provided information regarding the confidence of the interactions. We selected the STRING database as the source of PPIs for our datasets as it is large, contains information for many species, and integrates confidence scores from a number of modalities [3]. We exclude all PPIs which do not have a combined confidence score greater or equal to 90%, and only include PPIs which are classified as "physical interactions". PPI information for seven species were included in our analyses.

| Scientific Name | Common Name | NCBI Taxon ID | Patristic Distance from *H. sapiens* |
|---|---|---|---|
| *Homo sapiens* | Human | 9606 | 0.000 |
| *Mus musculus* | House mouse | 10090 | 0.0455 |
| *Danio rerio* | Zebrafish | 7955 | 0.156 |
| *Drosophila melanogaster* | Fruit fly | 7227 | 0.358 |
| *Caenorhabditis elegans* | Nematode | 6239 | 0.455 |
| *Arabidopsis thaliana* | Thale cress | 3702 | 0.693 |
| *Saccharomyces cerevisiae* | Baker's yeast | 4932 | 0.734 |

Patristic distance is a measure of the divergence of two lineages. Patristic distances were computed using DendroPy and data from iTOL [4,5].

*Negative examples of PPIs*

As INTREPPPID is a supervised method, it requires examples of pairs of proteins which do not interact (negative examples), in addition to pairs which do (positive examples). Obtaining negative examples of PPIs is a fraught process as there exists no one agreed upon "gold standard" of proteins which do not interact with one another. While many approaches for obtaining negative PPI examples have been devised over the years, evidence has shown that many of these methods introduce a great amount of bias [6]. For this reason, we opted to sample pairs of proteins in a uniformly random manner to represent our negative examples. For the training set, negatives were randomly drawn without replacement from the set $N_{\text{Tr}}$, defined as:

$$N_{\text{Tr}} = \left\{\{p,q\} \,|\, p \in P_{\text{Tr}}, q \in P_{\text{Tr}}, \{p,q\} \notin \mathcal{D}\right\}$$

Negative examples for the testing and validation sets were drawn from $N_{\text{Te}}$ and $N_{\text{V}}$, which were similarly defined as the set of protein-pairs of $P_{\text{Te}}$ and $P_{\text{V}}$, respectively, which were not in $\mathscr{D}$ (*i.e.*, were not considered interacting in the dataset).

Random sampling of protein-pairs requires the fewest assumptions about the distribution of negative pairs, thereby reducing bias, albeit at the risk of false-negatives. Through the exclusion of known interactions from $N_{\text{Tr}}$, however, we've reduced the likelihood of drawing false-negatives.

*Considerations for cross-species datasets*

For experiments herein, it was necessary to prepare datasets suitable for testing the cross-species task. Namely, we needed to construct training sets whose *ppi*s were composed of proteins of one species, and many corresponding testing sets each belonging to a different species. All combinations of the training set and the many testing sets must satisfy both Criteria 1 and 2. The first criterion is typically met *de facto*, as protein identifiers between species normally do not overlap. In order for the second criterion to be met, positive and negative interactions of proteins that do not satisfy Criterion 2 were excluded from the datasets.

# Supplementary Tables and Figures

## Tables

### Table S1

Table S1 is provided as a separate .xlsx spreadsheet file.

Its caption is as follows:

> Statistics of each dataset used in this study. Datasets are listed by source organism and random seed used to generate the random splits.

This file is available for download from OUP Briefings in Bioinformatics:
https://doi.org/10.1093/bib/bbae405.

**Table S2: Hyperparameters used to train INTREPPPID in all experiments.**

| Hyperparameter | Value |
|---|---|
| Batch Size | 80 |
| Embedding Size | 64 |
| Number of RNN Layers | 2 |
| RNN Drop-connect Rate | 0.3 |
| Embedding Drop-out Rate | 0.3 |

### Table S3

Table S3 is provided as a separate .xlsx spreadsheet file.

Its caption is as follows:

> Confusion matrix statistics of various methods trained and tested on *H. sapiens* data.

This file is available for download from OUP Briefings in Bioinformatics:
https://doi.org/10.1093/bib/bbae405.

**Table S4: Performance of INTREPPPID trained on *H. sapiens* datasets with a 1:1 negative-to-positive interaction ratios, and tested on increasing negative-to-positive ratios.**

| Negative:Positive | 1:1 | 2:1 | 10:1 | 50:1 | 100:1 |
|---|---|---|---|---|---|
| AUROC Mean | 0.752 | 0.723 | 0.733 | 0.728 | 0. 733 |
| AUROC Std | 0.017 | 0.004 | 0.006 | 0.011 | 0.004 |

### Table S5

Table S5 is provided as a separate .xlsx spreadsheet file.

Its caption is as follows:

> Metrics of various PPI inference methods trained on *H. sapiens* data, and tested on various species. Each model was tested on a total of twenty-four dataset: four different random testing splits for each six different organism. The metrics reported here are averaged over all four datasets, with the standard deviation between the four datasets also reported. The first worksheet (AUROC) reports the area-under-the-receiver-operator-curve (AUROC). The second worksheet (AP) reports the average-precision. The third worksheet (F150) reports the $F_1$ measure with a 50% threshold. The fourth worksheet (MCC50) reports the Matthew's correlation coefficient (MCC) with a 50% threshold. The fifth and final worksheet (Brier) reports the Brier score. Higher values indicate better performance for all metrics but the Brier score, where the inverse is true.

This file is available for download from OUP Briefings in Bioinformatics:
https://doi.org/10.1093/bib/bbae405.

**Table S6: Performance of INTREPPPID trained on *H. sapiens* datasets with scrambled and not scrambled orthologue information.** Bold-face indicates the best value for each metric.

| Scramble Status | Test AUROC | Test AP | Test MCC@50% | Test Brier Score |
|---|---|---|---|---|
| Not Scrambled | **0.761** | **0.737** | **0.378** | **0.198** |
| Scrambled | 0.524 | 0.527 | 0.017 | 0.314 |

**Table S7: Performance of INTREPPPID when trained on *H. sapiens* dataset which omits orthology data for the seven species tested below, relative to the performance of INTREPPPIED trained on dataset which includes all orthology data available.**

| Testing Dataset | Test AUROC | Test AP | Test MCC@50% | Test Brier Score |
|---|---|---|---|---|
| *H. sapiens* | -0.49% | -1.36% | -9.00% | 2.00% |
| *M. musculus* | -13.21% | -14.79% | 34.39% | 21.06% |
| *D. rerio* | -13.69% | -16.16% | -38.85% | 8.66% |
| *D. melanogaster* | -11.29% | -11.67% | -26.48% | 5.81% |
| *C. elegans* | -12.51% | -17.58% | -13.45% | -0.73% |
| *S. cerevisiae* | -9.21% | -15.12% | -13.31% | -3.17% |
| *A. thaliana* | -9.29% | -11.55% | -30.75% | 5.39% |

### Table S8

Table S8 is provided as a separate .xlsx spreadsheet file.

Its caption is as follows:

> Results of the statistical over-expression analysis conducted on top INTREPPPID-unique positive hits for seven species. The statistical over-expression of GO Molecular Function annotations among proteins that make up the top INTREPPPID-unique positive hits was measured relative to the proteins which make up the testing dataset for the specific species tested. A Fisher's Exact test was performed by the online PANTHER Classification System. The fold enrichment, raw $p$ values, and corrected $p$ values using Benjamini-Hochberg false-discover rate (FDR) correction are reported for GO Molecular Function annotations found to be over-expressed. Only results with a corrected $p$ value of less than 0.05 are included in this table. Each worksheet in the spreadsheet is labelled with a species name, and reports the over-expression analysis for the testing dataset that belongs to that species.

This file is available for download from OUP Briefings in Bioinformatics:
https://doi.org/10.1093/bib/bbae405.

**Table S9: Analysis of 22 randomly sampled articles advertising online PPI-related tools and the online availability of said tools as of January 2024.** Articles were found using the SemanticScholar.org academic search engine with the search terms "protein-protein interaction website".

| DOI | CorpusID | Year Published | URI | Status |
|---|---|---|---|---|
| 10.1093/nar/gks434 | 9914396 | 2012 | Link | Dead |
| 10.1007/978-1-0716-0892-0_12 | 226218388 | 2020 | Link | Alive |
| 10.1007/978-1-4939-9873-9_12 | 203659921 | 2020 | Link | Alive |
| 10.1515/jib-2006-27 | 16195629 | 2006 | Link | Dead |
| 10.1089/adt.2013.506 | 12969509 | 2013 | Link | Dead |
| 10.1093/nar/gkx333 | 31083221 | 2017 | Link | Alive |
| 10.1371/journal.pone.0056228 | 2014834 | 2013 | Link | Inaccessible[1] |
| 10.1093/nar/gkp335 | 3041302 | 2009 | Link | Inaccessible[2] |
| 10.1093/NAR/GKG056 | 15003973 | 2003 | Link | Dead |
| 10.1109/CBMS.2008.113 | 19040991 | 2009 | Link | Dead |
| 10.1093/nar/gkx385 | 3268569 | 2017 | Link | Alive |
| 10.1093/nar/gkac1000 | 253479599 | 2022 | Link | Alive |
| 10.7726/AJBCB.2014.1003 | 12004796 | 2014 | Link | Alive |
| 10.1093/bioinformatics/btn584 | 206340817 | 2009 | Link | Dead |
| 10.1186/1471-2105-11-S10-P1 | 40906621 | 2010 | Link | Alive |
| 10.1093/bioinformatics/btx312 | 4420839 | 2017 | Link | Alive |
| 10.1155/2013/737591 | 5548876 | 2013 | Link | Dead |
| 10.1038/nmeth.1931 | 52853893 | 2012 | Link | Alive |
| 10.3389/fgene.2021.722198 | 229549097 | 2020 | Link | Alive |
| 10.1093/nar/gkv1072 | 8843610 | 2015 | Link | Alive |
| 10.1093/nar/gkt1210 | 560515 | 2013 | Link | Dead |

[1]Presents a login page with no context or registration option.

[2]URI resolves but the described online web server is not present.

## Figures

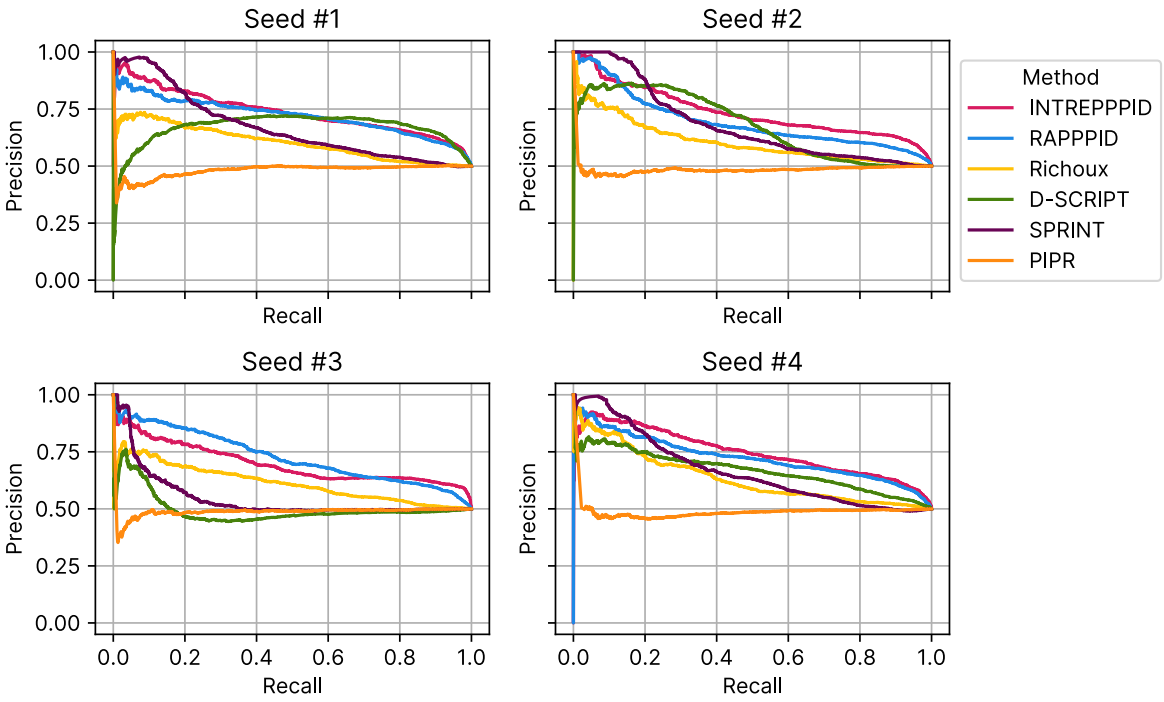



**Figure S1: Precision-Recall curves of various PPI inference methods tested and trained on *H. sapiens* data.** Each panel reports the Precision-Recall curve for one of four different datasets.

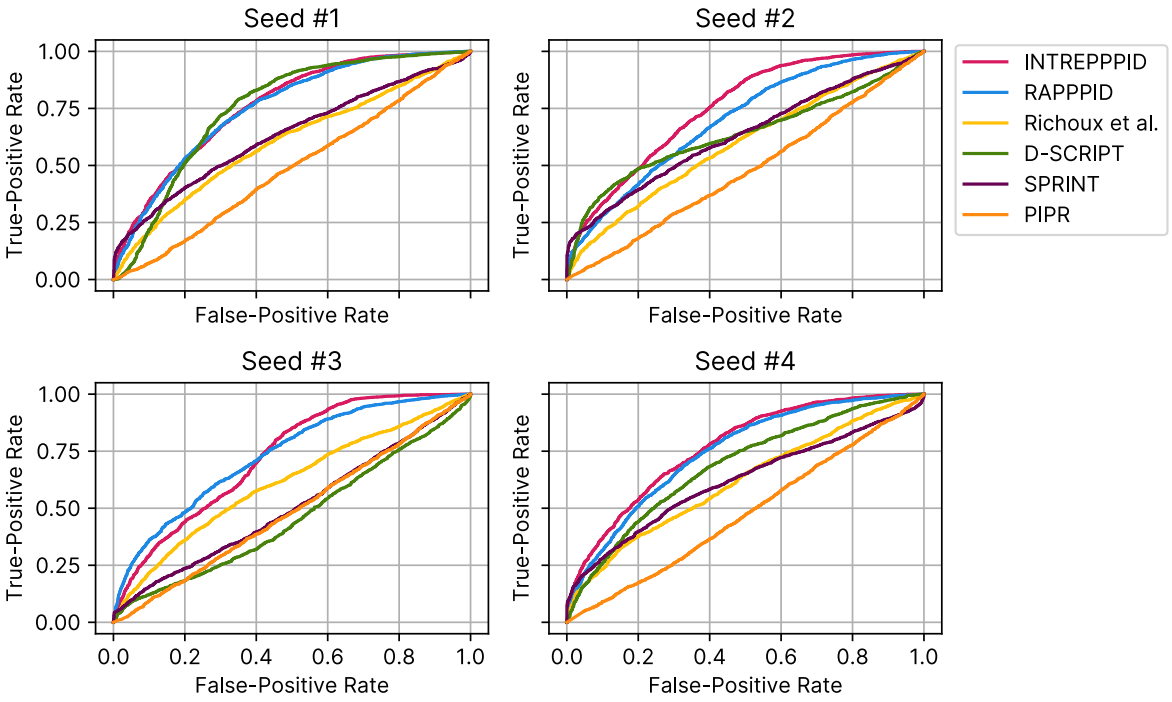



**Figure S2: Receiver-Operator curves of various PPI inference methods tested and trained on *H. sapiens* data.** Each panel reports the Receiver-Operator curve for one of four different datasets.

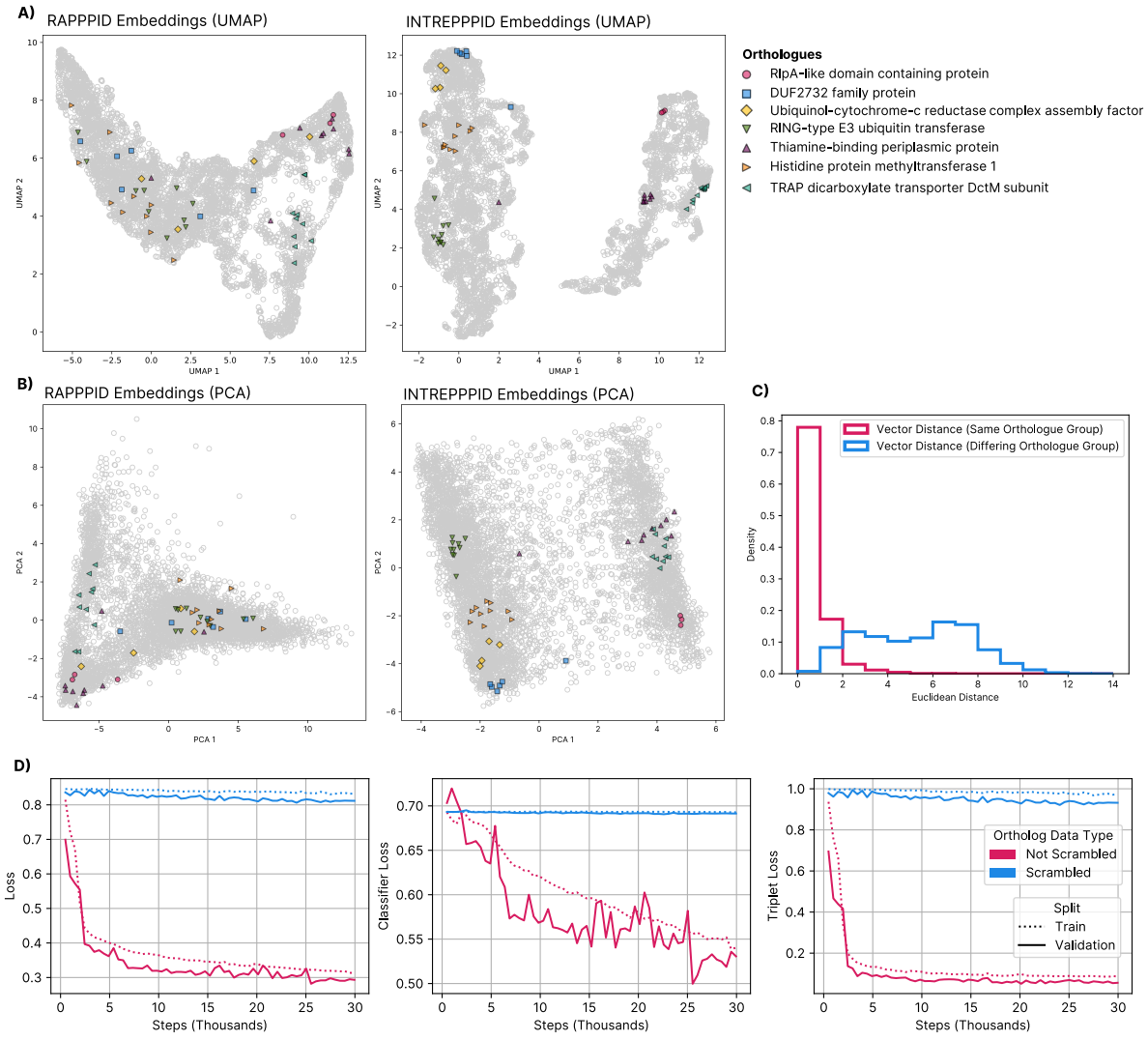



**Figure S3: The effect of orthologous locality task. (A-B)** A random sample of protein sequences from various species were embedded using RAPPPID (left) and INTREPPPID (right) and projected onto a 2D space using the **(A)** UMAP and **(B)** PCA algorithm. Distinct markers indicate proteins which belong to the same orthologous group according to the OMA database. The orthologous groups shown were chosen at random. **(C)** Histogram of the Euclidean distance between INTREPPPID latent vector representations of protein sequences of the same orthologous group (magenta) and of differing orthologous groups (blue). **(D)** Training (dotted line) and validation loss (solid line) of INTREPPPID trained on *H. sapiens* datasets whose orthologue data is "scrambled" (blue) and "not scrambled" (magenta). The overall loss (left), the loss of the classification network (middle), and the loss of the triplet network (right) are plotted above, demonstrating that the "scrambled" dataset fails to optimize any aspect of the loss in any appreciable way.

## References

1. Suzek BE, Wang Y, Huang H, *et al.* UniRef clusters: a comprehensive and scalable alternative for improving sequence similarity searches. *Bioinformatics* 2015; 31:926–932

2. The UniProt Consortium, Bateman A, Martin M-J, *et al.* UniProt: the Universal Protein Knowledgebase in 2023. *Nucleic Acids Research* 2023; 51:D523–D531

3. Szklarczyk D, Kirsch R, Koutrouli M, *et al.* The *string* database in 2023: protein–protein association networks and functional enrichment analyses for any sequenced genome of interest. *Nucleic Acids Research* 2023; 51:D638–D646

4. Sukumaran J, Holder MT. DendroPy: a Python library for phylogenetic computing. *Bioinformatics* 2010; 26:1569–1571

5. Letunic I, Bork P. Interactive Tree Of Life (iTOL) v5: an online tool for phylogenetic tree display and annotation. *Nucleic Acids Research* 2021; 49:W293–W296

6. Ben-Hur A, Noble WS. Choosing negative examples for the prediction of protein-protein interactions. *BMC Bioinformatics* 2006; 7:S2–S2

# Bibliography

[1] Sherlyn Jemimah and M. Michael Gromiha. Insights into changes in binding affinity caused by disease mutations in protein-protein complexes. *Computers in Biology and Medicine*, 123:103829, August 2020.

[2] Roger G. Ptak, William Fu, Brigitte E. Sanders-Beer, Jonathan E. Dickerson, John W. Pinney, David L. Robertson, Mikhail N. Rozanov, Kenneth S. Katz, Donna R. Maglott, Kim D. Pruitt, and Carl W. Dieffenbach. Short communication: Cataloguing the HIV type 1 human protein interaction network. *AIDS Research and Human Retroviruses*, 24(12):1497–1502, December 2008.

[3] David E. Gordon, Gwendolyn M. Jang, Mehdi Bouhaddou, Jiewei Xu, Kirsten Obernier, Kris M. White, Matthew J. O'Meara, Veronica V. Rezelj, Jeffrey Z. Guo, Danielle L. Swaney, Tia A. Tummino, Ruth Hüttenhain, Robyn M. Kaake, Alicia L. Richards, Beril Tutuncuoglu, Helene Foussard, Jyoti Batra, Kelsey Haas, Maya Modak, Minkyu Kim, Paige Haas, Benjamin J. Polacco, Hannes Braberg, Jacqueline M. Fabius, Manon Eckhardt, Margaret Soucheray, Melanie J. Bennett, Merve Cakir, Michael J. McGregor, Qiongyu Li, Bjoern Meyer, Ferdinand Roesch, Thomas Vallet, Alice Mac Kain, Lisa Miorin, Elena Moreno, Zun Zar Chi Naing, Yuan Zhou, Shiming Peng, Ying Shi, Ziyang Zhang, Wenqi Shen, Ilsa T. Kirby, James E. Melnyk, John S. Chorba, Kevin Lou, Shizhong A. Dai, Inigo Barrio-Hernandez, Danish Memon, Claudia Hernandez-Armenta, Jiankun Lyu, Christopher J. P. Mathy, Tina Perica, Kala Bharath Pilla, Sai J. Ganesan, Daniel J. Saltzberg, Ramachandran Rakesh, Xi Liu, Sara B. Rosenthal, Lorenzo Calviello, Srivats Venkataramanan, Jose Liboy-Lugo, Yizhu Lin, Xi-Ping Huang, YongFeng Liu, Stephanie A. Wankowicz, Markus Bohn, Maliheh Safari, Fatima S. Ugur, Cassandra Koh, Nastaran Sadat Savar, Quang Dinh Tran, Djoshkun Shengjuler, Sabrina J. Fletcher, Michael C. O'Neal, Yiming Cai, Jason C. J. Chang, David J. Broadhurst, Saker Klippsten, Phillip P. Sharp, Nicole A. Wenzell, Duygu Kuzuoglu-Ozturk, Hao-Yuan Wang, Raphael Trenker, Janet M. Young, Devin A. Cavero, Joseph Hiatt, Theodore L. Roth, Ujjwal Rathore, Advait Subramanian, Julia Noack, Mathieu Hubert, Robert M. Stroud, Alan D. Frankel, Oren S. Rosenberg, Kliment A. Verba, David A. Agard, Melanie Ott, Michael

Emerman, Natalia Jura, Mark von Zastrow, Eric Verdin, Alan Ashworth, Olivier Schwartz, Christophe d'Enfert, Shaeri Mukherjee, Matt Jacobson, Harmit S. Malik, Danica G. Fujimori, Trey Ideker, Charles S. Craik, Stephen N. Floor, James S. Fraser, John D. Gross, Andrej Sali, Bryan L. Roth, Davide Ruggero, Jack Taunton, Tanja Kortemme, Pedro Beltrao, Marco Vignuzzi, Adolfo García-Sastre, Kevan M. Shokat, Brian K. Shoichet, and Nevan J. Krogan. A SARS-CoV-2 protein interaction map reveals targets for drug repurposing. *Nature*, 583(7816):459–468, July 2020.

[4] Marc Vidal and Stanley Fields. The yeast two-hybrid assay: still finding connections after 25 years. *Nature Methods*, 11(12):1203–1206, Dec 2014.

[5] Jamie Snider, Max Kotlyar, Punit Saraon, Zhong Yao, Igor Jurisica, and Igor Stagljar. Fundamentals of protein interaction network mapping. *Molecular Systems Biology*, 11(12):848, Dec 2015.

[6] Teck Yew Low, Saiful Effendi Syafruddin, M. Aiman Mohtar, Adaikkalam Vellaichamy, Nisa Syakila A Rahman, Yuh-Fen Pung, and Chris Soon Heng Tan. Recent progress in mass spectrometry-based strategies for elucidating protein–protein interactions. *Cellular and Molecular Life Sciences*, 78(13):5325–5339, July 2021.

[7] Melina Altmann, Stefan Altmann, Claudia Falter, and Pascal Falter-Braun. High-quality yeast-2-hybrid interaction network mapping. *Current Protocols in Plant Biology*, 3(3):e20067, September 2018.

[8] Jitender Mehla, J. Harry Caufield, and Peter Uetz. Mapping protein-protein interactions using yeast two-hybrid assays. *Cold Spring Harbor Protocols*, 2015(5):442–452, May 2015.

[9] Joseph Szymborski and Amin Emad. INTREPPPID — an orthologue-informed quintuplet network for cross-species prediction of protein–protein interaction. *Briefings in Bioinformatics*, 25(5):bbae405, September 2024.

[10] Joseph Szymborski and Amin Emad. Rapppid: towards generalizable protein interaction prediction with awd-lstm twin networks. *Bioinformatics*, 38(16):3958–3967, August 2022.

[11] Damian Szklarczyk, Rebecca Kirsch, Mikaela Koutrouli, Katerina Nastou, Farrokh Mehryary, Radja Hachilif, Annika L. Gable, Tao Fang, Nadezhda T. Doncheva, Sampo Pyysalo, Peer Bork, Lars J. Jensen, and Christian von Mering. The string database in 2023: protein-protein association networks and functional enrichment analyses for any sequenced genome of interest. *Nucleic Acids Research*, 51(D1):D638–D646, January 2023.

[12] Noemi del Toro, Anjali Shrivastava, Eliot Ragueneau, Birgit Meldal, Colin Combe, Elisabet Barrera, Livia Perfetto, Karyn How, Prashansa Ratan, Gautam Shirodkar, Odilia Lu, Bálint Mészáros, Xavier Watkins, Sangya Pundir, Luana Licata, Marta Iannuccelli, Matteo Pellegrini, Maria Jesus Martin, Simona Panni, Margaret Duesbury, Sylvain D Vallet, Juri Rappsilber, Sylvie Ricard-Blum, Gianni Cesareni, Lukasz Salwinski, Sandra Orchard, Pablo Porras, Kalpana Panneerselvam, and Henning Hermjakob. The IntAct database: efficient access to fine-grained molecular interaction data. *Nucleic Acids Research*, 50(D1):D648–D653, January 2022.

[13] Yungki Park and Edward M. Marcotte. Flaws in evaluation schemes for pair-input computational predictions. *Nature Methods*, 9(12):1134–1136, December 2012.

[14] Tobias Hamp and Burkhard Rost. More challenges for machine-learning protein interactions. *Bioinformatics (Oxford, England)*, 31(10):1521–1525, May 2015.

[15] Judith Bernett, David B Blumenthal, and Markus List. Cracking the black box of deep sequence-based protein–protein interaction prediction. *Briefings in Bioinformatics*, 25(2):bbae076, March 2024.

[16] Michael P. H. Stumpf, Thomas Thorne, Eric de Silva, Ronald Stewart, Hyeong Jun An, Michael Lappe, and Carsten Wiuf. Estimating the size of the human interactome. *Proceedings of the National Academy of Sciences of the United States of America*, 105(19):6959–6964, May 2008.

[17] Marc Vidal. How much of the human protein interactome remains to be mapped? *Science Signaling*, 9(427):eg7–eg7, May 2016.

[18] Philipp Blohm, Goar Frishman, Pawel Smialowski, Florian Goebels, Benedikt Wachinger, Andreas Ruepp, and Dmitrij Frishman. Negatome 2.0: a database of non-interacting proteins derived by literature mining, manual annotation and protein structure analysis. *Nucleic Acids Research*, 42(D1):D396–D400, January 2014.

[19] Anne-Claude Gingras and Brian Raught. Beyond hairballs: The use of quantitative mass spectrometry data to understand protein–protein interactions. *FEBS Letters*, 586(17):2723–2731, August 2012.

[20] Jonathan Flint and Trey Ideker. The great hairball gambit. *PLOS Genetics*, 15(11):e1008519, November 2019.

[21] Bijay Jassal, Lisa Matthews, Guilherme Viteri, Chuqiao Gong, Pascual Lorente, Antonio Fabregat, Konstantinos Sidiropoulos, Justin Cook, Marc Gillespie, Robin Haw, Fred Loney, Bruce May, Marija Milacic, Karen Rothfels, Cristoffer Sevilla, Veronica Shamovsky,

Solomon Shorser, Thawfeek Varusai, Joel Weiser, Guanming Wu, Lincoln Stein, Henning Hermjakob, and Peter D'Eustachio. The reactome pathway knowledgebase. *Nucleic Acids Research*, 48(D1):D498–D503, January 2020.

[22] George Tsitsiridis, Ralph Steinkamp, Madalina Giurgiu, Barbara Brauner, Gisela Fobo, Goar Frishman, Corinna Montrone, and Andreas Ruepp. CORUM: the comprehensive resource of mammalian protein complexes–2022. *Nucleic Acids Research*, 51(D1):D539–D545, January 2023.

[23] Matthias W. Hentze, Alfredo Castello, Thomas Schwarzl, and Thomas Preiss. A brave new world of RNA-binding proteins. *Nature Reviews. Molecular Cell Biology*, 19(5):327–341, May 2018.

[24] Jamie Heimburg-Molinaro, Akul Y. Mehta, Catherine A. Tilton, and Richard D. Cummings. Insights into glycobiology and the protein-glycan interactome using glycan microarray technologies. *Molecular Cellular Proteomics□: MCP*, 23(11):100844, September 2024.

[25] Kaspar P. Locher. Mechanistic diversity in ATP-binding cassette (ABC) transporters. *Nature Structural Molecular Biology*, 23(6):487–493, June 2016.

[26] Anna Kocyła, Józef Ba Tran, and Artur Krężel. Galvanization of protein–protein interactions in a dynamic zinc interactome. *Trends in Biochemical Sciences*, 46(1):64–79, January 2021.

[27] James W. Torrance, Malcolm W. MacArthur, and Janet M. Thornton. Evolution of binding sites for zinc and calcium ions playing structural roles. *Proteins: Structure, Function, and Bioinformatics*, 71(2):813–830, 2008.

[28] Setare Mostajabi Sarhangi and Dmitry V. Matyushov. Driving forces of protein diffusion. *The Journal of Physical Chemistry Letters*, 11(23):10137–10143, December 2020.

[29] Paul E. Smith and Wilfred F. van Gunsteren. Translational and rotational diffusion of proteins. *Journal of Molecular Biology*, 236(2):629–636, February 1994.

[30] Gideon Schreiber. *Protein–Protein Interactions: Kinetics*, page 2098–2101. Springer, Berlin, Heidelberg, 2013.

[31] Chongle Pan. Measuring dissociation rate constants of protein complexes through subunit exchange: Experimental design and theoretical modeling. *PLOS ONE*, 6(12):e28827, December 2011.

[32] Megan J Robinson and Melanie H Cobb. Mitogen-activated protein kinase pathways. *Current Opinion in Cell Biology*, 9(2):180–186, April 1997.

[33] Gavin Yong Quan Ng, Zachary Wai-Loon Loh, David Y. Fann, Karthik Mallilankaraman, Thiruma V. Arumugam, and M. Prakash Hande. Role of mitogen-activated protein (map) kinase pathways in metabolic diseases. *Genome Integrity*, 15, 2024.

[34] J. Nathaniel Diehl, Priya S. Hibshman, Irem Ozkan-Dagliyan, Craig M. Goodwin, Sarah V. Howard, Adrienne D. Cox, and Channing J. Der. *Chapter Four - Targeting the ERK mitogen-activated protein kinase cascade for the treatment of KRAS-mutant pancreatic cancer*, volume 153 of *RAS: Past, Present, and Future*, page 101–130. Academic Press, January 2022.

[35] Nadia Barbero, Lucia Napione, Sonja Visentin, Maria Alvaro, Andrea Veglio, Federico Bussolino, and Guido Viscardi. A transient kinetic study between signaling proteins: the case of the MEK–ERK interaction. *Chemical Science*, 2(9):1804–1809, 2011.

[36] Joshua R. Wayment and Joel M. Harris. Biotin–Avidin binding kinetics measured by single-molecule imaging. *Analytical Chemistry*, 81(1):336–342, January 2009.

[37] Matthias Blazek, Tomas Silva Santisteban, Roland Zengerle, and Matthias Meier. Analysis of fast protein phosphorylation kinetics in single cells on a microfluidic chip. *Lab on a Chip*, 15(3):726–734, January 2015.

[38] Harvey Lodish, Arnold Berk, Chris A. Kaiser, Monty Kreiger, Anthony Bretscher, Hidde Ploegh, Angelika Amon, and Kelsey C. Martin. Protein binding and enzyme catalysis. In *Molecular cell biology*, page 89–97. W H Freeman, New York, NY, USA, 8th edition, 2016.

[39] J. C. Kendrew, G. Bodo, H. M. Dintzis, R. G. Parrish, H. Wyckoff, and D. C. Phillips. A three-dimensional model of the myoglobin molecule obtained by X-Ray analysis. *Nature*, 181(4610):662–666, March 1958.

[40] M. F. Perutz, M. G. Rossmann, Ann F. Cullis, Hilary Muirhead, Georg Will, and A. C. T. North. Structure of hæmoglobin: A three-dimensional fourier synthesis at 5.5-Å. resolution, obtained by X-Ray analysis. *Nature*, 185(4711):416–422, February 1960.

[41] Alan R. Fersht. From the first protein structures to our current knowledge of protein folding: delights and scepticisms. *Nature Reviews Molecular Cell Biology*, 9(8):650–654, August 2008.

[42] C. B. Anfinsen, E. Haber, M. Sela, and F. H. White. The kinetics of formation of native ribonuclease during oxidation of the reduced polypeptide chain. *Proceedings of the National Academy of Sciences of the United States of America*, 47(9):1309–1314, September 1961.

[43] C. B. Anfinsen. Principles that govern the folding of protein chains. *Science*, 181(4096):223–230, July 1973.

[44] Jorge A. Vila. Metamorphic proteins in light of Anfinsen's dogma. *The Journal of Physical Chemistry Letters*, 11(13):4998–4999, July 2020.

[45] Prakash Kulkarni, Vitor B. P. Leite, Susmita Roy, Supriyo Bhattacharyya, Atish Mohanty, Srisairam Achuthan, Divyoj Singh, Rajeswari Appadurai, Govindan Rangarajan, Keith Weninger, John Orban, Anand Srivastava, Mohit Kumar Jolly, Jose N. Onuchic, Vladimir N. Uversky, and Ravi Salgia. Intrinsically disordered proteins: Ensembles at the limits of Anfinsen's dogma. *Biophysics Reviews*, 3(1):011306, March 2022.

[46] Francis Crick. Central dogma of molecular biology. *Nature*, 227(5258):561–563, August 1970.

[47] Kris Pauwels, Inge Van Molle, Jan Tommassen, and Patrick Van Gelder. Chaperoning Anfinsen: the steric foldases. *Molecular Microbiology*, 64(4):917–922, 2007.

[48] Ursula Jakob, Richard Kriwacki, and Vladimir N. Uversky. Conditionally and transiently disordered proteins: Awakening cryptic disorder to regulate protein function. *Chemical Reviews*, 114(13):6779–6805, 2014. PMID: 24502763.

[49] Arunachalam Vinayagam, Travis E. Gibson, Ho-Joon Lee, Bahar Yilmazel, Charles Roesel, Yanhui Hu, Young Kwon, Amitabh Sharma, Yang-Yu Liu, Norbert Perrimon, and Albert-László Barabási. Controllability analysis of the directed human protein interaction network identifies disease genes and drug targets. *Proceedings of the National Academy of Sciences*, 113(18):4976–4981, May 2016.

[50] Ian W. Taylor and Jeffrey L. Wrana. Protein interaction networks in medicine and disease. *PROTEOMICS*, 12(10):1706–1716, May 2012.

[51] Quan Zhong, Nicolas Simonis, Qian-Ru Li, Benoit Charloteaux, Fabien Heuze, Niels Klitgord, Stanley Tam, Haiyuan Yu, Kavitha Venkatesan, Danny Mou, Venus Swearingen, Muhammed A Yildirim, Han Yan, Amélie Dricot, David Szeto, Chenwei Lin, Tong Hao, Changyu Fan, Stuart Milstein, Denis Dupuy, Robert Brasseur, David E Hill, Michael E Cusick, and Marc Vidal. Edgetic perturbation models of human inherited disorders. *Molecular Systems Biology*, 5(1):321, January 2009.

[52] Yongsheng Li, Brandon Burgman, Ishaani S Khatri, Sairahul R Pentaparthi, Zhe Su, Daniel J McGrail, Yang Li, Erxi Wu, S Gail Eckhardt, Nidhi Sahni, and S Stephen Yi. e-MutPath: computational modeling reveals the functional landscape of genetic mutations rewiring interactome networks. *Nucleic Acids Research*, 49(1):e2–e2, January 2021.

[53] Minkyu Kim, Jisoo Park, Mehdi Bouhaddou, Kyumin Kim, Ajda Rojc, Maya Modak, Margaret Soucheray, Michael J. McGregor, Patrick O'Leary, Denise Wolf, Erica Stevenson, Tzeh Keong Foo, Dominique Mitchell, Kari A. Herrington, Denise P. Muñoz, Beril Tutuncuoglu, Kuei-Ho Chen, Fan Zheng, Jason F. Kreisberg, Morgan E. Diolaiti, John D. Gordan, Jean-Philippe Coppé, Danielle L. Swaney, Bing Xia, Laura Van 'T Veer, Alan Ashworth, Trey Ideker, and Nevan J. Krogan. A protein interaction landscape of breast cancer. *Science*, 374(6563):eabf3066, October 2021.

[54] Cendrine Tourette, Biao Li, Russell Bell, Shannon O'Hare, Linda S. Kaltenbach, Sean D. Mooney, and Robert E. Hughes. A large scale huntingtin protein interaction network implicates Rho GTPase signaling pathways in huntington disease. *Journal of Biological Chemistry*, 289(10):6709–6726, March 2014.

[55] Lloyd Mabonga and Abidemi Paul Kappo. Protein-protein interaction modulators: advances, successes and remaining challenges. *Biophysical Reviews*, 11(4):559–581, August 2019.

[56] Duncan E. Scott, Andrew R. Bayly, Chris Abell, and John Skidmore. Small molecules, big targets: drug discovery faces the protein–protein interaction challenge. *Nature Reviews Drug Discovery*, 15(8):533–550, August 2016.

[57] Stanley Fields and Ok-kyu Song. A novel genetic system to detect protein–protein interactions. *Nature*, 340(6230):245–246, July 1989. Publisher: Nature Publishing Group.

[58] Jamie Snider, Saranya Kittanakom, Dunja Damjanovic, Jasna Curak, Victoria Wong, and Igor Stagljar. Detecting interactions with membrane proteins using a membrane two-hybrid assay in yeast. *Nature Protocols*, 5(7):1281–1293, July 2010.

[59] Julia Petschnigg, Bella Groisman, Max Kotlyar, Mikko Taipale, Yong Zheng, Christoph F. Kurat, Azin Sayad, J. Rafael Sierra, Mojca Mattiazzi Usaj, Jamie Snider, Alex Nachman, Irina Krykbaeva, Ming-Sound Tsao, Jason Moffat, Tony Pawson, Susan Lindquist, Igor Jurisica, and Igor Stagljar. The mammalian-membrane two-hybrid assay (MaMTH) for probing membrane-protein interactions in human cells. *Nature Methods*, 11(5):585–592, May 2014. Publisher: Nature Publishing Group.

[60] Tom K. Kerppola. Bimolecular fluorescence complementation (BiFC) analysis as a probe of protein interactions in living cells. *Annual Review of Biophysics*, 37:465–487, 2008.

[61] Irma Lemmens, Sam Lievens, and Jan Tavernier. MAPPIT, a mammalian two-hybrid method for in-cell detection of protein-protein interactions. *Methods in Molecular Biology (Clifton, N.J.)*, 1278:447–455, 2015.

[62] Sam Lievens, Sarah Gerlo, Irma Lemmens, Dries J. H. De Clercq, Martijn D. P. Risseeuw, Nele Vanderroost, Anne-Sophie De Smet, Elien Ruyssinck, Eric Chevet, Serge Van Calenbergh, and Jan Tavernier. Kinase Substrate Sensor (KISS), a mammalian in situ protein interaction sensor. *Molecular & cellular proteomics: MCP*, 13(12):3332–3342, December 2014.

[63] Ian M. Evans and Ketevan Paliashvili. Co-immunoprecipitation Assays. In Lorna R. Fiedler and Caroline Pellet-Many, editors, *VEGF Signaling: Methods and Protocols*, pages 125–132. Springer US, New York, NY, 2022.

[64] Prashant Kaushal, Manisha R. Ummadi, Gwendolyn M. Jang, Yennifer Delgado, Sara K. Makanani, Kareem Alba, Declan M. Winters, Sophie F. Blanc, Jiewei Xu, Benjamin Polacco, Yuan Zhou, Erica Stevenson, Manon Eckhardt, Lorena Zuliani-Alvarez, Robyn Kaake, Danielle L. Swaney, Nevan J. Krogan, and Mehdi Bouhaddou. Protocol for mapping differential protein-protein interaction networks using affinity purification-mass spectrometry. *STAR Protocols*, 5(4):103286, November 2024.

[65] Kyle J. Roux, Dae In Kim, Brian Burke, and Danielle G. May. BioID: A Screen for Protein-Protein Interactions. *Current Protocols in Protein Science*, 91(1):19.23.1–19.23.15, 2018. _eprint: https://currentprotocols.onlinelibrary.wiley.com/doi/pdf/10.1002/cpps.51.

[66] Linlin Ma, Fan Yang, and Jie Zheng. Application of fluorescence resonance energy transfer in protein studies. *Journal of Molecular Structure*, 1077:87–100, December 2014.

[67] Chris Soon Heng Tan, Ka Diam Go, Xavier Bisteau, Lingyun Dai, Chern Han Yong, Nayana Prabhu, Mert Burak Ozturk, Yan Ting Lim, Lekshmy Sreekumar, Johan Lengqvist, Vinay Tergaonkar, Philipp Kaldis, Radoslaw M. Sobota, and Pär Nordlund. Thermal proximity coaggregation for system-wide profiling of protein complex dynamics in cells. *Science*, 359(6380):1170–1177, March 2018.

[68] R. M. Kini and H. J. Evans. Prediction of potential protein-protein interaction sites from amino acid sequence. Identification of a fibrin polymerization site. *FEBS letters*, 385(1–2):81–86, April 1996.

[69] F. Pazos, M. Helmer-Citterich, G. Ausiello, and A. Valencia. Correlated mutations contain information about protein-protein interaction. *Journal of Molecular Biology*, 271(4):511–523, August 1997.

[70] Susan Jones and Janet M Thornton. Prediction of protein-protein interaction sites using patch analysis. *Journal of Molecular Biology*, 272(1):133–143, September 1997.

[71] Hani Neuvirth, Ran Raz, and Gideon Schreiber. ProMate: A Structure Based Prediction Program to Identify the Location of Protein–Protein Binding Sites. *Journal of Molecular Biology*, 338(1):181–199, April 2004.

[72] Michael S. Chapman. Sequence similarity scores and the inference of structure—function relationships. *Bioinformatics*, 10(2):111–119, 1994.

[73] Edward M. Marcotte, Matteo Pellegrini, Ho-Leung Ng, Danny W. Rice, Todd O. Yeates, and David Eisenberg. Detecting Protein Function and Protein-Protein Interactions from Genome Sequences. *Science*, 285(5428):751–753, July 1999.

[74] Shailesh V Date and Edward M Marcotte. Discovery of uncharacterized cellular systems by genome-wide analysis of functional linkages. *Nature Biotechnology*, 21(9):1055–1062, September 2003.

[75] S. Altschul. Gapped BLAST and PSI-BLAST: a new generation of protein database search programs. *Nucleic Acids Research*, 25(17):3389–3402, September 1997.

[76] M. Kanehisa. KEGG: Kyoto Encyclopedia of Genes and Genomes. *Nucleic Acids Research*, 28(1):27–30, January 2000.

[77] Sylvain Pitre, Frank Dehne, Albert Chan, Jim Cheetham, Alex Duong, Andrew Emili, Marinella Gebbia, Jack Greenblatt, Mathew Jessulat, Nevan Krogan, Xuemei Luo, and Ashkan Golshani. PIPE: a protein-protein interaction prediction engine based on the re-occurring short polypeptide sequences between known interacting protein pairs. *BMC Bioinformatics*, 7(1):365, July 2006.

[78] S. Pitre, C. North, M. Alamgir, M. Jessulat, A. Chan, X. Luo, J. R. Green, M. Dumontier, F. Dehne, and A. Golshani. Global investigation of protein–protein interactions in yeast Saccharomyces cerevisiae using re-occurring short polypeptide sequences. *Nucleic Acids Research*, 36(13):4286–4294, August 2008.

[79] Andrew Schoenrock. *PIPE3 : a massively parallel protein-protein interatction prediction engine*. Master of Computer Science, Carleton University, Ottawa, Ontario, 2010.

[80] Andrew Schoenrock, Frank Dehne, James R. Green, Ashkan Golshani, and Sylvain Pitre. MP-PIPE: a massively parallel protein-protein interaction prediction engine. In *Proceedings of the international conference on Supercomputing*, ICS '11, pages 327–337, New York, NY, USA, May 2011. Association for Computing Machinery.

[81] K. Dick, Bahram Samanfar, Bradley Barnes, E. Cober, B. Mimee, Le Hoa Tan, S. Molnar, K. Biggar, A. Golshani, F. Dehne, and J. Green. PIPE4: Fast PPI Predictor for Comprehensive Inter- and Cross-Species Interactomes. *Scientific Reports*, 2020.

[82] Yiwei Li and Lucian Ilie. SPRINT: ultrafast protein-protein interaction prediction of the entire human interactome. *BMC Bioinformatics*, 2017.

[83] Koki Tsukamoto, Tatsuya Yoshikawa, Kiyonobu Yokota, Yuichiro Hourai, and Kazuhiko Fukui. The development of an affinity evaluation and prediction system by using protein–protein docking simulations and parameter tuning. *Advances and Applications in Bioinformatics and Chemistry*, 2:1–15, January 2009. Publisher: Dove Press.

[84] Yuri Matsuzaki, Yusuke Matsuzaki, Toshiyuki Sato, and Yutaka Akiyama. In silico screening of protein–protein interactions with all-to-all rigid docking and clustering: an application to pathway analysis. *Journal of Bioinformatics and Computational Biology*, 07(06):991–1012, December 2009. Publisher: World Scientific Publishing Co.

[85] Rong Chen, Li Li, and Zhiping Weng. ZDOCK: an initial-stage protein-docking algorithm. *Proteins*, 52(1):80–87, July 2003.

[86] Masahito Ohue, Yuri Matsuzaki, Nobuyuki Uchikoga, Takashi Ishida, and Yutaka Akiyama. MEGADOCK: An All-to-All Protein-Protein Interaction Prediction System Using Tertiary Structure Data. *Protein & Peptide Letters*, 21(8):766–778, July 2013.

[87] Joel R. Bock and David A. Gough. Predicting protein–protein interactions from primary structure. *Bioinformatics*, 17(5):455–460, May 2001.

[88] Alexey Ya. Chervonenkis. Early History of Support Vector Machines. In Bernhard Schölkopf, Zhiyuan Luo, and Vladimir Vovk, editors, *Empirical Inference*, pages 13–20. Springer Berlin Heidelberg, Berlin, Heidelberg, 2013.

[89] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, September 1995.

[90] William S Noble. What is a support vector machine? *Nature Biotechnology*, 24(12):1565–1567, December 2006.

[91] B. Scholkopf. The Kernel Trick for Distances. 2000.

[92] Shawn M. Gomez, William Stafford Noble, and Andrey Rzhetsky. Learning to predict protein-protein interactions from protein sequences. *Bioinformatics (Oxford, England)*, 19(15):1875–1881, October 2003.

[93] Shinsuke Dohkan, Asako Koike, and Toshihisa Takagi. Support Vector Machines for Predicting Protein-Protein Interactions. *Genome Informatics*, 14:502–503, 2003.

[94] Yanzhi Guo, Lezheng Yu, Zhining Wen, and Menglong Li. Using support vector machine combined with auto covariance to predict protein–protein interactions from protein sequences. *Nucleic Acids Research*, 36(9):3025–3030, May 2008.

[95] Lun Hu and Keith C. C. Chan. Discovering Variable-Length Patterns in Protein Sequences for Protein-Protein Interaction Prediction. *IEEE Transactions on NanoBioscience*, 14(4):409–416, June 2015.

[96] Shawn Martin, Diana Roe, and Jean-Loup Faulon. Predicting protein–protein interactions using signature products. *Bioinformatics*, 21(2):218–226, January 2005.

[97] A. Ben-Hur and W. S. Noble. Kernel methods for predicting protein-protein interactions. *Bioinformatics*, 21(Suppl 1):i38–i46, June 2005.

[98] Zhu-Hong You, Jian-Zhong Yu, Lin Zhu, Shuai Li, and Zhen-Kun Wen. A MapReduce based parallel SVM for large-scale predicting protein–protein interactions. *Neurocomputing*, 145:37–43, December 2014.

[99] Jeffrey Dean and Sanjay Ghemawat. MapReduce: simplified data processing on large clusters. *Commun. ACM*, 51(1):107–113, January 2008.

[100] Yanjun Qi, Judith Klein-Seetharaman, and Ziv Bar-Joseph. A mixture of feature experts approach for protein-protein interaction prediction. *BMC Bioinformatics*, 8(10):S6, December 2007.

[101] Eric W. Bell, Jacob H. Schwartz, Lydia Freddolino, and Yang Zhang. PEPPI: Whole-proteome Protein-protein Interaction Prediction through Structure and Sequence Similarity, Functional Association, and Machine Learning. *Journal of Molecular Biology*, 434(11):167530, June 2022.

[102] Leo Breiman. Random Forests. *Machine Learning*, 45(1):5–32, October 2001.

[103] Xiuquan Du, Jiaxing Cheng, Tingting Zheng, Zheng Duan, and Fulan Qian. A Novel Feature Extraction Scheme with Ensemble Coding for Protein–Protein Interaction Prediction. *International Journal of Molecular Sciences*, 15(7):12731–12749, July 2014.

[104] Zhu-Hong You, Keith C. C. Chan, and Pengwei Hu. Predicting Protein-Protein Interactions from Primary Protein Sequences Using a Novel Multi-Scale Local Feature Representation Scheme and the Random Forest. *PLOS ONE*, 10(5):e0125811, May 2015.

[105] Yijie Ding, Jijun Tang, and Fei Guo. Predicting protein-protein interactions via multivariate mutual information of protein sequences. *BMC Bioinformatics*, 17(1):398, September 2016.

[106] Bin Yu, Cheng Chen, Hongyan Zhou, Bingqiang Liu, and Qin Ma. GTB-PPI: Predict Protein–Protein Interactions Based on L1-Regularized Logistic Regression and Gradient Tree Boosting. *Genomics, Proteomics & Bioinformatics*, 18(5):582–592, October 2020.

[107] Hongli Gao, Cheng Chen, Shuangyi Li, Congjing Wang, Weifeng Zhou, and Bin Yu. Prediction of protein-protein interactions based on ensemble residual convolutional neural network. *Computers in Biology and Medicine*, 152:106471, January 2023.

[108] Mario R. Guarracino and Adriano Nebbia. Predicting Protein-Protein Interactions with K-Nearest Neighbors Classification Algorithm. In David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Madhu Sudan, Demetri Terzopoulos, Doug Tygar, Moshe Y. Vardi, Gerhard Weikum, Francesco Masulli, Leif E. Peterson, and Roberto Tagliaferri, editors, *Computational Intelligence Methods for Bioinformatics and Biostatistics*, volume 6160, pages 139–150. Springer Berlin Heidelberg, Berlin, Heidelberg, 2010. Series Title: Lecture Notes in Computer Science.

[109] E. Fix and J. Hodges. Discriminatory Analysis - Nonparametric Discrimination: Small Sample Performance. August 1952.

[110] Friedrich Riesz. Untersuchungen über Systeme integrierbarer Funktionen. *Mathematische Annalen*, 69(4):449–497, December 1910.

[111] David D. Lewis. Naive (Bayes) at forty: The independence assumption in information retrieval. In Claire Nédellec and Céline Rouveirol, editors, *Machine Learning: ECML-98*, pages 4–15, Berlin, Heidelberg, 1998. Springer.

[112] Chia Hsin Liu, Ker-Chau Li, and Shinsheng Yuan. Human protein–protein interaction prediction by a novel sequence-based co-evolution method: co-evolutionary divergence. *Bioinformatics*, 29(1):92–98, January 2013.

[113] Piero Fariselli, Florencio Pazos, Alfonso Valencia, and Rita Casadio. Prediction of protein–protein interaction sites in heterocomplexes with neural networks. *European Journal of Biochemistry*, 269(5):1356–1361, March 2002.

[114] Frank Rosenblatt. The perceptron a perceiving and recognizing automaton. *tech. rep., Technical Report 85-460-1*, 1957.

[115] J J Hopfield. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8):2554–2558, April 1982. Publisher: Proceedings of the National Academy of Sciences.

[116] Paul J. Werbos. Applications of advances in nonlinear sensitivity analysis. In R. F. Drenick and F. Kozin, editors, *System Modeling and Optimization*, pages 762–770, Berlin, Heidelberg, 1982. Springer.

[117] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, October 1986. Publisher: Nature Publishing Group.

[118] Kunihiko Fukushima. Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4):193–202, April 1980.

[119] Jeffrey L. Elman. Finding Structure in Time. *Cognitive Science*, 14(2):179–211, 1990. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1207/s15516709cog1402_1.

[120] John D. Kelleher. A Brief History of Deep Learning. In *Deep Learning*, pages 101–159. MIT Press, Cambridge, Massachusetts, 2019.

[121] Igor N. Aizenberg, Naum N. Aizenberg, and Joos Vandewalle. *Multi-Valued and Universal Binary Neurons*. Springer US, Boston, MA, 2000.

[122] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.

[123] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 6000–6010, Red Hook, NY, USA, December 2017. Curran Associates Inc.

[124] Tanlin Sun, Bo Zhou, Luhua Lai, and Jianfeng Pei. Sequence-based prediction of protein protein interaction using a deep-learning algorithm. *BMC Bioinformatics*, 18(1):277, May 2017.

[125] Dor Bank, Noam Koenigstein, and Raja Giryes. Autoencoders. In Lior Rokach, Oded Maimon, and Erez Shmueli, editors, *Machine Learning for Data Science Handbook: Data Mining and Knowledge Discovery Handbook*, pages 353–374. Springer International Publishing, Cham, 2023.

[126] Hua Wang, Heng Huang, Chris Ding, and Feiping Nie. Predicting Protein–Protein Interactions from Multimodal Biological Data Sources via Nonnegative Matrix Tri-Factorization. *Journal of Computational Biology*, 20(4):344–358, April 2013.

[127] K. Fukushima. Cognitron: a self-organizing multilayered neural network. *Biological Cybernetics*, 20(3-4):121–136, November 1975.

[128] D. Hubel and T. Wiesel. Functional architecture of macaque monkey visual cortex. 1977.

[129] D. H. Hubel and T. N. Wiesel. Receptive Fields and Functional Architecture in two Nonstirate Visual Areas (18 and 19) of the Cat. *Journal of Neurophysiology*, 28:229–289, March 1965.

[130] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation Applied to Handwritten Zip Code Recognition. *Neural Computation*, 1(4):541–551, December 1989.

[131] Yann LeCun. Generalization and Network Design Strategies. Technical Report CRG-TR-89-4, University of Toronto, Toronto, ON, June 1989.

[132] Serkan Kiranyaz, Onur Avci, Osama Abdeljaber, Turker Ince, Moncef Gabbouj, and Daniel J. Inman. 1D convolutional neural networks and applications: A survey. *Mechanical Systems and Signal Processing*, 151:107398, April 2021.

[133] Somaye Hashemifar, Behnam Neyshabur, Aly A Khan, and Jinbo Xu. Predicting protein–protein interactions through sequence-based deep learning. *Bioinformatics*, 34(17):i802–i810, September 2018.

[134] Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, November 1997.

[135] Taku Kudo. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates, April 2018.

[136] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. In Eduardo Blanco and Wei Lu, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium, November 2018. Association for Computational Linguistics.

[137] Muhao Chen, Chelsea J T Ju, Guangyu Zhou, Xuelu Chen, Tianran Zhang, Kai-Wei Chang, Carlo Zaniolo, and Wei Wang. Multifaceted protein–protein interaction prediction based on Siamese residual RCNN. *Bioinformatics*, 35(14):i305–i314, July 2019.

[138] Florian Richoux, Charlène Servantie, Cynthia Borès, and Stéphane Téletchéa. Comparing two deep learning sequence-based models for protein-protein interaction prediction. (arXiv:1901.06268), January 2019. number: arXiv:1901.06268 arXiv:1901.06268 [cs, q-bio, stat].

[139] Claude E. Shannon. Prediction and Entropy of Printed English. In *Claude E. Shannon: Collected Papers*, pages 194–208. IEEE, 1993.

[140] T. Cover and R. King. A convergent gambling estimate of the entropy of English. *IEEE Transactions on Information Theory*, 24(4):413–421, July 1978.

[141] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A Neural Probabilistic Language Model. *Journal of Machine Learning Research*, 3(Feb):1137–1155, 2003.

[142] Stephen Merity, Nitish Shirish Keskar, and Richard Socher. An Analysis of Neural Language Modeling at Multiple Scales, March 2018. arXiv:1803.08240 [cs].

[143] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv:1810.04805 [cs]*, May 2019. arXiv: 1810.04805.

[144] Ethan C. Alley, Grigory Khimulya, Surojit Biswas, Mohammed AlQuraishi, and George M. Church. Unified rational protein engineering with sequence-based deep representation learning. *Nature Methods*, 16(12):1315–1322, December 2019. Publisher: Nature Publishing Group.

[145] Tristan Bepler and Bonnie Berger. Learning protein sequence embeddings using information from structure, October 2019. arXiv:1902.08661 [cs, q-bio, stat].

[146] Tristan Bepler and Bonnie Berger. Learning the protein language: Evolution, structure, and function. *Cell Systems*, 12(6):654–669.e3, June 2021. Publisher: Elsevier.

[147] Nadav Brandes, Dan Ofer, Yam Peleg, Nadav Rappoport, and Michal Linial. ProteinBERT: a universal deep-learning model of protein sequence and function. *Bioinformatics*, 38(8):2102–2110, April 2022.

[148] Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rehawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, Debsindhu Bhowmik, and Burkhard Rost. ProtTrans: Toward Understanding the Language of Life Through Self-Supervised Learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):7112–7127, October 2022.

[149] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, March 2023. Publisher: American Association for the Advancement of Science.

[150] Ahmed Elnaggar, Hazem Essam, Wafaa Salah-Eldin, Walid Moustafa, Mohamed Elkerdawy, Charlotte Rochereau, and Burkhard Rost. Ankh □: Optimized Protein Language Model Unlocks General-Purpose Modelling. January 2023. Repository: Bioinformatics.

[151] Tuoyu Liu, Han Gao, Xiaopu Ren, Guoshun Xu, Bo Liu, Ningfeng Wu, Huiying Luo, Yuan Wang, Tao Tu, Bin Yao, Feifei Guan, Yue Teng, Huoqing Huang, and Jian Tian. Protein-protein interaction and site prediction using transfer learning. *Briefings in Bioinformatics*, 24(6):bbad376, September 2023.

[152] Liuzhenghao Lv, Zongying Lin, Hao Li, Yuyang Liu, Jiaxi Cui, Calvin Yu-Chian Chen, Li Yuan, and Yonghong Tian. Prollama: A protein language model for multi-task protein language processing. 2024.

[153] Daria Frolova, Marina A. Pak, Anna Litvin, Ilya Sharov, Dmitry N. Ivankov, and Ivan Oseledets. Mulan: Multimodal protein language model for sequence and structure encoding. page 2024.05.30.596565, June 2024.

[154] Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J. Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q. Tran, Jonathan Deaton, Marius Wiggert, Rohil Badkundri, Irhum Shafkat, Jun Gong, Alexander Derry, Raul S. Molina, Neil Thomas, Yousuf A. Khan, Chetan Mishra, Carolyn Kim, Liam J. Bartie, Matthew Nemeth, Patrick D. Hsu, Tom Sercu, Salvatore Candido, and Alexander Rives. Simulating 500 million years of evolution with a language model. *Science*, 387(6736):850–858, February 2025. Publisher: American Association for the Advancement of Science.

[155] Samuel Sledzieski, Rohit Singh, Lenore Cowen, and Bonnie Berger. D-SCRIPT translates genome to phenome with sequence-based, structure-aware, genome-scale predictions of protein-protein interactions. *Cell Systems*, 12(10):969–982.e6, October 2021. Publisher: Elsevier.

[156] Yanlei Kang, Arne Elofsson, Yunliang Jiang, Weihong Huang, Minzhe Yu, and Zhong Li. AFTGAN: prediction of multi-type PPI based on attention free transformer and graph attention network. *Bioinformatics*, 39(2):btad052, February 2023.

[157] Kewei Zhou, Chenping Lei, Jingyan Zheng, Yan Huang, and Ziding Zhang. Pre-trained protein language model sheds new light on the prediction of Arabidopsis protein–protein interactions. *Plant Methods*, 19(1):141, December 2023.

[158] Young Su Ko, Jonathan Parkinson, Cong Liu, and Wei Wang. TUnA: an uncertainty-aware transformer model for sequence-based protein–protein interaction prediction. *Briefings in Bioinformatics*, 25(5):bbae359, September 2024.

[159] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, November 2005.

[160] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, September 2023.

[161] Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th international joint conference on Artificial intelligence - Volume 2*, IJCAI'95, pages 1137–1143, San Francisco, CA, USA, August 1995. Morgan Kaufmann Publishers Inc.

[162] M. Stone. Cross-Validatory Choice and Assessment of Statistical Predictions. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(2):111–133, January 1974.

[163] Joseph Szymborski and Amin Emad. A flaw in using pre-trained pLLMs in protein-protein interaction inference models, April 2025. Pages: 2025.04.21.649858 Section: New Results.

[164] Stephen Merity, Nitish Shirish Keskar, and Richard Socher. Regularizing and optimizing LSTM language models. *arXiv:1708.02182 [cs]*, August 2017. arXiv: 1708.02182.

[165] Gang Fang, Nitin Bhardwaj, Rebecca Robilotto, and Mark B. Gerstein. Getting Started in Gene Orthology and Functional Analysis. *PLOS Computational Biology*, 6(3):1–8, March 2010. Publisher: Public Library of Science.

[166] Roded Sharan, Silpa Suthram, Ryan M. Kelley, Tanja Kuhn, Scott McCuine, Peter Uetz, Taylor Sittler, Richard M. Karp, and Trey Ideker. Conserved patterns of protein interaction in multiple species. *Proceedings of the National Academy of Sciences*, 102(6):1974–1979, February 2005.

[167] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A Simple Framework for Contrastive Learning of Visual Representations, June 2020. Number: arXiv:2002.05709 arXiv:2002.05709 [cs, stat].

[168] Pierre Sermanet, Corey Lynch, Yevgen Chebotar, Jasmine Hsu, Eric Jang, Stefan Schaal, and Sergey Levine. Time-Contrastive Networks: Self-Supervised Learning from Video, March 2018. Number: arXiv:1704.06888 arXiv:1704.06888 [cs].

[169] Adrian de Wynter, Xun Wang, Alex Sokolov, Qilong Gu, and Si-Qing Chen. An Evaluation on Large Language Model Outputs: Discourse and Memorization. *Natural Language Processing Journal*, 4:100024, September 2023. arXiv:2304.08637 [cs].

[170] Li Wan, Matthew Zeiler, Sixin Zhang, Yann Le Cun, and Rob Fergus. Regularization of neural networks using DropConnect. In Sanjoy Dasgupta and David McAllester, editors, *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, page 1058–1066, Atlanta, Georgia, USA, June 2013. PMLR.

[171] Nitish Srivastava, Geoffrey E. Hinton, A. Krizhevsky, Ilya Sutskever, and R. Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. 15:1929–1958, 2014.

[172] Justin Bayer, Christian Osendorfer, Daniela Korhammer, Nutan Chen, Sebastian Urban, and Patrick van der Smagt. On fast dropout and its applicability to recurrent networks. (arXiv:1311.0701), December 2013. arXiv:1311.0701 [stat].

[173] Luke Taylor and Geoff Nitschke. Improving deep learning with generic data augmentation. In *2018 IEEE Symposium Series on Computational Intelligence (SSCI)*, page 1542–1547, November 2018.

[174] Rui Sun, Lirong Wu, Haitao Lin, Yufei Huang, and Stan Z. Li. Enhancing protein predictive models via proteins data augmentation: A benchmark and new directions. (arXiv:2403.00875), March 2024. arXiv:2403.00875 [q-bio].

[175] Limin Fu, Beifang Niu, Zhengwei Zhu, Sitao Wu, and Weizhong Li. CD-HIT: accelerated for clustering the next-generation sequencing data. *Bioinformatics (Oxford, England)*, 28(23):3150–3152, December 2012.

[176] Chris Sander and Reinhard Schneider. Database of homology-derived protein structures and the structural meaning of sequence alignment. 9:56–68, 1991.

[177] Wouter G. Touw, Coos Baakman, Jon Black, Tim A. H. te Beek, E. Krieger, Robbie P. Joosten, and Gert Vriend. A series of PDB-related databanks for everyday needs. *Nucleic Acids Research*, 43(D1):D364–D368, January 2015.

[178] Burkhard Rost. Twilight zone of protein sequence alignments. *Protein Engineering, Design and Selection*, 12(2):85–94, February 1999.

[179] Mustafa Umit Oner, Yi-Chih Cheng, Hwee Kuan Lee, and Wing-Kin Sung. Training machine learning models on patient level data segregation is crucial in practical clinical applications. page 2020.04.23.20076406, April 2020.

[180] Sean Whalen, Jacob Schreiber, William S. Noble, and Katherine S. Pollard. Navigating the pitfalls of applying machine learning in genomics. *Nature Reviews Genetics*, 23(3):169–181, March 2022.

[181] Joseph Szymborski and Amin Emad. A flaw in using pre-trained pllms in protein-protein interaction inference models. page 2025.04.21.649858, April 2025.

[182] Stuart Geman, Elie Bienenstock, and René Doursat. Neural networks and the bias/variance dilemma. *Neural Computation*, 4(1):1–58, January 1992.

[183] Pierre Baldi. Deep learning over-parameterization: the shallow fallacy. page 7–12. PMLR, January 2024.

[184] Brady Neal. On the bias-variance tradeoff: Textbooks need an update, 2019.

[185] TOP500. June 2025 | top500, September 2025.

[186] Jack J. Dongarra, Piotr Luszczek, and Antoine Petitet. The linpack benchmark: past, present and future. *Concurrency and Computation: Practice and Experience*, 15(9):803–820, 2003.

[187] Sara Hooker. The hardware lottery. (arXiv:2009.06489), September 2020. arXiv:2009.06489 [cs].

[188] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena: A benchmark for efficient transformers. (arXiv:2011.04006), November 2020. arXiv:2011.04006 [cs].

[189] Itamar Zimerman and Lior Wolf. On the long range abilities of transformers. (arXiv:2311.16620), November 2023. arXiv:2311.16620 [cs].

[190] Jiayu Ding, Shuming Ma, Li Dong, Xingxing Zhang, Shaohan Huang, Wenhui Wang, Nanning Zheng, and Furu Wei. Longnet: Scaling transformers to 1,000,000,000 tokens. (arXiv:2307.02486), July 2023. arXiv:2307.02486 [cs].

[191] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. (arXiv:2205.14135), June 2022. arXiv:2205.14135 [cs].

[192] Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. (arXiv:2006.04768), June 2020. arXiv:2006.04768 [cs].

[193] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. (arXiv:2004.05150), December 2020. arXiv:2004.05150 [cs].

[194] Forrest N. Iandola, Albert E. Shaw, Ravi Krishna, and Kurt W. Keutzer. Squeezebert: What can computer vision teach nlp about efficient neural networks? (arXiv:2006.11316), June 2020. arXiv:2006.11316 [cs].

[195] Roland Moore-Colyer. Nvidia GeForce RTX 3090 release date, price, specs, restocks and more, July 2022.

[196] John Keefer. Nvidia launches 3, 000*titanvpcgpubasedonteslav*100, *September*2025.

[197] The UniProt Consortium, Alex Bateman, Maria-Jesus Martin, Sandra Orchard, Michele Magrane, Shadab Ahmad, Emanuele Alpi, Emily H Bowler-Barnett, Ramona Britto, Hema Bye-A-Jee, Austra Cukura, Paul Denny, Tunca Dogan, ThankGod Ebenezer, Jun Fan, Penelope Garmiri, Leonardo Jose Da Costa Gonzales, Emma Hatton-Ellis, Abdulrahman Hussein, Alexandr Ignatchenko, Giuseppe Insana, Rizwan Ishtiaq, Vishal Joshi, Dushyanth Jyothi, Swaathi Kandasaamy, Antonia Lock, Aurelien Luciani, Marija Lugaric, Jie Luo, Yvonne Lussi, Alistair MacDougall, Fabio Madeira, Mahdi Mahmoudy, Alok Mishra, Katie Moulang, Andrew Nightingale, Sangya Pundir, Guoying Qi, Shriya Raj, Pedro Raposo, Daniel L Rice, Rabie Saidi, Rafael Santos, Elena Speretta, James Stephenson, Prabhat Totoo, Edward Turner, Nidhi Tyagi, Preethi Vasudev, Kate Warner, Xavier Watkins, Rossana Zaru, Hermann Zellner, Alan J Bridge, Lucila Aimo, Ghislaine Argoud-Puy, Andrea H Auchincloss, Kristian B Axelsen, Parit Bansal, Delphine Baratin, Teresa M Batista Neto, Marie-Claude Blatter, Jerven T Bolleman, Emmanuel Boutet, Lionel Breuza, Blanca Cabrera Gil, Cristina Casals-Casas, Kamal Chikh Echioukh, Elisabeth Coudert, Beatrice Cuche, Edouard De Castro, Anne Estreicher, Maria L Famiglietti, Marc Feuermann, Elisabeth Gasteiger, Pascale Gaudet, Sebastien Gehant, Vivienne Gerritsen, Arnaud Gos, Nadine Gruaz, Chantal Hulo, Nevila Hyka-Nouspikel, Florence Jungo, Arnaud Kerhornou, Philippe Le Mercier, Damien Lieberherr, Patrick Masson, Anne Morgat, Venkatesh Muthukrishnan, Salvo Paesano, Ivo Pedruzzi, Sandrine Pilbout, Lucille Pourcel, Sylvain Poux, Monica Pozzato, Manuela Pruess, Nicole Redaschi, Catherine Rivoire, Christian J A Sigrist, Karin Sonesson, Shyamala Sundaram, Cathy H Wu, Cecilia N Arighi, Leslie Arminski, Chuming Chen, Yongxing Chen, Hongzhan Huang, Kati Laiho, Peter McGarvey, Darren A Natale, Karen Ross, C R Vinayaka, Qinghua Wang, Yuqi Wang, and Jian Zhang. UniProt: the Universal Protein Knowledgebase in 2023. *Nucleic Acids Research*, 51(D1):D523–D531, January 2023.

[198] Statistics Canada. Table 25-10-0060-01 Household energy consumption, Canada and provinces, March 2024.

[199] Canada Energy Regulator. Canada's Energy Future Data Appendices, 2021.

[200] Jennifer Gabrys. *Silicon Elephants: the transformative materiality of microchips*, page 20–44. University of Michigan Press, 2014.

[201] Sarker M Parvez, Farjana Jahan, Marie-Noel Brune, Julia F Gorman, Musarrat J Rahman, David Carpenter, Zahir Islam, Mahbubur Rahman, Nirupam Aich, Luke D Knibbs, and Peter D Sly. Health consequences of exposure to e-waste: an updated systematic review. *The Lancet Planetary Health*, 5(12):e905–e920, December 2021.

[202] Blake Alcott. Jevons' paradox. *Ecological Economics*, 54(1):9–21, July 2005.

[203] Salem Endris, Zemenu Tamir, and Abay Sisay. Medical laboratory waste generation rate, management practices and associated factors in Addis Ababa, Ethiopia. *PloS One*, 17(4):e0266888, 2022.

[204] Joana Alves, Fiona A. Sargison, Hanne Stawarz, Willow B. Fox, Samuel G. Huete, Amany Hassan, Brian McTeir, and Amy C. Pickering. A case report: insights into reducing plastic waste in a microbiology laboratory. *Access Microbiology*, 3(3), March 2021.

[205] Anjali Thakur, Tapasyapreeti Mukhopadhyay, and Ashok Kumar Ahirwar. Approaching sustainability in Laboratory Medicine. *Clinical Chemistry and Laboratory Medicine*, 62(9):1787–1794, August 2024.

[206] Mei Lan Tan, Chee Keat Ying, and Shahrul Bariyah Sahul Hamid. Plastic Pollution and Sustainable Managing of Single-Use Laboratory Plastic Waste. *Sustainability and Climate Change*, 15(1):6–16, February 2022. Publisher: Mary Ann Liebert, Inc., publishers.

[207] Christoph Becker. What is Just Sustainability Design? In *Insolvent: How to Reorient Computing for Just Sustainability*. MIT Press, Cambridge, MA, USA, June 2023.